

Arquitetura Limpa. O Guia do Artesão Para Estrutura e Design de Software PDF

Robert C. Martin



Mais Livros Grátis



Escaneie para baixar



Ouçá-o

Arquitetura Limpa. O Guia do Artesao Para Estrutura e Design de Software

Princípios Essenciais para Arquitetura e Design de Software Profissional.

Escrito por Bookey

[Saiba mais sobre o resumo de Arquitetura Limpa. O Guia do Artesao Para Estrutura e Design de Software](#)

[Ouvir Arquitetura Limpa. O Guia do Artesao Para Estrutura e Design de Software Audiolivro](#)

Mais Livros Grátis



Escaneie para baixar



[Ouça-o](#)

Sobre o livro

Em "Arquitetura Limpa. O Guia do Artesão Para Estrutura e Design de Software", o renomado especialista em software Robert C. "Uncle Bob" Martin expande as bases estabelecidas em seus aclamados livros, "O Criador Limpo" e "Código Limpo", para elevar o profissionalismo e a disciplina na arquitetura e design de aplicações. Ao contrário de outros textos que apenas apresentam opções sem orientação, Martin oferece recomendações claras sobre escolhas arquitetônicas essenciais e explica a importância delas para o sucesso. Ele aborda questões críticas sobre estruturas de alto nível para diversas aplicações, os princípios fundamentais da arquitetura e design de software, o verdadeiro papel de um arquiteto e estratégias para evitar armadilhas de design. Este livro é um recurso inestimável para arquitetos de software, analistas de sistemas, designers e programadores aspirantes que desejam aprimorar sua compreensão e impacto na área.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Sobre o autor

Robert C. Martin, frequentemente chamado de "Tio Bob", é uma figura de grande influência na comunidade de desenvolvimento de software, conhecido por suas contribuições para os princípios e práticas da engenharia de software. Com mais de cinquenta anos de experiência na área, ele é um dos pioneiros da metodologia Ágil e coautor do Manifesto Ágil, que moldou o desenvolvimento de software moderno. Martin também é autor de vários textos fundamentais, incluindo "Código Limpo" e "O Programador Limpo", que enfatizam a importância de escrever código que seja sustentável, eficiente e ético. Seu trabalho promove a ideia de Arquitetura Limpa. O Guia do Artesão Para Estrutura e Design de Software, defendendo uma abordagem estruturada para o design de software que melhora a adaptabilidade e a longevidade, o que o torna um palestrante, consultor e instrutor muito requisitado no âmbito da capacitação em software.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Ad



Escanear para baixar



Experimente o aplicativo Bookey para ler mais de 1000 resumos dos melhores livros do mundo

Desbloqueie **1000+** títulos, **80+** tópicos

Novos títulos adicionados toda semana

Product & Brand



Liderança & Colaboração



Gerenciamento de Tempo



Relacionamento & Comunicação

Estratégia de Negócios



Criatividade



Memórias



Conheça a Si Mesmo



Psicologia

Empreendedorismo



História Mundial



Comunicação entre Pais e Filhos



Autocuidado



Mind

Visões dos melhores livros do mundo

amento
pos

Os 7 Hábitos das
Pessoas Altamente
Eficazes



Mini Hábitos



Hábitos Atômicos



O Clube das 5
da Manhã



Como Fazer Amigos
e Influenciar
Pessoas



Com
Não



Teste gratuito com Bookey



Lista de conteúdo do resumo

Capítulo 1 : O Que É Design e Arquitetura?

Capítulo 2 : Uma História de Dois Valores

Capítulo 3 : Visão Geral dos Paradigmas

Capítulo 4 : Programação Estruturada

Capítulo 5 : Programação Orientada a Objetos

Chapter 6 : Functional Programming

Capítulo 7 : SRP: O Princípio da Responsabilidade Única

Chapter 8 : OCP: The Open-Closed Principle

Capítulo 9 : LSP: O Princípio da Substituição de Liskov

Chapter 10 : ISP: The Interface Segregation Principle

Chapter 11 : DIP: The Dependency Inversion Principle

Chapter 12 : Components

Chapter 13 : Component Cohesion

Chapter 14 : Component Coupling

Chapter 15 : What Is Architecture?

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Chapter 16 : Independence

Chapter 17 : Boundaries: Drawing Lines

Chapter 18 : Boundary Anatomy

Chapter 19 : Policy and Level

Chapter 20 : Business Rules

Chapter 21 : Screaming Architecture

Chapter 22 : The Clean Architecture

Chapter 23 : Presenters and Humble Objects

Chapter 24 : Partial Boundaries

Chapter 25 : Layers and Boundaries

Chapter 26 : The Main Component

Chapter 27 : Services: Great and Small

Chapter 28 : The Test Boundary

Chapter 29 : Clean Embedded Architecture

Chapter 30 : The Database Is a Detail

Chapter 31 : The Web Is a Detail

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Chapter 32 : Frameworks Are Details

Chapter 33 : Case Study: Video Sales

Chapter 34 : The Missing Chapter

Chapter 35 : Appendix A Architecture Archaeology

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Capítulo 1 Resumo : O Que É Design e Arquitetura?



Seção	Resumo
O QUE É DESIGN E ARQUITETURA?	Design e arquitetura são essencialmente a mesma coisa em software, trabalhando juntos como um continuum de decisões que moldam o sistema.
QUAL É O OBJETIVO?	O objetivo de um bom design de software é minimizar os recursos humanos necessários para construir e manter sistemas; um design de qualidade leva a um menor esforço contínuo.
ESTUDO DE CASO	Um estudo de caso mostra uma empresa com crescimento rápido de pessoal enfrentando queda na produtividade, indicando caos onde a qualidade sofre em prol da rapidez.
A ASSINATURA DE UMA BAGUNÇA	Uma bagunça no desenvolvimento surge de uma codificação rápida e desorganizada, levando a uma menor produtividade e aprisionando os desenvolvedores na gestão de problemas.
A VISÃO DOS EXECUTIVOS	Os executivos estão preocupados com o aumento da folha de pagamento sem funcionalidade correspondente, mas as ações específicas para resolver isso muitas vezes não são claras.
O QUE DEU ERRADO?	A fábula da Tartaruga e da Lebre serve como metáfora para a autoestima excessiva dos desenvolvedores e a crença de que podem corrigir erros mais tarde, perpetuando um ciclo de bagunça.
CONCLUSÃO	Os desenvolvedores devem assumir a responsabilidade pela qualidade do código, desafiando a noção de que bagunça é sinônimo de rapidez, uma vez que a qualidade é crucial para o sucesso a longo prazo.

O QUE É DESIGN E ARQUITETURA?

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Design e arquitetura muitas vezes criam confusão na indústria de software, mas, em essência, não há diferença entre eles. Tanto a arquitetura de alto nível quanto o design de baixo nível trabalham juntos como um todo, criando um continuum de decisões que moldam o sistema.

QUAL É O OBJETIVO?

O objetivo principal de um bom design de software é minimizar os recursos humanos necessários para construir e manter o sistema. Um design de qualidade permite um esforço contínuo baixo, enquanto um design ruim resulta em um aumento do esforço ao longo do tempo.

ESTUDO DE CASO

Um estudo de caso ilustrou os desafios enfrentados por uma empresa que experimentava um rápido crescimento na equipe de engenharia e, ainda assim, uma produtividade em declínio, medida por linhas de código e custos crescentes. Isso significa um processo caótico onde a qualidade é sacrificada pela velocidade, levando a um ciclo prejudicial.

A ASSINATURA DE UMA BAGUNÇA

Mais Livros Grátis



Escaneie para baixar



Ouça-o

A situação descreve uma bagunça no desenvolvimento de software, onde a codificação rápida e desorganizada leva a uma produtividade em declínio. Os desenvolvedores se tornam prisioneiros da gestão das consequências dessa bagunça, em vez de inovar ou adicionar novos recursos.

A VISÃO EXECUTIVA

Do ponto de vista executivo, o crescimento preocupante da folha de pagamento sem a funcionalidade correspondente das entregas leva a apelos por mudanças imediatas. No entanto, pode ser incerto quais passos específicos tomar.

O QUE DEU ERRADO?

A fábula de Esopo da Tartaruga e da Lebre serve como uma metáfora para a autoconfiança excessiva observada em desenvolvedores modernos. Muitos acreditam que podem corrigir erros posteriores, mergulhando em um ciclo de bagunça sem perceber os efeitos negativos na produtividade e na qualidade do design.

CONCLUSÃO

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Para enfrentar os declínios na produtividade, os desenvolvedores precisam abraçar a responsabilidade pela qualidade da codificação. A ideia equivocada de que a desorganização se traduz em velocidade deve ser contestada, já que a qualidade é fundamental para o sucesso a longo prazo em projetos de software. O livro tem como objetivo esclarecer e promover os princípios de uma arquitetura de software eficaz para construir sistemas com longevidade e mínimo esforço de manutenção.

Mais Livros Grátis



Escaneie para baixar



Ouçá-o

Exemplo

Ponto chave:Qualidade em vez de Velocidade

Exemplo:Imagine que você está codificando um novo recurso para um cliente com um prazo se aproximando. Na sua pressa para entregar, você opta por soluções rápidas em vez de um código limpo e fácil de manter. Inicialmente, o cliente fica satisfeito, e o recurso é lançado a tempo. No entanto, com o passar das semanas, você tem dificuldade em adicionar melhorias ou corrigir erros porque sua base de código se tornou confusa—um exemplo clássico de sacrificar qualidade por velocidade. Essa experiência ilustra o ponto principal: priorizar a qualidade no design é vital para a produtividade a longo prazo e reduz a necessidade de manutenção extensa, o que, em última análise, economiza tempo e recursos.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Pensamento crítico

Ponto chave: A relação entre design de software e arquitetura é crítica para um desenvolvimento de sistema eficaz.

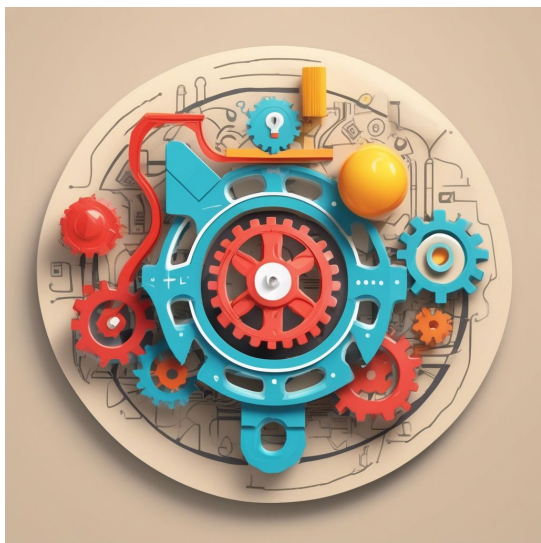
Interpretação crítica: Enquanto Robert C. Martin argumenta que design e arquitetura são fundamentalmente a mesma coisa e que um bom design minimiza a demanda por recursos, essa perspectiva pode desconsiderar as complexidades e variações em diferentes projetos de software. O autor enfatiza a importância de manter a qualidade para garantir a longevidade de um sistema, mas há o risco de simplificação excessiva, já que cada sistema pode exigir designs únicos adaptados a contextos específicos. Críticos, como Martin Fowler, sugerem que o design não pode ser uma solução única para todos e que as nuances de cada projeto devem ser reconhecidas. Isso levanta questões sobre a universalidade dos princípios defendidos por Martin e instiga os leitores a considerar pontos de vista e metodologias alternativas.

Ponto chave: Um bom design minimiza o esforço de manutenção no desenvolvimento de software.



Interpretação crítica: A afirmação de Martin de que um design de software eficaz reduz a necessidade de manutenção contínua é fundamental, pois destaca a importância de construir sistemas robustos. No entanto, essa perspectiva pressupõe que todas as falhas de engenharia possam ser resolvidas apenas por meio do design. Essa visão pode merecer uma análise crítica, pois negligencia a natureza dinâmica da tecnologia e das capacidades da equipe, além de fatores externos potenciais, como a mudança nos requisitos dos usuários e as prioridades de negócios. Além disso, estudos, incluindo os da Agile Alliance, sugerem que o desenvolvimento de software inclui adaptabilidade e melhoria iterativa como elementos-chave. Portanto, ao reconhecer o valor de um bom design, os leitores também devem considerar como a flexibilidade no design e na arquitetura pode ser igualmente crucial para o sucesso.

Capítulo 2 Resumo : Uma História de Dois Valores



Seção	Resumo
Visão Geral	Sistemas de software oferecem dois valores: comportamento e estrutura. Os desenvolvedores frequentemente priorizam o comportamento em relação à estrutura, o que pode reduzir o valor geral do software.
Comportamento	O principal valor do software é o comportamento, medido pela satisfação das partes interessadas. Os desenvolvedores incorretamente focam apenas na implementação de requisitos e na correção de bugs.
Arquitetura	O segundo valor é a flexibilidade da arquitetura, que deve permitir mudanças fáceis em resposta aos requisitos em evolução das partes interessadas. Rigidez leva ao aumento dos custos e da complexidade em modificações futuras.
O Maior Valor	Há uma tensão entre a funcionalidade imediata e a adaptabilidade de longo prazo. Programas estáticos podem se tornar obsoletos com a mudança dos requisitos, enquanto programas adaptáveis podem evoluir para atender novas necessidades, mesmo que atualmente não funcionem.
Matriz de Eisenhower	Comportamento é frequentemente visto como urgente, mas não importante; arquitetura é importante, mas raramente urgente. Esse desalinhamento pode levar a ignorar necessidades arquitetônicas críticas para recursos percebidos como urgentes.
Luta pela Arquitetura	Os desenvolvedores devem defender uma arquitetura robusta contra pressões para priorizar recursos urgentes. Garantir uma arquitetura sólida é crucial para evitar futuras complicações e custos associados a modificações.

2 UMA HISTÓRIA DE DOIS VALORES

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Visão Geral

Todo sistema de software oferece aos stakeholders dois valores: comportamento e estrutura. Os desenvolvedores devem equilibrar ambos, mas frequentemente priorizam o valor menos importante, levando a um retorno decrescente no valor do software.

COMPORTAMENTO

O valor primário do software é seu comportamento, representado por quão bem ele atende às necessidades dos stakeholders. Os desenvolvedores muitas vezes acreditam erroneamente que sua única responsabilidade é implementar esses requisitos e resolver quaisquer erros.

ARQUITETURA

O segundo valor reside na 'maleabilidade' do software, o que significa que ele deve ser facilmente modificável. Mudanças nos requisitos dos stakeholders não devem ser difíceis de implementar. A discrepância entre o alcance das mudanças e a forma exige uma arquitetura flexível para evitar custos

Mais Livros Grátis



Escaneie para baixar



Ouça-o

crescentes, já que a rigidez arquitetônica complica modificações futuras.

O MAIOR VALOR

Uma questão crítica surge: é o software que funciona ou o software que é facilmente ajustável que tem mais valor? Os gerentes de negócios costumam priorizar a funcionalidade imediata em detrimento da adaptabilidade a longo prazo, o que pode levar a problemas significativos. Um programa que funciona perfeitamente, mas é estático, torna-se inútil à medida que os requisitos evoluem, enquanto um programa não funcional, mas facilmente modificável, ainda pode ser ajustado para atender às necessidades em mudança.

MATRIZ DE EISENHOWER

A matriz de Eisenhower diferencia entre tarefas urgentes e importantes. O comportamento se encaixa na categoria urgente, mas nem sempre importante, enquanto a arquitetura é importante, mas raramente urgente. Essa priorização muitas vezes leva à negligência da integridade arquitetônica em favor de recursos que não podem ser perdidos.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

LUTE PELA ARQUITETURA

Os desenvolvedores de software devem defender a arquitetura, enfrentando lutas contra outros stakeholders que podem priorizar recursos urgentes em detrimento da estrutura subjacente. Os desenvolvedores têm interesse em garantir que a arquitetura seja sólida para evitar complicações futuras e custos crescentes, tornando isso parte de seu dever essencial. A falta de defesa da arquitetura pode levar a um sistema que se torna caro e difícil de adaptar ao longo do tempo.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Exemplo

Ponto chave: Equilibrar comportamento e arquitetura no desenvolvimento de software é crucial para o valor a longo prazo.

Exemplo: Imagine que você recebeu a tarefa de desenvolver um novo aplicativo que deve se adaptar às necessidades em mudança dos usuários ao longo dos próximos anos. Enquanto todos estão empolgados em entregar uma interface brilhante e repleta de recursos, você entende que a arquitetura subjacente precisa ser flexível o suficiente para acomodar mudanças futuras sem grandes refatorações. Você recomenda investir tempo adicional na criação de uma estrutura adaptativa, em vez de correr para atender a solicitações imediatas de recursos. Ao fazer isso, você garante que, embora os comportamentos imediatos possam não ser perfeitos, o software permaneça valioso e relevante a longo prazo. Essa previsibilidade ao defender uma arquitetura robusta, no final, economizará tempo e custos à medida que os requisitos de negócios evoluem, levando a uma solução de software mais adaptável e sustentável.



Pensamento crítico

Ponto chave: O Equilíbrio Entre Comportamento e Arquitetura

Interpretação crítica: Robert C. Martin enfatiza o equilíbrio crítico entre o comportamento do software e a arquitetura, instando os desenvolvedores a não negligenciarem a integridade arquitetônica em prol de uma funcionalidade imediata. No entanto, pode-se argumentar que sua perspectiva favorece fortemente a adaptabilidade a longo prazo, sem considerar suficientemente as realidades de negócios, onde os objetivos de curto prazo muitas vezes dominam a tomada de decisões. Por exemplo, em mercados dinâmicos, as organizações podem priorizar a implementação rápida de funcionalidades para manter a competitividade, o que pode entrar em conflito com uma abordagem arquitetônica mais estruturada. Críticos da visão de Martin poderiam referenciar os princípios do Lean Startup, que defendem a iteração rápida com base no feedback dos usuários, às vezes às custas de considerações arquitetônicas.



Capítulo 3 Resumo : Visão Geral dos Paradigmas

3 VISÃO GERAL DOS PARADIGMAS

O capítulo apresenta uma visão geral de três paradigmas de programação principais: programação estruturada, programação orientada a objetos e programação funcional.

PROGRAMAÇÃO ESTRUTURADA

A programação estruturada foi popularizada pela primeira vez por Edsger W. Dijkstra em 1968, que argumentou contra o uso prejudicial de saltos descontrolados (instruções goto) em programas. Dijkstra introduziu construções como if/then/else e do/while/until para controlar o fluxo do programa, enfatizando que a programação estruturada impõe disciplina na transferência de controle direto.

PROGRAMAÇÃO ORIENTADA A OBJETOS

A programação orientada a objetos surgiu em 1966, fundada



por Ole Johan Dahl e Kristen Nygaard. Seu trabalho permitiu que variáveis locais de funções persistissem após a execução da função, levando ao conceito de classes e métodos. Este paradigma promove uma certa disciplina em relação à transferência de controle indireta.

PROGRAMAÇÃO FUNCIONAL

A programação funcional foi desenvolvida antes dos dois primeiros paradigmas, enraizada na invenção por Alonzo Church em 1936, que mais tarde influenciou a linguagem LISP. Um conceito chave na programação funcional é a imutabilidade, que significa que as variáveis não podem mudar seu valor de forma arbitrária, impondo, assim, disciplina na atribuição.

ALIMENTAÇÃO PARA REFLEXÃO

**Instalar o aplicativo Bookey para desbloquear
texto completo e áudio**

Mais Livros Grátis



Escaneie para baixar



Ouça-o



Escanear para baixar



Por que o Bookey é um aplicativo indispensável para amantes de livros



Conteúdo de 30min

Quanto mais profunda e clara for a interpretação que fornecemos, melhor será sua compreensão de cada título.



Clipes de Ideias de 3min

Impulsione seu progresso.



Questionário

Verifique se você dominou o que acabou de aprender.



E mais

Várias fontes, Caminhos em andamento, Coleções...

Teste gratuito com Bookey



Capítulo 4 Resumo : Programação Estruturada

PROGRAMAÇÃO ESTRUTURADA

Introdução a Edsger Dijkstra

Edsger Wybe Dijkstra, uma figura proeminente na computação, começou sua carreira como o primeiro programador da Holanda em 1952. Ele, eventualmente, mudou seu foco da física teórica para a programação devido aos seus desafios intelectuais e ao potencial de desenvolver a programação como uma disciplina reconhecida.

O Desafio da Programação

Dijkstra identificou que a programação é inerentemente complexa, com muitos detalhes desafiadores para a gestão humana. Para combater isso, ele propôs a aplicação de técnicas de prova matemática para criar uma hierarquia para os programadores usarem, semelhante àquela na matemática.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Ele descobriu que o uso indiscriminado de declarações 'goto' dificultava a divisão das tarefas de programação em componentes gerenciáveis.

Fundamentos da Programação Estruturada

Dijkstra sugeriu que certas estruturas de controle, nomeadamente sequência, seleção e iteração (anteriormente estabelecidas por Böhm e Jacopini), permitem uma metodologia de desenvolvimento—programação estruturada—que enfatiza clareza e rigor. Esse método possibilita a decomposição recursiva de tarefas em unidades mais simples e testáveis.

A Declaração Go To Considerada Prejudicial

Em um artigo de 1968, Dijkstra criticou o uso de declarações 'goto', iniciando um extenso debate na comunidade de programação. Com o tempo, à medida que as linguagens de programação evoluíram, o uso de 'goto' foi diminuindo, alinhando-se aos princípios da programação estruturada.

Decomposição Funcional

Mais Livros Grátis



Escaneie para baixar



Ouça-o

A programação estruturada permite a decomposição funcional, onde problemas complexos são divididos em funções mais simples e de alto nível. Isso levou a métodos como análise e design estruturados, promovidos por figuras como Ed Yourdon e Tom DeMarco.

Sem Provas Formais

Apesar de sua promessa teórica, a aplicação de provas formais na programação foi, em grande parte, não realizada. A maioria dos programadores hoje não busca provas formais para a correção do software, uma vez que elas não se mostraram práticas no desenvolvimento cotidiano.

Ciência ao Resgate

A abordagem matemática de Dijkstra contrastou com o método científico, que enfatiza a falseabilidade em vez da provabilidade. Testes, como Dijkstra observou, só podem revelar bugs, mas não podem provar correção, estabelecendo assim a importância da testabilidade no software.

Testes

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Os testes são cruciais porque identificam erros, mas não podem confirmar a ausência de bugs. Eles se alinham bem com o foco da programação estruturada em produzir funções pequenas e testáveis.

Conclusão

A programação estruturada continua relevante hoje, promovendo a criação de unidades de software testáveis e gerenciáveis, reforçando, assim, sua importância nas práticas de programação modernas. Os princípios da programação estruturada continuam a influenciar a abordagem da arquitetura de software em níveis tanto micro quanto macro.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Capítulo 5 Resumo : Programação Orientada a Objetos

PROGRAMAÇÃO ORIENTADA A OBJETOS

Introdução à PO

Programação Orientada a Objetos (POO) é essencialmente sobre a integração de dados e funções. Embora definições comuns como "modelagem do mundo real" sejam imprecisas, os componentes principais da POO incluem encapsulamento, herança e polimorfismo.

Encapsulamento

O encapsulamento garante que dados e funções estejam agrupados, com os dados ocultos de acesso externo, exceto através de funções especificadas. Este conceito é prevalente nas linguagens POO, mas também era possível em linguagens mais antigas como C. Linguagens POO modernas, como C++ e Java, introduziram complexidades



que enfraquecem o encapsulamento, já que os detalhes internos das estruturas de dados muitas vezes precisam ser tornados conhecidos durante a compilação, levando a uma exposição indireta.

Herança

A herança permite a criação de novas classes com base nas já existentes, possibilitando a reutilização de código. Antes das linguagens POO, programadores de C emulavam a herança através da gestão manual de structs. Apesar de apresentar uma sintaxe mais conveniente, a POO não inventou o conceito, mas tornou sua implementação mais fácil.

Polimorfismo

O polimorfismo permite que funções se comportem de maneira diferente com base no tipo de entrada. Embora o C não suportasse um verdadeiro polimorfismo em um sentido de POO, o princípio de usar ponteiros para funções existe desde os primórdios da programação. As linguagens POO, porém, oferecem uma maneira mais segura e estruturada de alcançar o polimorfismo, eliminando muitos riscos inerentes à gestão manual de ponteiros de função.



O Poder do Polimorfismo

O polimorfismo permite que sistemas funcionem com novos tipos de forma contínua, sem exigir alterações no código existente, apoiando uma arquitetura de plugins que promove flexibilidade estrutural. A POO possibilita uma separação de políticas de alto nível das implementações de baixo nível, facilitando a implantação e o desenvolvimento independentes.

Inversão de Dependência

O polimorfismo permite que as dependências do código-fonte sejam invertidas, dando aos arquitetos controle sobre a direção das dependências dentro de um sistema. Esta mudança cria uma separação clara entre diferentes componentes, levando à independência na implantação e no desenvolvimento.

Conclusão

Para arquitetos de software, a POO é fundamentalmente sobre o uso do polimorfismo para controlar dependências do



código-fonte, permitindo a criação de sistemas que podem se adaptar e evoluir sem reescritas custosas. Este controle é vital para formar uma arquitetura limpa e sustentável.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Chapter 6 Resumo : Functional Programming

FUNCTIONAL PROGRAMMING

In many ways, the concepts of functional programming predate programming itself, rooted in the work by Alonzo Church in the 1930s.

SQUARES OF INTEGERS

To illustrate functional programming, we can compare implementations of printing the squares of the first 25 integers in different languages. In Java, a mutable loop control variable is used, while Clojure demonstrates immutability without changing variable values. The stark difference highlights the critical feature of functional programming: variables in functional languages do not vary.

IMMUTABILITY AND ARCHITECTURE

Architects must be concerned with variable mutability

Mais Livros Grátis



Escaneie para baixar



Ouça-o

because mutable variables lead to race conditions, deadlocks, and other concurrency issues in applications. By ensuring immutability, we can minimize these problems, as concurrent applications do not face issues when no variable is updated. However, achieving full immutability is nuanced and may necessitate compromises in practical applications.

SEGREGATION OF MUTABILITY

One common approach is to segregate applications into mutable and immutable components. Immutable components operate without mutable variables, while mutable components handle state changes. Using transactional memory can help protect mutable variables from concurrent modifications. Clojure's atom facility is an example of managing mutable state within a disciplined framework.

EVENT SOURCING

**Instalar o aplicativo Bookey para desbloquear
texto completo e áudio**

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Ad



Escanear para baixar



App Store
Escolha dos Editores



22k avaliações de 5 estrelas

Feedback Positivo

Afonso Silva

...cada resumo de livro não só
..., mas também tornam o
...divertido e envolvente. O
...tizou a leitura para mim.

Fantástico!



Estou maravilhado com a variedade de livros e idiomas
que o Bookey suporta. Não é apenas um aplicativo, é
um portal para o conhecimento global. Além disso,
ganhar pontos para caridade é um grande bônus!

Brígida Santos

F



O
só
o
O

na Oliveira

...correr as
...ém me dá
...omprar a
...ar!

Adoro!



Usar o Bookey ajudou-me a cultivar um hábito de
leitura sem sobrecarregar minha agenda. O design do
aplicativo e suas funcionalidades são amigáveis,
tornando o crescimento intelectual acessível a todos.

Duarte Costa

Economiza tempo!



O Bookey é o meu apli
crescimento intelectual
perspicazes e lindame
um mundo de conheci

Aplicativo incrível!



Eu amo audiolivros, mas nem sempre tenho tempo para
ouvir o livro inteiro! O Bookey permite-me obter um resumo
dos destaques do livro que me interessa!!! Que ótimo
conceito!!! Altamente recomendado!

Estevão Pereira

Aplicativo lindo



Este aplicativo é um salva-vidas para
de livros com agendas lotadas. Os re
precisos, e os mapas mentais ajudar
o que aprendi. Altamente recomend

Teste gratuito com Bookey



Capítulo 7 Resumo : SRP: O Princípio da Responsabilidade Única

Seção	Resumo
Título	7SRP: O PRINCÍPIO DA RESPONSABILIDADE ÚNICA
Definição	O SRP afirma que um módulo deve ter uma, e apenas uma, razão para mudar, focando em um interessado ou ator específico.
Sintomas de Violação do SRP	Sintoma 1: Duplicação Acidental - Um módulo contém múltiplas responsabilidades, levando a erros devido a mudanças entrelaçadas de diferentes atores. Sintoma 2: Mesclas - Conflitos de código frequentes durante a integração surgem quando vários desenvolvedores modificam classes compartilhadas sem respeitar o SRP.
Soluções para Violação do SRP	Separar funções em diferentes classes ou módulos com base nas responsabilidades; usar o padrão Facade para delegar funções enquanto se mantém o SRP.
Conclusão	O SRP é crucial para um design modular, melhorando a manutenibilidade, reduzindo erros e facilitando mudanças na arquitetura de software.

7SRP: O PRINCÍPIO DA RESPONSABILIDADE ÚNICA

Dos princípios SOLID, o Princípio da Responsabilidade Única (SRP) é frequentemente mal interpretado devido ao seu nome, que pode sugerir que cada módulo deve realizar apenas uma tarefa. Na verdade, o SRP afirma que um módulo deve ter um, e apenas um, motivo para mudar, que está ligado à sua responsabilidade para com um acionista ou "ator" específico. Um módulo deve ser coeso, o que significa

Mais Livros Grátis



Escaneie para baixar



Ouça-o

que suas funções e estruturas de dados devem atender a um único conjunto de requisitos de um ator.

Sintomas de Violação do SRP

Sintoma 1: Duplicação Acidental

Um exemplo de violação do SRP pode ser observado em uma classe `Funcionário` que contém métodos com diferentes responsabilidades: `calcularSalario()`, `reportarHoras()`, e `salvar()`. Cada método atende a diferentes atores (contabilidade, RH e DBAs, respectivamente). Vincular esses métodos em uma única classe pode levar a efeitos colaterais indesejados quando as mudanças de um ator afetam os outros, resultando em erros e equívocos custosos.

Sintoma 2: Fusões

Fusões no código-fonte podem se tornar comuns quando vários desenvolvedores modificam uma classe que compromete o SRP. Por exemplo, mudanças feitas por uma equipe de RH podem entrar em conflito com as dos DBAs, levando a situações arriscadas e complicações durante a



integração do código.

Soluções para Violação do SRP

Para aderir ao SRP, é recomendado separar funções em diferentes classes ou módulos com base em suas responsabilidades. Ao isolar funções (por exemplo, através de um objeto `DadosFuncionário`), a duplicação acidental de código pode ser reduzida. Um método eficaz é usar o padrão Facade, que permite que uma classe sirva como um delegado para outras classes, mantendo as funções cruciais enquanto apoia o SRP.

Conclusão

Em última análise, o SRP enfatiza a importância do design modular na arquitetura de software. Ele leva a uma melhor manutenibilidade, reduz o risco de erros e cria uma estrutura limpa que facilita o uso extensivo e mudanças em diferentes componentes em um nível arquitetônico mais alto. Os princípios do SRP se estendem a outros conceitos relacionados ao design de componentes e arquiteturas, que serão discutidos em capítulos subsequentes.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Exemplo

Ponto chave: A importância da responsabilidade única no design de software

Exemplo: Imagine que você é um desenvolvedor de software encarregado de projetar um sistema de processamento de pagamentos. Em vez de juntar todas as funções, como gerenciar contas de usuários, gerar relatórios de transações e lidar com reembolsos em uma única classe `Payment`, você as separa em classes distintas, como `AccountManager`, `TransactionReporter` e `RefundHandler`. Isso significa que, quando um contador solicita alterações nas funcionalidades de relatórios, você pode adaptar o `TransactionReporter` sem correr o risco de causar interrupções nas funcionalidades dos usuários, garantindo que seu código permaneça estável e mais fácil de manter.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Pensamento crítico

Ponto chave: Compreender o SRP além de sua interpretação simplista é crucial para um design de software eficaz.

Interpretação crítica: O Princípio da Responsabilidade Única (SRP) é frequentemente mal interpretado como meramente exigir que um módulo realize uma única tarefa. A realidade é mais sutil; o SRP enfatiza que um módulo deve ter apenas uma razão para mudar, o que está alinhado com suas restrições em relação a partes interessadas específicas. Essa interpretação pode impactar significativamente as decisões de design, como visto no exemplo da classe Employee, onde os métodos atendem a atores distintos, mas interconectados. Assim, conflitar múltiplas responsabilidades em um único módulo pode levar a complicações e conflitos desnecessários, tornando a manutenção do código mais difícil e stretching os limites da modularidade. No entanto, enquanto o SRP oferece orientações valiosas para a arquitetura de software, é necessário ter cautela ao aceitá-lo de forma acrítica. Perspectivas alternativas sugerem que uma granulação excessiva no design pode levar a um número esmagador de classes, tornando os



sistemas pesados. A literatura de apoio, como 'Patterns of Enterprise Application Architecture' de Martin Fowler, explora essa tensão, defendendo princípios de design equilibrados que ponderam coesão contra complexidade.

Chapter 8 Resumo : OCP: The Open-Closed Principle

Section	Summary
OCP: The Open-Closed Principle	The Open-Closed Principle states that software should be open for extension but closed for modification, emphasizing the need for good software architecture to avoid extensive changes for simple extensions.
A Thought Experiment	An example illustrates the need for minimal changes when adapting a system for additional output formats, highlighting the importance of the Single Responsibility Principle and organized dependencies.
Design Architecture	Separation of tasks like data calculation and formatting allows structured code with minimal inter-dependencies, using unidirectional dependencies to maintain stability among components.
Directional Control	Correct dependency directions are ensured by using interfaces like FinancialDataGateway, allowing components to interact without direct dependencies.
Information Hiding	Internal workings are hidden through interfaces, protecting components from unnecessary dependencies and aligning with the Interface Segregation Principle.
Conclusion	The OCP leads to an architecture that supports easy extensions with minimal changes, promoting a hierarchy of components and unidirectional dependencies for robustness and adaptability.

8 OCP: The Open-Closed Principle

The Open-Closed Principle (OCP) states that a software artifact should be open for extension but closed for modification, allowing it to be extended without changing the existing codebase. This principle highlights the importance of software architecture, as poor architecture that requires extensive changes for simple extensions reflects failure in design.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

A Thought Experiment

Consider a system displaying a financial summary that must be adapted to create a printable report. Ideally, minimal old code should change if the system architecture is well-designed, emphasizing the separation of responsibilities (Single Responsibility Principle - SRP) and organized dependencies (Dependency Inversion Principle).

Design Architecture

By separating tasks such as calculating reported data and formatting it for different outputs, the code can be structured with minimal inter-dependencies. Components like Controllers, Interactors, Databases, Presenters, and Views must have unidirectional dependencies to ensure that high-level components remain unaffected by changes at lower levels. The Interactor serves as the core, containing the application's business rules, shielded from peripheral concerns.

Directional Control

Design complexity serves the purpose of ensuring correct



dependency directions. The use of interfaces like `FinancialDataGateway` and `FinancialReportPresenter` supports this directional control by allowing components to interact without direct dependencies.

Information Hiding

Hiding internal workings through interfaces protects components from unnecessary dependencies. This aligns with the Interface Segregation Principle, ensuring that components focus solely on what they directly use.

Conclusion

The OCP promotes system architecture that facilitates easy extensions without significant changes. It involves creating a hierarchy of components and establishing unidirectional dependencies to minimize the impact of lower-level changes on higher-level structures. The overarching goal is a robust, extendable architecture that remains adaptable over time.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Exemplo

Ponto chave: Open for extension, closed for modification

Exemplo: Imagine you are developing an e-commerce application that needs a loyalty rewards feature. Instead of modifying existing checkout procedures directly each time you introduce new loyalty rules, you create an interface for loyalty program integrations, which allows you to add new reward systems without altering the foundational payment process. This approach not only keeps your existing code intact but also aligns with the Open-Closed Principle, allowing your application to evolve flexibly while maintaining robust functionality.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Pensamento crítico

Ponto chave: The viability of the Open-Closed Principle may rely heavily on specific applications.

Interpretação crítica: While Robert C. Martin advocates for the Open-Closed Principle (OCP) as a best practice in software design, it is essential to recognize that its effectiveness can vary by context and implementation. Not every project may benefit from rigid adherence to OCP, particularly smaller or less complex systems where the overhead of implementing such an architecture could outweigh its proposed benefits. Critics argue that when OCP is pushed too far, it can lead to over-engineering and unnecessary complexity (S. Martin, 'Clean Architecture'). Moreover, agile methodologies promote rapid iterations which may require adjustments that contradict OCP principles. Therefore, while OCP presents an appealing aspirational model for robust design, it is crucial to weigh its applicability carefully against real-world constraints and project-specific requirements. Supporting literature, such as "Refactoring: Improving the Design of Existing Code" by Martin Fowler, suggests that sometimes, direct modifications to existing code may be more



practical than maintaining a strictly OCP-compliant architecture.

Capítulo 9 Resumo : LSP: O Princípio da Substituição de Liskov

Seção	Resumo
Definição do LSP	O Princípio da Substituição de Liskov (LSP) afirma que um objeto do tipo S pode ser substituído por um objeto do tipo T sem afetar as propriedades desejáveis do programa, confirmando que S é um subtipo de T.
Orientando o Uso da Herança	Uma ilustração do LSP é fornecida através da classe License e suas subclasses, PersonalLicense e BusinessLicense, que podem intercambiavelmente calcular taxas sem alterar o comportamento de uma aplicação de faturamento.
O Problema do Quadrado/Retângulo	Este problema exemplifica uma violação do LSP, onde o Quadrado herda incorretamente do Retângulo devido às suas dimensões dependentes, causando inconsistências quando a substituição é esperada.
LSP e Arquitetura	Embora inicialmente seja uma diretriz para herança, o LSP agora se aplica ao design de software, enfatizando interfaces bem definidas que incentivam a substituíbilidade, especialmente em contextos como serviços REST.
Exemplo de Violação do LSP	Um exemplo mostra um sistema de despacho de táxis onde as divergências de diferentes empresas de uma interface REST consistente complicam a arquitetura por meio de lógica condicional adicional, comprometendo a estabilidade e a segurança.
Conclusão	O Princípio da Substituição de Liskov destaca a importância de manter a substituíbilidade para garantir uma arquitetura de sistema limpa e adaptável, uma vez que as violações podem levar a complexidade e ineficiência.

9 LSP: O PRINCÍPIO DA SUBSTITUIÇÃO DE LISKOV

Definição de LSP

Em 1988, Barbara Liskov definiu o Princípio da Substituição de Liskov (LSP) da seguinte forma: Se um objeto do tipo S pode ser substituído por um objeto do tipo T sem alterar as

Mais Livros Grátis



Escaneie para baixar



Ouça-o

propriedades desejáveis do programa, então S é um subtipo de T.

Orientando o Uso da Herança

Um exemplo do LSP é ilustrado com uma classe chamada Licença, com suas subclasses LicençaPessoal e LicençaEmpresarial. Ambos os subtipos implementam seus próprios métodos de cálculo de taxas, mas são intercambiáveis sem afetar o comportamento da aplicação de Faturamento.

O Problema do Quadrado/Retração

Uma violação clássica do LSP é o problema do quadrado/retrato. Nesse caso, o Quadrado não herda adequadamente do Retângulo devido à mutabilidade diferente: onde o Retângulo pode ter altura e largura

**Instalar o aplicativo Bookey para desbloquear
texto completo e áudio**

Mais Livros Grátis



Escaneie para baixar



Ouça-o



Ler, Compartilhar, Empoderar

Conclua Seu Desafio de Leitura, Doe Livros para Crianças Africanas.

O Conceito



Esta atividade de doação de livros está sendo realizada em conjunto com a Books For Africa. Lançamos este projeto porque compartilhamos a mesma crença que a BFA: Para muitas crianças na África, o presente de livros é verdadeiramente um presente de esperança.

A Regra



Ganhe 100 pontos



Resgate um livro



Doe para a África

Seu aprendizado não traz apenas conhecimento, mas também permite que você ganhe pontos para causas beneficentes! Para cada 100 pontos ganhos, um livro será doado para a África.

Teste gratuito com Bookey



Chapter 10 Resumo : ISP: The Interface Segregation Principle

Section	Summary
Overview of ISP	The Interface Segregation Principle advocates that clients should not depend on unused methods, promoting interface segregation for relevance.
Example Scenario	An example with a Java class OPS illustrates that segregating operations into separate interfaces for users prevents unnecessary recompilation for unused methods.
ISP and Language Types	Statically typed languages like Java face issues with unnecessary dependencies, while dynamic languages like Ruby and Python manage dependencies at runtime, reducing coupling.
ISP and Architecture	Architects should avoid dependencies on modules with unnecessary features to prevent inefficiencies and risks during redeployments due to unrelated changes.
Conclusion	ISP emphasizes minimizing dependencies in code and architecture to enhance flexibility and avert complications, aligning with broader architectural principles.

10 ISP: THE INTERFACE SEGREGATION PRINCIPLE

Overview of ISP

The Interface Segregation Principle (ISP) suggests that no client should be forced to depend on methods it does not use. This principle promotes the idea of segregating interfaces to ensure that clients only interact with the methods relevant to them.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Example Scenario

Consider a class OPS in Java with multiple operation methods (op1, op2, op3). Different users (User1, User2, User3) interact with only specific methods. If User1 only uses op1, changes in op2 or op3 will unnecessarily force recompilation for User1. By segregating these operations into distinct interfaces per user (e.g., U1Ops for User1), we avoid unnecessary dependencies.

ISP and Language Types

The problem of unnecessary dependencies is particularly critical in statically typed languages like Java, where declarations must be included. Dynamic languages like Ruby and Python avoid this issue since their dependencies are inferred at runtime, leading to less tightly coupled systems.

ISP and Architecture

Beyond just coding practices, ISP has significant architectural implications. Architects should avoid depending on modules that carry unnecessary "baggage." For example, if a system depends on a framework tied to a specific



database, any unrelated changes in the database can force redeployments, which is inefficient and risky.

Conclusion

The primary lesson of ISP is to minimize excess dependencies in both code and architecture to maintain flexibility and reduce unforeseen complications. This perspective aligns with higher-level architectural considerations and principles, such as the Common Reuse Principle.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Exemplo

Ponto chave: Understanding and implementing ISP can significantly improve system flexibility.

Exemplo: Imagine you're a software developer tasked with creating a user application that interacts with various services. As you're coding, you notice that your application includes methods for many features, including some that certain users will never utilize. By adhering to the Interface Segregation Principle, you take the necessary steps to create smaller, targeted interfaces for each user role. This way, when a change is made, say, to a feature used only by one user group, the rest of your users' interfaces remain untouched, avoiding the need for recompilation and redeployments. This proactive segregation helps your application stay agile and responsive to change, ultimately enhancing collaboration and reducing potential breaking changes for your clients.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Pensamento crítico

Ponto chave: Questioning the absoluteness of ISP.

Interpretação crítica: While the Interface Segregation Principle (ISP) advocates for minimal dependencies by segregating interfaces to enhance flexibility, one must critically assess its applicability across various programming paradigms and contextual scenarios. Martin's assertion that clients should only rely on necessary methods may overlook situations where a broader interface could streamline development efforts or improve cohesion within a tightly-knit module. Additionally, in certain systems, too much interface segregation could lead to increased fragmentation, complicating the understanding of relationships among components. It's essential to balance the principles of ISP with practical development needs, as highlighted in sources such as "Design Patterns" by Gamma et al. or "Domain-Driven Design" by Eric Evans, which stress the importance of context in architectural decisions.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Chapter 11 Resumo : DIP: The Dependency Inversion Principle

Section	Summary
DIP: THE DEPENDENCY INVERSION PRINCIPLE	The Dependency Inversion Principle (DIP) focuses on ensuring that source code dependencies pertain to abstractions rather than concrete classes, which promotes system adaptability and reduces risk from changes in implementations.
Stable Abstractions	Stable abstractions are crucial for stable architectures. Changes in abstract interfaces often lead to corresponding changes in concrete implementations. Design practices should favor stable, abstract interfaces over volatile concrete classes.
Key Guidelines	- Avoid references to volatile concrete classes; prefer abstractions.- Exercise caution when deriving from volatile concrete classes due to strong dependencies.- Define functions as abstract to avoid unwanted dependencies.- Do not reference volatile concrete components explicitly.
Factories	Adhere to the DIP by using the Factory pattern for instantiating volatile concrete objects, like the Abstract Factory, which abstracts the instantiation process through a service factory interface.
Concrete Components	Concrete components may contain some DIP violations but should be minimal and managed effectively to maintain a clean architecture.
Conclusion	DIP is a key principle in structuring dependencies in software systems, promoting cleaner and maintainable architectures that will be revisited throughout the book.

11 DIP: THE DEPENDENCY INVERSION PRINCIPLE

The Dependency Inversion Principle (DIP) emphasizes flexible systems by ensuring that source code dependencies focus only on abstractions, rather than concrete classes or implementations. In both statically and dynamically typed languages, dependencies should not refer to concrete modules—this enhances the system's adaptability and

Mais Livros Grátis



Escaneie para baixar



Ouça-o

reduces risk associated with changes in concrete implementations.

Stable Abstractions

Stable abstractions form the backbone of stable architectures. Changes in abstract interfaces often lead to necessary changes in concrete implementations, whereas changes in concrete classes typically do not require interface modifications. Good design practices favor moving towards stable, abstract interfaces, which can involve several specific coding practices:

- Avoid references to volatile concrete classes; prefer abstract interfaces.
- Deriving from volatile concrete classes should be approached with caution, particularly in statically typed languages where inheritance creates strong dependencies.
- Avoid overriding concrete functions to prevent inheriting unwanted dependencies; instead, define functions as abstract and provide various implementations.
- Do not explicitly reference any concrete and volatile components.

Factories

Mais Livros Grátis



Escaneie para baixar



Ouça-o

To adhere to the DIP, the instantiation of volatile concrete objects should utilize a Factory pattern. An example includes the Abstract Factory, where the application abstracts the instantiation process via a service factory interface, thereby preventing direct dependencies on the underlying concrete classes.

Concrete Components

Typically, concrete components may still contain some DIP violations but should be limited to a small number of them. While it is impractical to eliminate all concrete dependencies, they should be managed efficiently to maintain a clean architecture.

Conclusion

The DIP is a fundamental principle that will recur throughout the architectural discussions in this book, establishing a clear guideline for structuring dependencies in software systems, leading to cleaner, more maintainable architectures. As we progress, the understanding of DIP will shape the architectural boundaries and enhance the overall design quality.



Exemplo

Ponto chave: Focus on abstractions to create adaptable systems.

Exemplo: Imagine coding a shopping cart application. Instead of tightly coupling your payment gateway to a specific payment processor, like `CreditCardProcessor` or `PayPalProcessor`, you create an abstraction called `PaymentProcessor`. This way, when you want to add new payment methods like Cryptocurrency, you simply implement the `PaymentProcessor` interface without changing the existing system. This flexibility makes it easier to adapt to new payment trends without significant rework, showcasing the essence of the Dependency Inversion Principle.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Pensamento crítico

Ponto chave: The overemphasis on abstractions may lead to an overly complex architecture.

Interpretação crítica: While Robert C. Martin argues that software systems should prioritize stability and flexibility through the Dependency Inversion Principle (DIP), it's essential to consider that excessive reliance on abstractions can create a convoluted design that may confuse developers and hinder maintenance rather than help it. There are scenarios, as outlined in sources like 'Domain-Driven Design' by Eric Evans, where pragmatic approaches to concrete implementations can facilitate quicker and more understandable development cycles. Overall, while the DIP promotes a clean architecture, its application should be balanced with practicality to avoid unnecessary complexity.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Chapter 12 Resumo : Components

Section	Summary
Definition of Components	Components are the smallest deployable units in a system, represented by files like jar, gem, or DLL, functioning as independently deployable plugins.
A Brief History of Components	Programming initially involved manual memory location specification, leading to challenges that were resolved with relocatable binaries and smart loaders.
Relocatability and Linkers	Relocatable binaries allowed separate code compilation and efficient external reference resolution, but led to bottlenecks, prompting a phased approach to linking and loading for improved speed.
Evolution Through Decades	Increased hardware efficiency in the 80s and 90s fostered the component plugin architecture, facilitating quick linking of components like .jar and DLL files.
Conclusion	The modern component plugin architecture, supporting dynamic linking at runtime, represents significant advancements in software design over the decades.

12 COMPONENTS

Definition of Components

Components are the smallest deployable units in a system, varying by programming language as jar files in Java, gem files in Ruby, or DLLs in .NET. They may be compiled into an executable or aggregated into archives like .war files and can function as independently deployable plugins.

A Brief History of Components

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Initially, software programmers manually specified memory locations within their programs, which led to challenges due to increasing memory and functionality limits. The solution came with the introduction of relocatable binaries, which the smart loader could adapt to various memory addresses, allowing for more flexible programming practices.

Relocatability and Linkers

The advent of relocatable binaries enabled programmers to compile code separately and resolve external references more efficiently. With the evolution of larger programs in the 60s and 70s, linking loaders became a bottleneck, prompting the separation of linking and loading into a distinct phase to enhance compilation speed.

Evolution Through Decades

**Instalar o aplicativo Bookey para desbloquear
texto completo e áudio**

Mais Livros Grátis



Escaneie para baixar



Ouçá-o



As melhores ideias do mundo desbloqueiam seu potencial

Essai gratuit avec Bookey



Escanear para baixar



Chapter 13 Resumo : Component Cohesion

COMPONENT COHESION

Which classes belong in which components is a crucial decision informed by sound software engineering principles. This chapter discusses three key principles of component cohesion:

-

REP

: The Reuse/Release Equivalence Principle

-

CCP

: The Common Closure Principle

-

CRP

: The Common Reuse Principle

THE REUSE/RELEASE EQUIVALENCE PRINCIPLE



The principle states that the granule of reuse is the granule of release. As software reuse has become prevalent, it is vital that developers track components through a release process and assign release numbers. Classes and modules within a component must be cohesive and share a common purpose. Moreover, all items in a component should be releasable together under the same version number and documentation.

THE COMMON CLOSURE PRINCIPLE

Gather classes that change together for the same reasons. This principle emphasizes that components should not have multiple reasons to change, akin to the Single Responsibility Principle (SRP) for classes. By confining changes to single components, the need to redeploy multiple components is minimized, enhancing maintainability.

SIMILARITY WITH SRP

Both the CCP and SRP echo the idea of grouping related entities together based on change frequency.

THE COMMON REUSE PRINCIPLE

Mais Livros Grátis



Escaneie para baixar



Ouça-o

The CRP dictates that classes reused together should be in the same component, while those that are not tightly coupled or dependent should not be grouped. This principle helps prevent unnecessary dependencies and ensures that components are as self-contained as possible.

THE TENSION DIAGRAM FOR COMPONENT COHESION

The principles of cohesion can conflict. REP and CCP promote larger components, while CRP advocates for smaller ones. Good architectural design seeks to navigate the tension between these principles, adapting as project needs evolve over time.

CONCLUSION

Component cohesion principles offer a nuanced view of how to effectively group classes within components. Balancing reusability and maintainability is a dynamic process, with the right composition likely changing as project focus shifts.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Chapter 14 Resumo : Component Coupling

14 COMPONENT COUPLING

The next three principles address the relationships between components, navigating the tension between developability and logical design—this involves managing technical, political, and volatile forces.

THE ACYCLIC DEPENDENCIES PRINCIPLE

-

Definition

: Allow no cycles in the component dependency graph.

-

Morning After Syndrome

: This phenomenon occurs when multiple developers change shared code, causing instability and integration issues, especially in larger projects.

-

Solutions

Mais Livros Grátis



Escaneie para baixar



Ouça-o

:

-

Weekly Build

: A practice where developers work on private code copies and integrate changes once a week, but this approach can become inefficient over time.

-

Acyclic Dependencies Principle (ADP)

: Engage in releasing components that can function independently, thus reducing direct dependencies and cycles.

ELIMINATING DEPENDENCY CYCLES

A solution to manage project scale and integration efficiently is to partition the development into releasable components, allowing teams to work independently and integrate in small increments.

COMPONENT DIAGRAMS

Components are represented as a directed acyclic graph (DAG), ensuring dependency management is straightforward.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

EFFECT OF CYCLES

- Creating cyclic dependencies makes release and testing difficult, leading to tight coupling where changes in one component force changes in another and complicate integration.

BREAKING CYCLES

1.

Dependency Inversion Principle (DIP)

: Introduce interfaces to invert dependencies.

2.

Creating New Components

: Move shared classes into a new component to eliminate cycles.

TOP-DOWN DESIGN

The design of the component structure should evolve with the system's growth rather than being dictated initially. This is necessary to manage volatility and changes effectively.

THE STABLE DEPENDENCIES PRINCIPLE

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Depend in the direction of stability. Design cannot be entirely static; volatility is inherent but must be managed. Volatile components should not burden stable ones with dependencies.

STABILITY MEASUREMENT

Stability can be gauged through metrics like Fan-in, Fan-out, and Instability (I), which indicate how many dependencies exist and their direction.

NOT ALL COMPONENTS SHOULD BE STABLE

A healthy system has a mix of stable and unstable components where changes are expected and manageable, promoting flexible architecture.

THE STABLE ABSTRACTIONS PRINCIPLE

A component should be as abstract as it is stable—stable components should be abstract and flexible, while unstable components may contain concrete implementations.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

MAIN SEQUENCE

Components positioned on the line connecting maximally stable and abstract (0, 1) and maximally unstable and concrete (1, 0) represent an ideal state, balancing abstractness and stability.

CONCLUSION

This chapter emphasizes the management of component coupling through sound principles and metrics, aiming to maintain efficient, flexible designs in software architecture that evolve with the application's development.



Chapter 15 Resumo : What Is Architecture?

WHAT IS ARCHITECTURE?

The term "architecture" evokes a sense of power and importance, particularly in software development. A software architect is primarily a programmer who engages actively in programming tasks while also guiding the team's design efforts to enhance productivity. Good software architecture involves dividing a system into well-defined components and establishing effective communication among them. The main goal is to keep development, deployment, operation, and maintenance efficient and flexible, emphasizing the importance of leaving options open for future changes.

DEVELOPMENT

An effective architecture makes a software system easy to develop. For smaller teams, a loose structure may be beneficial initially, whereas larger teams require clear component definitions and stable interfaces for productivity



and success.

DEPLOYMENT

Efficient deployment is crucial for a software system's usefulness. Often, deployment strategies aren't considered early in development, which can lead to architecture that complicates deployment. Decisions around architecture should include considerations for a seamless deployment process.

OPERATION

While architecture influences development and deployment significantly, its effect on operation is often less direct. Operational issues can often be resolved by increasing hardware resources, but it's essential for architecture to aid in understanding system operations and simplify maintenance.

**Instalar o aplicativo Bookey para desbloquear
texto completo e áudio**

Mais Livros Grátis



Escaneie para baixar



Ouçá-o

Ad



Escanear para baixar



Experimente o aplicativo Bookey para ler mais de 1000 resumos dos melhores livros do mundo

Desbloqueie **1000+** títulos, **80+** tópicos

Novos títulos adicionados toda semana

Product & Brand

 Liderança & Colaboração

 Gerenciamento de Tempo

 Relacionamento & Comunicação

 Estratégia de Negócios

 Criatividade

 Memórias

 Conheça a Si Mesmo

 Psicologia

Empreendedorismo

 História Mundial

 Comunicação entre Pais e Filhos

 Autocuidado

 Mente

Visões dos melhores livros do mundo

amento
pos

Os 7 Hábitos das
Pessoas Altamente
Eficazes



Mini Hábitos



Hábitos Atômicos



O Clube das 5
da Manhã



Como Fazer Amigos
e Influenciar
Pessoas



Com
Não



Teste gratuito com Bookey



Chapter 16 Resumo : Independence

INDEPENDENCE

Overview of Architecture Goals

A good architecture must support several key aspects of a system:

- Use cases and operational support.
- Maintenance facilitation.
- Development efficiency.
- Deployment ease.

Use Cases

Architects must ensure that the architecture aligns with the system's intended use cases, making these cases clear and prominent. The architecture should reflect the system's functionality, making behaviors easily identifiable without requiring extensive searching.

Operation

Mais Livros Grátis



Escaneie para baixar



Ouça-o

The architecture needs to enable the system to handle required performance metrics, such as response times and throughput. A robust architecture should account for various operational configurations, from microservices to monolithic executions, allowing for flexibility as demands evolve.

Development

Conway's Law posits that a system's design mirrors a team's communication structure. Therefore, architecture should facilitate independent component development, enabling teams to work without interfering with each other by isolating components effectively.

Deployment

A well-architected system should support straightforward and immediate deployment post-build, avoiding complex configurations. Proper component isolation and structure can significantly enhance deployment efficiency.

Leaving Options Open

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Successful architecture finds a balance among its various concerns while maintaining flexibility to adapt to changes over its lifecycle. It should allow for easy modifications by keeping options open for as long as possible, aided by sound architectural principles that support the effective partitioning of components.

Decoupling Layers

The architecture should separately manage components that change for different reasons and keep business rules distinct from UI elements. This strategic decoupling ensures that the system can accommodate varying demands without unnecessary complications.

Decoupling Use Cases

Use cases themselves should also be decoupled to prevent changes in one from affecting another. This separation allows for independent development and easier expansion of functionality.

Decoupling Mode

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Decoupling can happen at different levels: source code, deployment, or service level. Each mode serves a purpose, and the choice of which mode to implement may evolve over the life of the project, accommodating changing needs.

Independent Development and Deployment

Well-decoupled components reduce inter-team interference and offer flexibility in deployment. Systems should ideally allow for components to be “hot-swapped,” enhancing adaptability without affecting the overall system integrity.

Duplication Concerns

While avoiding duplication is vital, architects must differentiate between true duplication and accidental duplication, ensuring that the system remains maintainable and flexible.

Decoupling Modes

There are three decoupling levels:

-

Source Code Level:

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Simple module dependencies.

-

Deployment Level:

Manageable units that can be independently deployed.

-

Service Level:

Components operate independently over a network.

Good architecture accommodates these transitions over time while protecting core functionality from excessive volatility. It should allow for significant evolution while remaining manageable, enabling adaptations in deployment mode as necessary.

Conclusion

Good architecture must support the evolving nature of software systems. The ability to change the decoupling mode allows for appropriate responses to varying operational, developmental, and deployment needs without sacrificing the integrity or clarity of the system.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Chapter 17 Resumo : Boundaries: Drawing Lines

BOUNDARIES: DRAWING LINES

Software architecture involves creating boundaries that separate software elements and restrict interactions between them. These boundaries help to delay decisions that could prematurely influence the core business logic. The aim is to minimize the resources required for building and maintaining the system while avoiding coupling with premature decisions that don't relate to business requirements.

A COUPLE OF SAD STORIES

The cautionary tales of two companies, P and W, illustrate the consequences of premature architectural decisions. Company P, originally successful with a monolithic application, overcomplicated its architecture when forced to adapt to web technologies, leading to excessive development effort without a proportional business need. Similarly, company W's Architect imposed a convoluted enterprise



architecture, resulting in a labor-intensive system that hampered development and flexibility. These examples showcase how a lack of foresight in architectural choices can multiply workload unnecessarily.

FITNESSE

An architectural success story is the development of FitNesse, which avoided premature commitments to frameworks or databases by drawing clear boundaries between business rules and data access. The decision to keep functionality abstracted allowed for flexibility and experimentation, illustrating how thoughtful architecture can defer difficult decisions.

WHICH LINES DO YOU DRAW, AND WHEN DO YOU DRAW THEM?

Effective boundaries are drawn between elements that have varying levels of importance to core business rules. For example, the GUI and database should remain agnostic concerning each other and the business logic, allowing flexibility in implementation choices. The principle emphasizes drawing lines between relevant and irrelevant



components.

PLUGIN ARCHITECTURE

A plugin architecture allows for the separation of core functions from optional components. This structure enables the system to replace or upgrade parts without significantly impacting the overall system. By treating components like GUIs or databases as plugins, it becomes possible to adapt to various implementation technologies while maintaining independence from the core business logic.

THE PLUGIN ARGUMENT

Using examples from different companies, the text highlights the benefits of a plugin architecture in minimizing dependencies. Just as ReSharper operates independently of Visual Studio despite their relationship, a well-structured system should ensure that changes in one component do not adversely affect others, enhancing stability and reducing fragility through well-defined boundaries.

CONCLUSION

Mais Livros Grátis



Escaneie para baixar



Ouça-o

To draw effective boundaries in software architecture, systems should first be partitioned into clearly defined components while avoiding premature decisions regarding implementation details. The architecture should facilitate the flow of changes from lower-level components to core business rules, thereby adhering to principles like Dependency Inversion and Stable Abstractions. This approach promotes maintainability, adaptability, and clarity in architectural design.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Chapter 18 Resumo : Boundary Anatomy

18 BOUNDARY ANATOMY

Architecture and Boundaries

The architecture of a system consists of software components and the boundaries that separate them, which may manifest in various forms. This chapter explores the significance of these boundaries, especially regarding boundary crossings in system architecture.

Boundary Crossing

At runtime, a boundary crossing occurs when a function calls another function across a boundary, transferring data. Effective management of source code dependencies is crucial, as changes in one module may necessitate changes or recompilations in others. Boundaries serve as a means to shield against such changes.



The Dreaded Monolith

The most basic architectural boundary often lacks a physical representation and is effectively a disciplined segregation of functionality within a single address space. From a deployment perspective, this manifests as a monolithic application, comprising a single executable file. Even though boundaries in monolithic systems may not be visible, they still play a critical role in managing internal dependencies via dynamic polymorphism. Poorly structured monoliths can lead to a dangerous reliance on function pointers—risking decoupling.

Deployment Components

Deployment-level components, such as dynamically linked libraries, represent discrete architectural boundaries where

**Instalar o aplicativo Bookey para desbloquear
texto completo e áudio**

Mais Livros Grátis



Escaneie para baixar



Ouça-o



Escanear para baixar



Por que o Bookey é um aplicativo indispensável para amantes de livros



Conteúdo de 30min

Quanto mais profunda e clara for a interpretação que fornecemos, melhor será sua compreensão de cada título.



Clipes de Ideias de 3min

Impulsione seu progresso.



Questionário

Verifique se você dominou o que acabou de aprender.



E mais

Várias fontes, Caminhos em andamento, Coleções...

Teste gratuito com Bookey



Chapter 19 Resumo : Policy and Level

19 POLICY AND LEVEL

Software Systems as Statements of Policy

Software systems are fundamentally expressions of policy, describing how inputs are transformed into outputs through various smaller policy statements, such as business rules and data validation. Effective software architecture involves separating these policies based on how they change.

Levels of Policy

A "level" in this context refers to the distance of a policy from inputs and outputs; higher-level policies are further removed and generally change less frequently than lower-level policies.

Architecture Design and Dependencies

A good software architecture organizes components into a



directed acyclic graph, where nodes represent components at the same level and dependencies (source code relationships) dictate how they interact. Dependencies should point from lower-level components to higher-level components.

Understanding Level

The strict definition of "level" relates to its distance from inputs and outputs, with policies managing direct interactions at the lowest level. Using a data flow example, components like Translate highlight higher-level design by decoupling functionality from specific input and output functions.

The Importance of Separation

Separating higher-level policies from lower-level details minimizes the impact of changes. Changes in the input/output policies are less likely to affect higher-level policy, fostering flexibility and reducing maintenance costs.

Contemplation of Design and Dependencies

Design errors can arise when high-level functions directly depend on lower-level operations. A proper architecture

Mais Livros Grátis



Escaneie para baixar



Ouça-o

ensures that each policy operates distinctly, maintaining flexibility for changes in the system.

Conclusion

In essence, the design of software architectures reflects a complex interplay of various principles like SRP, OCP, CCP, and others, emphasizing the continuous evaluation and management of component dependencies for improved flexibility and maintainability.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Chapter 20 Resumo : Business Rules

BUSINESS RULES

Business rules dictate the operations that can make or save a business money. These include Critical Business Rules which remain applicable regardless of whether they are automated or not, alongside the Critical Business Data necessary for their execution. Entities, which embody these Critical Business Rules using Critical Business Data, serve as specific objects within the system where relevant business logic resides.

ENTITIES

Entities are system objects that encapsulate critical business rules and data, offering a clean interface for business logic. They remain isolated from external concerns like databases or user interfaces.

USE CASES

Not all business rules can be directly tied to entities; many



involve operational constraints specific to how automated systems function. Use cases outline these operations, specifying user interactions while keeping the implementation details abstract and independent from the user interface. Use cases maintain business rules while controlling their execution order and accessibility.

REQUEST AND RESPONSE MODELS

Use cases generate request and response data structures that do not rely on specific frameworks, ensuring that the business logic remains decoupled from user interface specifics. This independence allows for flexible, future-proofed interactions between users and the business rules.

CONCLUSION

Business rules are essential for operational success and must be encapsulated within well-defined entities, isolated from implementation concerns like user interfaces or databases to maintain system integrity and adaptability.



Chapter 21 Resumo : Screaming Architecture

SCREAMING ARCHITECTURE

Understanding Architecture

- Architecture can be compared to blueprints of a building, which signify the intended usage of the structure (e.g., homes vs. libraries).
- Similarly, application architecture should communicate its purpose clearly—is it a “Health Care System” or just a tech stack like “Rails” or “Spring”?

THEME OF AN ARCHITECTURE

- Per Ivar Jacobson, software architectures should support the use cases of a system, emphasizing that architecture is not merely about frameworks.
- Frameworks serve as tools and should not dictate architecture decisions.



PURPOSE OF AN ARCHITECTURE

- Good architecture emphasizes use cases, allowing flexibility in framework, database, and environment choices until later in development.
- Architecture must ensure usability without committing to specific implementations right away.

BUT WHAT ABOUT THE WEB?

- The web is a delivery mechanism, and application architecture should remain ignorant of delivery details until necessary, maintaining the ability to adapt across different formats.

FRAMEWORKS ARE TOOLS, NOT WAYS OF LIFE

Instalar o aplicativo Bookey para desbloquear texto completo e áudio

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Ad



Escanear para baixar



App Store
Escolha dos Editores



22k avaliações de 5 estrelas

Feedback Positivo

Afonso Silva

...cada resumo de livro não só
...o, mas também tornam o
...n divertido e envolvente. O
...tizou a leitura para mim.

Fantástico!



Estou maravilhado com a variedade de livros e idiomas
que o Bookey suporta. Não é apenas um aplicativo, é
um portal para o conhecimento global. Além disso,
ganhar pontos para caridade é um grande bônus!

Brígida Santos

FI



O
só
o
O

na Oliveira

...correr as
...ém me dá
...omprar a
...ar!

Adoro!



Usar o Bookey ajudou-me a cultivar um hábito de
leitura sem sobrecarregar minha agenda. O design do
aplicativo e suas funcionalidades são amigáveis,
tornando o crescimento intelectual acessível a todos.

Duarte Costa

Economiza tempo!



O Bookey é o meu apli
crescimento intelectual
perspicazes e lindame
um mundo de conheci

Aplicativo incrível!



Eu amo audiolivros, mas nem sempre tenho tempo para
ouvir o livro inteiro! O Bookey permite-me obter um resumo
dos destaques do livro que me interessa!!! Que ótimo
conceito!!! Altamente recomendado!

Estevão Pereira

Aplicativo lindo



Este aplicativo é um salva-vidas para
de livros com agendas lotadas. Os re
precisos, e os mapas mentais ajudar
o que aprendi. Altamente recomend

Teste gratuito com Bookey



Chapter 22 Resumo : The Clean Architecture

THE CLEAN ARCHITECTURE

Overview of Architectural Ideas

- The evolution of software architecture includes concepts like Hexagonal Architecture (Ports and Adapters), DCI, and BCE, all aiming for separation of concerns via layered design.

Characteristics of Layers

- Independence from frameworks: Allows using them as tools.
- Testability: Business rules can be tested independently of external elements.
- UI Independence: UI can be swapped without affecting the core system.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

- Database Independence: Allows switching databases without impacting business rules.
- External Agency Independence: Business rules remain agnostic about external interfaces.

Dependency Rule

- Dependencies must point inward towards higher-level policies, ensuring inner circles are unaware of outer circle details.

Entities

- Entities embody critical business rules and hold business data, changing less frequently than their operational context.

Use Cases

- Use cases implement application-specific rules, orchestrating data flow between entities, and are isolated from external changes.

Interface Adapters

Mais Livros Grátis



Escaneie para baixar



Ouça-o

- They convert data formats between use cases/entities and external agencies (like databases); they encompass components like controllers and views.

Frameworks and Drivers

- The outermost layer encompasses frameworks, which are abstracted from core business logic to minimize constraints.

Architectural Layers

- The architecture follows concentric circles representing levels of software, from high-level policies (inner circles) to low-level details (outer circles).

Crossing Boundaries

- Communicating across boundaries involves controlling source code dependencies to conform with the Dependency Rule, allowing for flexible architecture.

Typical Scenario Example



- A web-based system workflow includes controllers, use cases, and output data management while respecting the Dependency Rule with simplified custom objects.

Conclusion

- Layered architecture supports testability, independence, and adaptability. Proper separation of concerns leads to a more maintainable and versatile system.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Exemplo

Ponto chave: Immersive Example of Layered Independence

Exemplo: Imagine you're developing an e-commerce platform. Picture a scenario where you need to upgrade the user interface to enhance the shopping experience. Instead of rewriting vast amounts of code, you simply swap out the UI layer with a modern framework. The beauty of Clean Architecture lies in its structured layers—your business logic remains untouched and functional, showcasing how separation of concerns allows you to innovate without disrupting core processes.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Pensamento crítico

Ponto chave: Dependency Rule and Independence from Frameworks

Interpretação crítica: A critical concept in Clean Architecture is the 'Dependency Rule', which suggests that all dependencies in a system should point inward to higher-level components to improve maintainability and testability. This principle challenges the traditional view that frameworks should dictate application design by advocating for a model where core business logic remains insulated from changes in external frameworks and technologies. Critics, including those aligned with emerging software design trends, argue that strict adherence to such rules can lead to additional complexity and hinder rapid development, suggesting a balance between adherence to architectural principles and pragmatic flexibility. As practices in the software industry evolve, it is crucial to interrogate the applicability of rigid architectural rules against real-world project dynamics, where changing requirements often demand faster adaptations. Supporting literature can be found in works like 'Designing Data-Intensive Applications' by Martin

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Kleppmann, which discuss the flexibility required in contemporary application architectures.

Chapter 23 Resumo : Presenters and Humble Objects

23 PRESENTERS AND HUMBLE OBJECTS

The Humble Object Pattern

The Humble Object pattern addresses the challenge of separating hard-to-test behaviors from easily testable ones. It splits behaviors into two classes: the humble object, which contains untestable behaviors, and the other class that holds testable behaviors. In the context of GUIs, the Presenter is testable, while the View is humble and primarily responsible for displaying data without processing it.

Presenters and Views

The View loads data for display but does not process it. Instead, the Presenter formats data received from the application and prepares it for the View through a View Model. The Presenter manages various data



presentations—dates, currencies, and text strings—thus keeping the View’s responsibilities minimal and straightforward.

Testing and Architecture

Testability is vital for good architecture. The Humble Object pattern illustrates this by highlighting the importance of dividing behaviors based on testability, defining architectural boundaries, especially between the Presenter and View.

Database Gateways

Database gateways bridge the use case interactors and the database. They provide interfaces for CRUD operations and are categorized as humble objects that simply manage data access using SQL or other database interfaces.

Data Mappers

Data mappers (or ORMs) transfer data between objects and databases. They belong in the database layer, creating a boundary that separates the application logic from data operations.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Service Listeners

Services also represent boundaries created by the Humble Object pattern. Data is loaded into simple structures for external services, and incoming service data is formatted into usable structures for the application.

Conclusion

The use of the Humble Object pattern at architectural boundaries enhances the testability of the system by visually marking distinctions between hard-to-test and easily testable elements, leading to clearer and more maintainable architectures.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Chapter 24 Resumo : Partial Boundaries

PARTIAL BOUNDARIES

Full-fledged architectural boundaries are costly and require significant effort for design and maintenance. A good architect may prefer to implement a partial boundary instead, holding space for a full one if necessary later. This anticipatory design is often critiqued in Agile as violating YAGNI (You Aren't Gonna Need It). However, architects may pragmatically consider future needs when implementing.

SKIP THE LAST STEP

One way to create a partial boundary is to do all the necessary design work and then compile and deploy all components as a single unit. This saves on administration overhead, such as version tracking and release management, while retaining the setup of a potential future full boundary. An example of this approach is the early strategy for FitNesse, where components were intended to be separable but were initially bundled due to the intention of a simplified



user experience.

ONE-DIMENSIONAL BOUNDARIES

A full architectural boundary involves reciprocal interfaces for isolation in both directions, but this can be expensive. A simpler structure can maintain basic separation, often illustrated through the Strategy pattern. A ServiceBoundary interface isolates clients from ServiceImpl classes, setting the stage for future expansion into full boundaries. However, without mutual interfaces, degradation can occur, allowing backchannel dependencies to form, which can complicate maintenance.

FACADES

Another way to define boundaries is through the Facade pattern, which sacrifices some dependency inversion. This

**Instalar o aplicativo Bookey para desbloquear
texto completo e áudio**

Mais Livros Grátis



Escaneie para baixar



Ouçá-o



Ler, Compartilhar, Empoderar

Conclua Seu Desafio de Leitura, Doe Livros para Crianças Africanas.

O Conceito



Esta atividade de doação de livros está sendo realizada em conjunto com a Books For Africa. Lançamos este projeto porque compartilhamos a mesma crença que a BFA: Para muitas crianças na África, o presente de livros é verdadeiramente um presente de esperança.

A Regra



Ganhe 100 pontos



Resgate um livro



Doe para a África

Seu aprendizado não traz apenas conhecimento, mas também permite que você ganhe pontos para causas beneficentes! Para cada 100 pontos ganhos, um livro será doado para a África.

Teste gratuito com Bookee



Chapter 25 Resumo : Layers and Boundaries

LAYERS AND BOUNDARIES

Overview of System Components

- Systems can usually be simplified into three components: User Interface (UI), Business Rules, and Database.
- However, more complex systems require a more nuanced decomposition.

Hunt the Wumpus Example

- A classic text-based game that demonstrates how to separate UI and game rules through a language-independent API, allowing flexibility in UI.
- Game rules communicate with a data storage system using an API, maintaining a clear separation from implementation details.



Clean Architecture Application

- The concept of clean architecture promotes well-defined architectural boundaries, which can be assessed along various axes of change, such as language and communication mechanisms.

Decoupling Use Cases

- Effective system architecture recognizes varying axes of change and organizes components accordingly, creating APIs that allow flexibility and isolation of different functionalities within the system.

Crossing Data Streams

- As systems grow in complexity, data streams can multiply.
- The GameRules component can have a higher-level policy that interacts with various subsystems.

Adding Complexity

- Introducing additional components like network services requires careful organization but preserves the central



component's structure for processing data.

Boundary Analysis

- Effective architecture requires identifying boundaries and ensuring that higher-level components retain independence from lower-level ones to avoid entanglement and enhance maintainability.

Conclusion

- Recognizing the necessity of boundaries in software architecture aids in understanding interactions and dependencies, ensuring effective design decisions are made to support future adjustments and scalability without high costs.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Chapter 26 Resumo : The Main Component

26 THE MAIN COMPONENT

The Ultimate Detail

The Main component is a crucial part of any software system that creates, coordinates, and oversees other components. It serves as the initial entry point for the system and is where dependencies are injected, typically using a Dependency Injection framework. This component is often called the "dirtiest" because it handles the management of dependencies and the orchestration of the system without exposing high-level functionality.

Main Component Example

For example, in a version of the game "Hunt the Wumpus", the Main component initializes the game by loading necessary configurations and issuing commands based on



user input. It coordinates various aspects of the game, ensuring that they work together seamlessly while remaining decoupled from lower-level details.

Conclusion

The Main component plays a vital role in maintaining cleanliness and organization within a software architecture. It establishes the foundational connection between high-level abstract functionalities and low-level details, making it easier for developers to manage dependencies and system behavior.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Chapter 27 Resumo : Services: Great and Small

27 SERVICES: GREAT AND SMALL

Service Architecture?

- Service-oriented and micro-service architectures are popular.
- They are believed to provide strong decoupling and independence in development, which is only partially true.
- Using services does not inherently define an architecture; they are often just expensive function calls.

Service Benefits?

- The benefits of service architecture are often exaggerated.
-

The Decoupling Fallacy

: Services are perceived as decoupled because they run in different processes. However, they can still be tightly



coupled through shared data and interfaces.

-

The Fallacy of Independent Development and Deployment

: Services are often supposed to be independently managed by different teams, but changes in data or behavior can still require significant coordination across teams.

The Kitty Problem

- The example of a taxi aggregator system illustrates that changes, such as onboarding a kitten delivery service, would require modifications across all involved services, highlighting the interdependencies.

Objects to the Rescue

- Using SOLID principles, we could structure classes that

**Instalar o aplicativo Bookey para desbloquear
texto completo e áudio**

Mais Livros Grátis



Escaneie para baixar



Ouça-o



As melhores ideias do mundo desbloqueiam seu potencial

Essai gratuit avec Bookey



Escanear para baixar



Chapter 28 Resumo : The Test Boundary

THE TEST BOUNDARY

Tests as System Components

Tests are integral to the architecture of a system, participating just like any other part. There is confusion regarding the nature of tests, but from an architectural perspective, all tests are equivalent—whether unit tests, integration tests, acceptance tests, etc. They follow the Dependency Rule, depending inward on the code being tested and operating as the outermost circle in the system architecture. Tests are independently deployable and do not affect system operation; their primary role is to support development.

Design for Testability

The isolation of tests often leads to the misconception that they are disconnected from the system's design, which can result in fragile tests that hinder system changes. Strong



coupling between tests and the system causes widespread test failures with seemingly minor system modifications, showcasing the Fragile Tests Problem. The solution lies in designing for testability, avoiding dependencies on volatile elements, such as GUIs. Tests should assess business rules without the need for GUI interaction.

The Testing API

To verify business rules, a specialized API can be created for tests, designed to bypass constraints and facilitate testing states within the application, ensuring independence from the application structure. Structural coupling, a detrimental form of test coupling, occurs when test classes correlate directly with production classes. Reducing structural coupling through a testing API allows production code and tests to evolve separately, promoting flexibility and generality in production code while ensuring the tests remain adaptable.

Conclusion

Tests are vital components that need to be integrated into the design of a system. Poorly designed tests lead to fragility and increased maintenance burdens, highlighting the necessity for



a structured testing framework within the overall architecture.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Chapter 29 Resumo : Clean Embedded Architecture

29 CLEAN EMBEDDED ARCHITECTURE

Introduction

- James Grenning reflects on the implications of a statement made by Doug Schmidt regarding the need for software updates due to evolving firmware and hardware.
- The distinction between software (which does not wear out) and firmware (which becomes obsolete) is emphasized.

Understanding Firmware

- Firmware definitions are discussed, noting that firmware is linked to dependencies on hardware, rather than its storage medium.
- Unmanaged dependencies on hardware can complicate embedded software and adversely affect its lifespan.



App-titude Test

- The focus on quickly making embedded code functional often leads to poor structuring for long-term maintenance.
- Kent Beck's three stages of software development highlight the common oversight of proper structuring when attaining functionality.

Target-Hardware Bottleneck

- Embedded developers face unique challenges including memory limitations and real-time constraints.
- Many embedded systems are intertwined with hardware, creating challenges in testing and debugging.

Layers in Embedded Architecture

- A clean architecture is proposed that includes separation between hardware and software, allowing for adaptability as hardware changes.
- The Hardware Abstraction Layer (HAL) is introduced to mitigate dependencies from contaminating software.

Operating System as a Detail

Mais Livros Grátis



Escaneie para baixar



Ouça-o

- An Operating System Abstraction Layer (OSAL) is suggested to mitigate dependencies on specific operating systems for greater flexibility.
- Clean architecture principles emphasize that embedded software must remain testable without reliance on hardware or operating systems.

Programming to Interfaces

- Following best practices of programming to interfaces and ensuring decoupling is vital for maintainability and testability.
- Limitations on header file contents are discussed to avoid unnecessary dependencies.

Conclusion

- The necessity for a clean embedded architecture is reiterated for the long-term health of products.
- Developers are encouraged to learn from broader software development practices to avoid incorporating unnecessary firmware in their codebases.

This summary encapsulates the main ideas from Chapter 29



of "Clean Architecture," emphasizing the need for effective structuring in embedded systems to ensure longevity and maintainability.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Chapter 30 Resumo : The Database Is a Detail

Chapter 30 Summary: The Database Is a Detail

Overview

The database is viewed as a non-entity in software architecture, akin to a doorknob in a house's architecture. While the data model is crucial, the database itself is just software that acts as a utility for data access—a low-level detail that architectural integrity should not be compromised by.

Relational Databases

Developed from Edgar Codd's principles in the 1970s, relational databases became the dominant technology due to their elegance and robustness. Despite their utility, their structure doesn't pertain to architecture. Arguably, use cases should remain indifferent to the relational table setup, which



should only concern low-level utility functions.

Prevalence of Database Systems

Database systems dominate software enterprises primarily due to disk technology, which has evolved but also suffers from speed limitations—requiring complex indexing and optimizations. Disks have transformed over time but remain a bottleneck compared to memory (RAM). In contexts without physical disks, data would be managed more effectively with data structures optimized for speed rather than relational models.

Implications of No Disk

With the decline of disk technology, consideration shifts to RAM, illustrating that data organization should focus on pointers and references rather than tables—reinforcing that

**Instalar o aplicativo Bookey para desbloquear
texto completo e áudio**

Mais Livros Grátis



Escaneie para baixar



Ouçá-o

Ad



Escanear para baixar



Experimente o aplicativo Bookey para ler mais de 1000 resumos dos melhores livros do mundo

Desbloqueie **1000+** títulos, **80+** tópicos

Novos títulos adicionados toda semana

Product & Brand

 Liderança & Colaboração

 Gerenciamento de Tempo

 Relacionamento & Comunicação

 Estratégia de Negócios

 Criatividade

 Memórias

 Conheça a Si Mesmo

 Psicologia

Empreendedorismo

 História Mundial

 Comunicação entre Pais e Filhos

 Autocuidado

 Mente

Visões dos melhores livros do mundo

amento
pos

Os 7 Hábitos das
Pessoas Altamente
Eficazes



Mini Hábitos



Hábitos Atômicos



O Clube das 5
da Manhã



Como Fazer Amigos
e Influenciar
Pessoas



Com
Não



Teste gratuito com Bookey



Chapter 31 Resumo : The Web Is a Detail

THE WEB IS A DETAIL

The Endless Pendulum

The perception of the web as a transformative force in technology is misleading; it is merely the latest of many oscillations in the computer industry. Historically, we have alternated between centralizing computing power in servers and distributing it to terminals. The web fits into this pattern, reflecting changing paradigms rather than representing a fundamental shift.

The Upshot

The key takeaway is that the web, as a graphical user interface (GUI), should be treated as a detail rather than a core component of system architecture. Architects must focus on isolating business rules from presentation concerns



to maintain flexibility in the face of evolving technologies.

Decoupling UI and Business Logic

A practical example highlights the importance of this segregation: Company Q attempted to adapt its popular finance software to mimic a web browser's design, ultimately causing dissatisfaction among users and prompting a return to a traditional desktop interface. The implication for architects is the necessity of decoupling business logic from GUIs to avoid undue influence from marketing trends or superficial changes.

Device Independence

Following principles established in prior decades, architects should regard the web as just another IO device, emphasizing the need for device independence. The architecture should allow business logic to operate independently of the GUI's specifics and other details, promoting flexibility and adaptability to change over time.

Conclusion

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Emphasizing this may lead to challenges, particularly in reconciling the rich and unique attributes of web interfaces with the need for abstraction. However, creating boundaries between UI and business logic is essential to safeguard against volatility driven by external pressures, ensuring that core functionalities remain unaffected by surface-level changes.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Chapter 32 Resumo : Frameworks Are Details

32 FRAMEWORKS ARE DETAILS

Framework Authors

Frameworks have gained popularity and can be useful, but they are not equivalent to software architecture. Authors often create frameworks to address their own issues, not the specific problems of users. Although there may be some overlap in challenges faced, this does not mean frameworks are inherently suited for your needs.

Asymmetric Marriage

The commitment to a framework is heavily one-sided. Users invest heavily in the framework while receiving no guarantees from the framework authors. Framework authors typically push for tight coupling with their frameworks, making it hard for users to detach from them later.



The Risks

Several risks arise from using frameworks:

- Frameworks may violate the Dependency Rule, leading to tight coupling with core business logic.
- Frameworks might support early development but can become a hindrance as the product matures.
- Updates to frameworks can introduce challenges, such as obsolete features or compatibility issues.
- A better framework may emerge, but switching could be difficult.

The Solution

Avoid tightly coupling to a framework. Keep it at a distance and treat it as a detail in the outer layers of the architecture. For example, if using Spring, refrain from sprinkling framework-specific annotations throughout core business logic. Instead, make the Main component aware of the framework, which should be the one to handle framework interactions.

I Now Pronounce You...

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Some frameworks are unavoidable, like STL in C++ or the Java standard library. Recognize the implications of binding your application to a framework, as this can affect the application's lifecycle and flexibility.

Conclusion

Approach frameworks cautiously. Ideally, maintain boundaries that allow the framework's influence to be minimized, enabling flexibility in future development.

Mais Livros Grátis



Escaneie para baixar



Ouçá-o

Chapter 33 Resumo : Case Study: Video Sales

Case Study: Video Sales

The Product

This case study focuses on an e-commerce website selling videos, inspired by cleancoders.com. The platform allows individuals to stream or purchase videos, while businesses can buy licenses in bulk. Key actors include video authors, administrators, and both individual and business users.

Use Case Analysis

Identifying actors and their use cases is crucial. Using the Single Responsibility Principle, the system will be structured to ensure that changes for one actor don't affect others. The use cases outlined are not exhaustive; some are abstract, catering to more generalized scenarios.



Component Architecture

A preliminary architecture partitions the system into views, presenters, interactors, and controllers, corresponding to each actor. Each component may represent a deployable unit, such as a .jar or .dll file, simplifying independent updates while still allowing for flexibility in combining deliverables.

Dependency Management

The architecture follows the Dependency Rule. Control flows from controllers to interactors, from interactors to presenters, and finally from presenters to views, maintaining a clear direction of dependencies. Changes in low-level components should not impact higher-level policies.

Conclusion

The architectural structure utilizes two dimensions of separation: actors based on changing responsibilities and the Dependency Rule for stable policies. This layered approach allows for adaptability in deployment while minimizing the ripple effects of changes across the system.





Escanear para baixar



Por que o Bookey é um aplicativo indispensável para amantes de livros



Conteúdo de 30min

Quanto mais profunda e clara for a interpretação que fornecemos, melhor será sua compreensão de cada título.



Clipes de Ideias de 3min

Impulsione seu progresso.



Questionário

Verifique se você dominou o que acabou de aprender.



E mais

Várias fontes, Caminhos em andamento, Coleções...

Teste gratuito com Bookey



Chapter 34 Resumo : The Missing Chapter

34 THE MISSING CHAPTER

Introduction to Design Approaches

The chapter highlights the significance of implementation details in software design, particularly when applying principles from sources like "Clean Architecture." It illustrates the concept using a practical example of designing an online bookstore.

Package by Layer

- Traditional approach where code is divided into layers (web, business logic, persistence).
- Simplifies initial development but can become cumbersome as the system scales.
- Layers do not inherently embody business domain concepts, leading to similar structures across different



projects.

Package by Feature

- An alternative approach that organizes code by features or aggregate roots.
- Provides better visibility into business context and can simplify finding related functionality.
- Seen as a minor improvement over package by layer, yet still considered suboptimal.

Ports and Adapters

- Emphasizes a separation between domain-focused code and technical implementation details.
- Advocates for creating an architecture where external dependencies like UI and databases depend on the business logic from a clean architecture perspective.

Package by Component

- Suggests a more integrated approach, combining related functionality for a single component.
- Focuses on reducing complexity during development while

Mais Livros Grátis



Escaneie para baixar



Ouça-o

maintaining a clean separation of concerns.

Implementation Details

- Reveals how improper implementation practices, such as overly broad access modifiers (e.g., public), can undermine architectural intentions.
- Discusses the implications of access restrictions and the importance of controlled dependencies within the architecture.

Conclusion

- Highlights the necessity of thoughtful implementation to ensure that the design principles translate effectively into maintainable code.
- Encourages an awareness of potential pitfalls and emphasizes the need for adherence to architectural rules to foster an optimal development experience.

This chapter ultimately reinforces the idea that good architectural practices, when coupled with careful attention to implementation, can lead to better software outcomes, while also cautioning against common pitfalls that could derail these efforts.



Chapter 35 Resumo : Appendix A

Architecture Archaeology

ARCHITECTURE ARCHAEOLOGY

This chapter takes a reflective journey through various projects spanning 45 years, aimed at uncovering principles of effective software architecture.

UNION ACCOUNTING SYSTEM

In the late 1960s, ASC Tabulating partnered with Local 705 of the Teamsters Union to create an accounting system. Utilizing a massive GE Datnet 30 computer, the project faced challenges including no operating system or file system, requiring the team to develop their own solutions.

THE MINICOMPUTER ORPHANAGE

Transitioning to the Varian 620/f minicomputer, the team utilized a novel overlay system to manage limited memory effectively. A preemptive supervisor was created,



showcasing essential architectural boundaries and dependency management.

LASER TRIM

At Teradyne, the team developed a laser trimming system for electronic components. The M365 computer used required manual tape loading and editing, illustrating the limitations of technology at that time and the complexity of tightly-coupled system architectures.

ALUMINUM DIE-CAST MONITORING

While at Outboard Marine Corporation, a real-time monitoring system was built using IBM's System/7 without any libraries or operating systems, highlighting the difficulty in software maintenance due to tightly-coupled architectures.

4-TEL

Developed to test telephone lines automatically, this system transitioned from a monolithic structure into a more decoupled architecture, separating data collection and analysis functions across two computers (COLTs and SACs),

Mais Livros Grátis



Escaneie para baixar



Ouça-o

improving response times and memory efficiency.

DISPATCH DETERMINATION

Encountering issues with rigid and poorly understood code led to a realization about the importance of clean, maintainable architecture, which ultimately influenced design approaches in future projects.

THE GRAND REDESIGN IN THE SKY

Efforts to modernize systems led to challenges with maintaining legacy code across different geographical areas, with different teams making conflicting changes, emphasizing the complications of software reuse and integration.

C LANGUAGE AND BOSS

The transition to using C and developing the Basic Operating System and Scheduler (BOSS) allowed the team to tackle a range of new projects with greater efficiency and flexibility in achieving concurrent tasks.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

VRS: VOICE RESPONSE SYSTEM

A prototype for a voice response system contributed insights into service-oriented architectures with methods for handling real-time responses and control, although it faced challenges with further database integrations.

CLEAR COMMUNICATIONS

At this startup, the author experienced fast-paced and chaotic environments, highlighting the pitfalls of over-engineering and the struggle to maintain viable projects amidst constant changes and a lack of market interest.

ARCHITECTS REGISTRY EXAM

In a consulting role, the development of reusable frameworks for applications demonstrated the challenges of ensuring true reusability, ultimately requiring parallels with hindsight and real-world experience to craft effective systems.

CONCLUSION

The chapter provides a high-level view of the intertwining



experiences and lessons learned in various projects throughout the author's career, culminating in the realization that software architecture must adapt to the nuances of specific requirements and environments. This narrative serves as a foundation for understanding clean architecture principles moving forward.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Pensamento crítico

Ponto chave: The importance of adaptable architecture amid varying project requirements and technological constraints.

Interpretação crítica: While Robert C. Martin emphasizes the necessity of clean architecture for maintaining software systems, one could argue that the nuances of each project may require flexible architectural choices that deviate from his principles. Critics like Martin Fowler in "Refactoring" point out that adaptability, not strict adherence to clean architecture, is sometimes paramount for innovation and responsiveness to change. It's possible that adhering rigidly to certain architectural ideals could stifle creativity and practical solutions in software development.



Ad



Escanear para baixar



App Store
Escolha dos Editores



22k avaliações de 5 estrelas

Feedback Positivo

Afonso Silva

...cada resumo de livro não só
..., mas também tornam o
...divertido e envolvente. O
...tizou a leitura para mim.

Fantástico!



Estou maravilhado com a variedade de livros e idiomas
que o Bookey suporta. Não é apenas um aplicativo, é
um portal para o conhecimento global. Além disso,
ganhar pontos para caridade é um grande bônus!

Brígida Santos

F



O
só
o
O

na Oliveira

...correr as
...ém me dá
...omprar a
...ar!

Adoro!



Usar o Bookey ajudou-me a cultivar um hábito de
leitura sem sobrecarregar minha agenda. O design do
aplicativo e suas funcionalidades são amigáveis,
tornando o crescimento intelectual acessível a todos.

Duarte Costa

Economiza tempo!



O Bookey é o meu apli
crescimento intelectual
perspicazes e lindame
um mundo de conheci

Aplicativo incrível!



Eu amo audiolivros, mas nem sempre tenho tempo para
ouvir o livro inteiro! O Bookey permite-me obter um resumo
dos destaques do livro que me interessa!!! Que ótimo
conceito!!! Altamente recomendado!

Estevão Pereira

Aplicativo lindo



Este aplicativo é um salva-vidas para
de livros com agendas lotadas. Os re
precisos, e os mapas mentais ajudar
o que aprendi. Altamente recomend

Teste gratuito com Bookey



Melhores frases do Arquitetura Limpa. O Guia do Artesao Para Estrutura e Design de Software por Robert C. Martin com números de página

Ver no site do Bookey e gerar imagens de citações bonitas

Capítulo 1 | Frases das páginas 38-82

- 1....não há diferença entre eles. Nenhuma mesmo.
- 2.O objetivo da arquitetura de software é minimizar os recursos humanos necessários para construir e manter o sistema requerido.
- 3.Se esse esforço é baixo e permanece baixo durante toda a vida do sistema, o design é bom. Se esse esforço cresce a cada nova versão, o design é ruim. É tão simples quanto isso.
- 4.A única maneira de ir rápido é ir bem.
- 5.Essas curvas drenarão catastroficamente o lucro do modelo de negócios e levarão a empresa a um impasse, se não a um colapso total.
- 6.Se tiver seu jeito, [a bagunça crescente de código] reduzirá

Mais Livros Grátis



Escaneie para baixar



Ouça-o

a produtividade a zero em questão de meses.

7. Mas para aqueles que não foram iludidos pela autoconfiança da Lebre, o resultado é esperado, porque eles conhecem esta simples verdade do desenvolvimento de software: A única maneira de ir rápido é ir bem.

Capítulo 2 | Frases das páginas 83-134

1. Todo sistema de software oferece dois valores diferentes aos stakeholders: comportamento e estrutura. Os desenvolvedores de software são responsáveis por garantir que ambos esses valores permaneçam altos.
2. O software foi inventado para ser 'flexível'. Ele foi concebido como uma forma de mudar facilmente o comportamento das máquinas.
3. Se você me der um programa que funciona perfeitamente, mas é impossível de mudar, então ele não funcionará quando os requisitos mudarem, e eu não poderei fazer com que funcione. Portanto, o programa se tornará inútil.
4. Aquelas coisas que são urgentes raramente são de grande



importância, e aquelas coisas que são importantes raramente são de grande urgência.

5. Equipes de desenvolvimento de software eficazes enfrentam essa luta de frente. Elas discutem sem receios com todos os outros stakeholders como iguais.

Capítulo 3 | Frases das páginas 138-190

1. A programação estruturada impõe disciplina na transferência direta de controle.
2. A programação orientada a objetos impõe disciplina na transferência indireta de controle.
3. A programação funcional impõe disciplina na atribuição.
4. Cada um dos paradigmas remove capacidades do programador. Nenhum deles adiciona novas capacidades.
5. Os três paradigmas juntos removem declarações goto, ponteiros de função e atribuição.
6. Assim, esses três paradigmas são provavelmente os únicos três que veremos—pelo menos os únicos três que são negativos.
7. Uma noção fundamental do cálculo lambda é a



imutabilidade—isto é, a noção de que os valores dos símbolos não mudam.

8.Cada paradigma nos diz o que não fazer, mais do que nos diz o que fazer.

9.O objetivo de um bom design de software é minimizar os recursos humanos necessários para construir e manter o sistema exigido.

Mais Livros Grátis



Escaneie para baixar



Ouçá-o

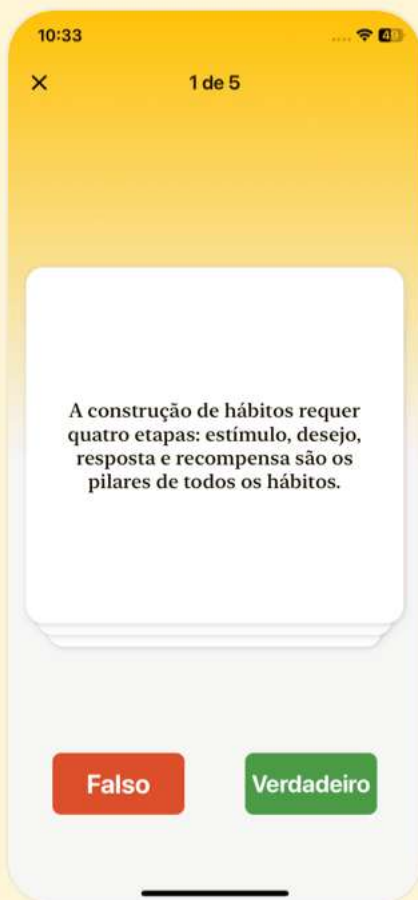


Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Escanear para baixar



Capítulo 4 | Frases das páginas 191-262

1. Dijkstra uma vez disse: 'Testes mostram a presença, não a ausência, de bugs.'
2. O problema que Dijkstra reconheceu, desde o início, foi que programar é difícil e que os programadores não fazem isso muito bem.
3. Eisenhower disse: Tenho dois tipos de problemas, os urgentes e os importantes. Os urgentes não são importantes e os importantes nunca são urgentes.
4. Se a arquitetura vem por último, então o sistema se tornará cada vez mais caro de desenvolver, e eventualmente a mudança se tornará praticamente impossível para parte ou todo o sistema.
5. A programação estruturada nos obriga a decompor recursivamente um programa em um conjunto de pequenas funções comprováveis.
6. A única maneira de ir rápido é ir bem.
7. Programar é difícil, e os programadores não fazem isso muito bem.



8.O desenvolvimento de software não é um empreendimento matemático, embora pareça manipular conceitos matemáticos.

Capítulo 5 | Frases das páginas 263-350

- 1.A base de uma boa arquitetura é o entendimento e a aplicação dos princípios do design orientado a objetos (OO).
- 2.A encapsulação é citada como parte da definição de OO, pois as linguagens OO oferecem encapsulação de dados e funções de forma fácil e eficaz.
- 3.Usar uma linguagem OO torna o polimorfismo trivial.
- 4.O fato de que as linguagens OO oferecem polimorfismo seguro e conveniente significa que qualquer dependência de código-fonte, não importa onde esteja, pode ser invertida.
- 5.O banco de dados e a interface do usuário podem ser plugins para as regras de negócio.

Chapter 6 | Frases das páginas 351-443

- 1.Variables in functional languages do not vary.



2. You cannot have a race condition or a concurrent update problem if no variable is ever updated.
3. The answer to that question is affirmative, if you have infinite storage and infinite processor speed.
4. Architects would be wise to push as much processing as possible into the immutable components, and to drive as much code as possible out of those components that must allow mutation.
5. Event sourcing is a strategy wherein we store the transactions, but not the state.



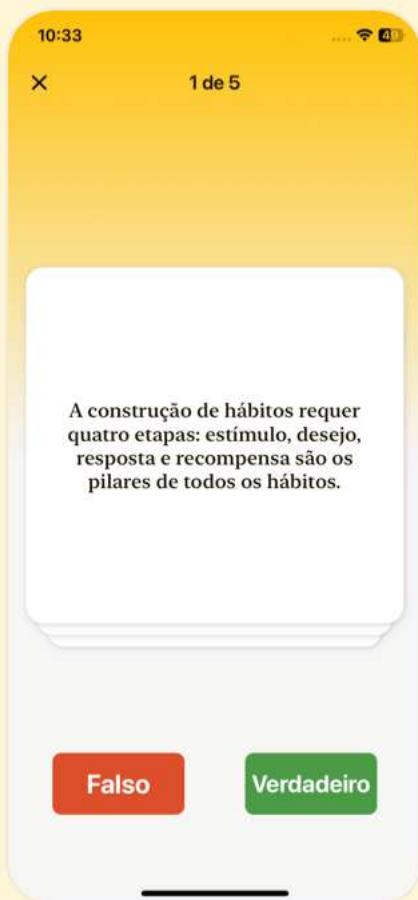


Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Escanear para baixar



Capítulo 7 | Frases das páginas 449-546

1. Um módulo deve ter uma, e somente uma, razão para mudar.
2. Um módulo deve ser responsável por um, e somente um, ator.
3. O SRP diz para separar o código do qual diferentes atores dependem.
4. Esses problemas ocorrem porque colocamos código do qual diferentes atores dependem muito próximos.
5. Não é inconcebível que o CFO também possa ser afetado.

Chapter 8 | Frases das páginas 547-649

1. A software artifact should be open for extension but closed for modification.
2. Clearly, if simple extensions to the requirements force massive changes to the software, then the architects of that software system have engaged in a spectacular failure.
3. By properly separating the things that change for different reasons (the Single Responsibility Principle), and then organizing the dependencies between those things properly



(the Dependency Inversion Principle).

- 4.If component A should be protected from changes in component B, then component B should depend on component A.
- 5.The goal is to make the system easy to extend without incurring a high impact of change.

Capítulo 9 | Frases das páginas 650-758

- 1.Se para cada objeto o1 do tipo S, existe um objeto o2 do tipo T tal que, para todos os programas P definidos em termos de T, o comportamento de P não muda quando o1 é substituído por o2, então S é um subtipo de T.
- 2.Este design está em conformidade com o LSP porque o comportamento da aplicação de Billing não depende, de forma alguma, de qual dos dois subtipos ela utiliza.
- 3.O famoso problema do quadrado/retângulo ilustra uma violação do LSP porque a altura e a largura do Retângulo são mutuamente mutáveis; em contraste, a altura e a largura do Quadrado devem mudar juntas.



4.O LSP se transformou em um princípio mais amplo de design de software que se refere a interfaces e implementações.

5.A violação do LSP pode causar a poluição da arquitetura de um sistema com uma quantidade significativa de mecanismos adicionais.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

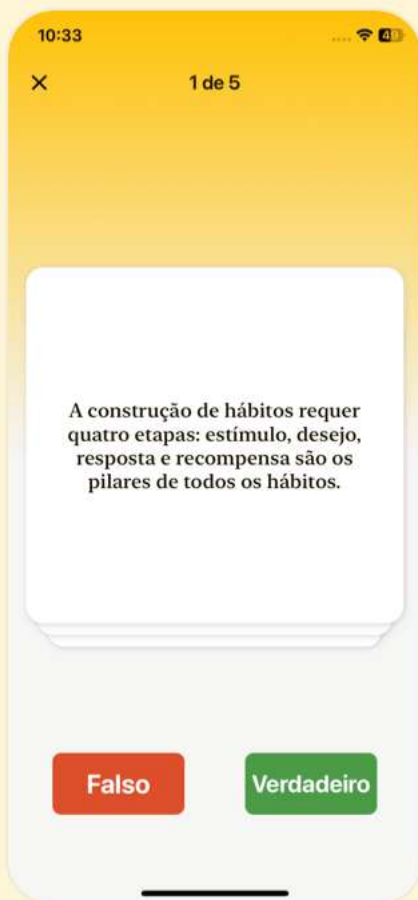


Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Escanear para baixar



Chapter 10 | Frases das páginas 759-864

- 1.It is harmful to depend on modules that contain more than you need.
- 2.Changes to those features within D may well force the redeployment of F and, therefore, the redeployment of S.
- 3.Depending on something that carries baggage that you don't need can cause you troubles that you didn't expect.

Chapter 11 | Frases das páginas 865-981

- 1.The Dependency Inversion Principle (DIP) tells us that the most flexible systems are those in which source code dependencies refer only to abstractions, not to concretions.
- 2.Every change to an abstract interface corresponds to a change to its concrete implementations.
- 3.Therefore interfaces are less volatile than implementations.
- 4.Good software designers and architects work hard to reduce the volatility of interfaces.
- 5.Don't refer to volatile concrete classes. Refer to abstract interfaces instead.



- 6.The flow of control crosses the curved line in the opposite direction of the source code dependencies.
- 7.DIP violations cannot be entirely removed, but they can be gathered into a small number of concrete components and kept separate from the rest of the system.
- 8.This means that all component relationships are unidirectional, as shown in the component graph in Figure 8.3.
- 9.The code that implements high-level policy should not depend on the code that implements low-level details.

Chapter 12 | Frases das páginas 984-1110

- 1.Regardless of how they are eventually deployed, well-designed components always retain the ability to be independently deployable and, therefore, independently developable.
- 2.Compiling a large program could take hours. To shorten the compile times, programmers separated the source code of the function library from the applications.
- 3.The solution was relocatable binaries. The idea behind



them was very simple. The compiler was changed to output binary code that could be relocated in memory by a smart loader.

4. A linking loader allowed programmers to divide their programs up into separately compilable and loadable segments.

5. By the mid-1990s, the time spent linking had begun to shrink faster than our ambitions could make programs grow.

6. Today we routinely ship .jar files or DLLs or shared libraries as plugins to existing applications.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

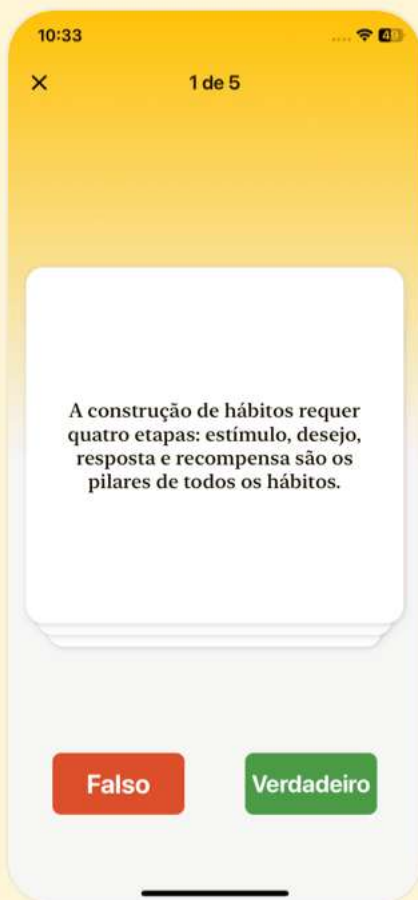


Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Escanear para baixar



Chapter 13 | Frases das páginas 1111-1250

1. The granule of reuse is the granule of release.
2. Gather into components those classes that change for the same reasons and at the same times.
3. Don't force users of a component to depend on things they don't need.
4. The tension between these principles that good architects seek to resolve.
5. Gather together those things that change at the same times and for the same reasons. Separate those things that change at different times or for different reasons.

Chapter 14 | Frases das páginas 1251-1433

1. Allow no cycles in the component dependency graph.
2. The solution to this problem is to partition the development environment into releasable components.
3. If there are cycles in the dependency structure, then the 'morning after syndrome' cannot be avoided.
4. Changes made to one component do not need to have an



immediate effect on other teams.

- 5.The component structure cannot be designed from the top down.
- 6.Designs cannot be completely static.
- 7.By conforming to the Common Closure Principle (CCP), we create components that are sensitive to certain kinds of changes but immune to others.
- 8.The ‘Jitters’ implies that the component structure is volatile in the presence of changing requirements.

Chapter 15 | Frases das páginas 1436-1631

- 1.The architecture of a software system is the shape given to that system by those who build it.
- 2.The primary purpose of architecture is to support the life cycle of the system.
- 3.The ultimate goal is to minimize the lifetime cost of the system and to maximize programmer productivity.
- 4.Good architecture makes the system easy to understand, easy to develop, easy to maintain, and easy to deploy.
- 5.The strategy behind that facilitation is to leave as many



options open as possible, for as long as possible.

6. The architecture of a system should elevate the use cases, the features, and the required behaviors of the system to first-class entities that are visible landmarks for the developers.
7. The primary cost of maintenance is in spelunking and risk.
8. The goal of the architect is to create a shape for the system that recognizes policy as the most essential element of the system while making the details irrelevant to that policy.
9. A good architect maximizes the number of decisions not made.
10. If you can develop the high-level policy without committing to the details that surround it, you can delay and defer decisions about those details for a long time.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

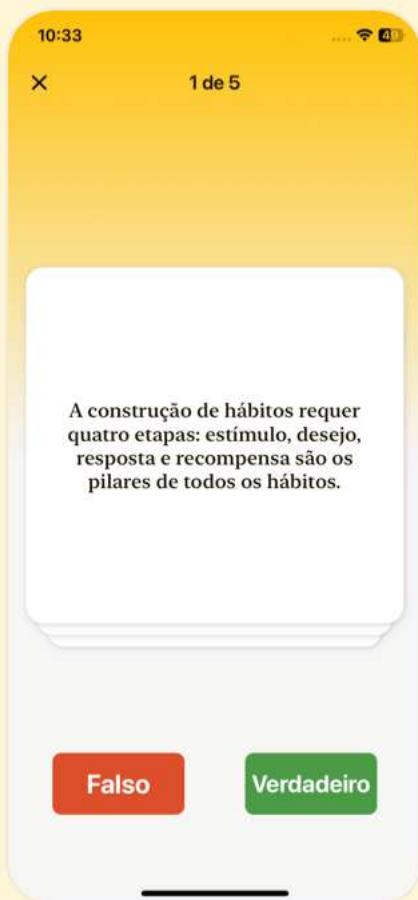


Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Escanear para baixar



Chapter 16 | Frases das páginas 1632-1848

- 1.The architecture must support the use cases.
- 2.A good architecture supports the operation of the system.
- 3.A good architecture leaves options open for the future.
- 4.The decoupling mode is one of those options that a good architect leaves open.
- 5.The only way to go fast, is to go well.
- 6.Good architecture makes the system easy to change, in all the ways that it must change, by leaving options open.

Chapter 17 | Frases das páginas 1849-2075

- 1.Software architecture is the art of drawing lines that I call boundaries.
- 2.A good system architecture is one in which decisions like these are rendered ancillary and deferrable.
- 3.The irony is that company P never sold a system that required a server farm.
- 4.The tragedy is that the architects, by making a premature decision, multiplied the development effort enormously.
- 5.A good architecture makes the system easy to understand,



easy to develop, easy to maintain, and easy to deploy.

6.The ultimate goal is to minimize the lifetime cost of the system and to maximize programmer productivity.

7.Good architects carefully separate details from policy, and then decouple the policy from the details so thoroughly that the policy has no knowledge of the details and does not depend on the details in any way.

8.Good architecture leaves the decoupling mode open as an option so that large deployments can use one mode, whereas small deployments can use another.

9.A good architecture makes the system easy to change, in all the ways that it must change, by leaving options open.

Chapter 18 | Frases das páginas 2076-2299

1.At runtime, a boundary crossing is nothing more than a function on one side of the boundary calling a function on the other side and passing along some data.

2.Managing and building firewalls against this change is what boundaries are all about.



3. Such architectures almost always depend on some kind of dynamic polymorphism to manage their internal dependencies.
4. Even when statically linked into a single executable, the ability to independently develop and marshal the various components for final assembly is immensely valuable.
5. High-level components remain independent of lower-level details.
6. Consequently, communications across source-level decoupled boundaries can be very chatty.
7. A much stronger physical architectural boundary is the local process.
8. The strongest boundary is a service.
9. Communication across local process boundaries involve operating system calls, data marshaling and decoding, and interprocess context switches, which are moderately expensive.
10. A good architecture leaves options open.



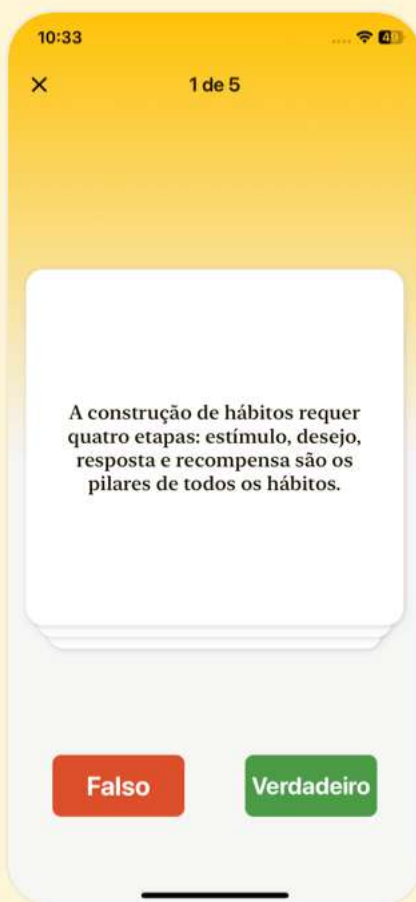


Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Escanear para baixar



Chapter 19 | Frases das páginas 2300-2519

1. A computer program is a detailed description of the policy by which inputs are transformed into outputs.
2. The art of developing a software architecture is carefully separating those policies from one another, and regrouping them based on the ways that they change.
3. Policies that change for the same reasons, and at the same times, are at the same level and belong together in the same component.
4. In a good architecture, the direction of those dependencies is based on the level of the components that they connect. In every case, low-level components are designed so that they depend on high-level components.
5. Higher-level policies—those that are farthest from the inputs and outputs—tend to change less frequently, and for more important reasons, than lower-level policies.
6. Keeping these policies separate, with all source code dependencies pointing in the direction of the higher-level



policies, reduces the impact of change.

Chapter 20 | Frases das páginas 2520-2751

1. Critical Business Rules usually require some data to work with. For example, our loan requires a loan balance, an interest rate, and a payment schedule.
2. The Entity is pure business and nothing else.
3. A use case describes application-specific business rules as opposed to the Critical Business Rules within the Entities.
4. Use cases do not describe how the system appears to the user. Instead, they describe the application-specific rules that govern the interaction between the users and the Entities.
5. The purpose of these two objects is very different. Over time they will change for very different reasons...

Chapter 21 | Frases das páginas 2752-2988

1. The architecture would scream: 'HOME.'
2. Architectures are not (or should not be) about frameworks.
Architectures should not be supplied by frameworks.



3. Good architectures are centered on use cases so that architects can safely describe the structures that support those use cases without committing to frameworks, tools, and environments.
4. The web is a delivery mechanism—an IO device—and your application architecture should treat it as such.
5. A good architecture allows decisions about frameworks, databases, web servers, and other environmental issues and tools to be deferred and delayed.
6. A good architecture emphasizes the use cases and decouples them from peripheral concerns.
7. Your Entity objects should be plain old objects that have no dependencies on frameworks or databases or other complications.
8. Your architecture should tell readers about the system, not about the frameworks you used in your system.



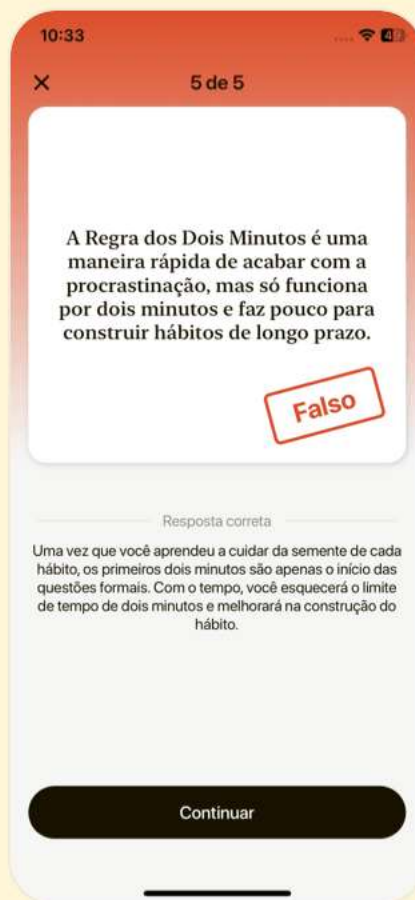
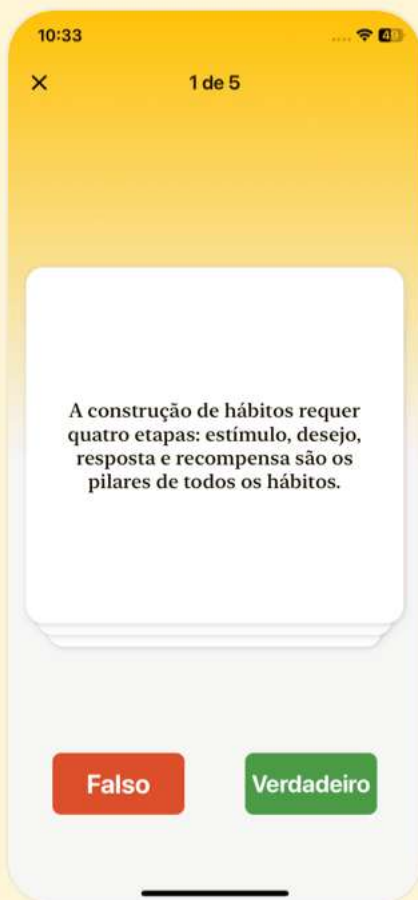


Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Escanear para baixar



Chapter 22 | Frases das páginas 2989-3234

1. The overriding rule that makes this architecture work is the Dependency Rule: Source code dependencies must point only inward, toward higher-level policies.
2. Each has at least one layer for business rules, and another layer for user and system interfaces.
3. We keep these things on the outside where they can do little harm.
4. Entities encapsulate enterprise-wide Critical Business Rules.
5. The architecture does not depend on the existence of some library of feature-laden software. This allows you to use such frameworks as tools, rather than forcing you to cram your system into their limited constraints.
6. If the details of a use case change, then some code in this layer will certainly be affected.
7. We want to protect the Controller from changes in the Presenters. We want to protect the Presenters from changes



in the Views.

8. A good architecture allows decisions about frameworks, databases, web servers, and other environmental issues and tools to be deferred and delayed.
9. We don't want our systems to exhibit that kind of fragility.
10. This leads us to our last metric. If it is desirable for components to be on, or close, to the Main Sequence, then we can create a metric that measures how far away a component is from this ideal.

Chapter 23 | Frases das páginas 3235-3488

1. A software architecture is the shape given to that system by those who build it.
2. The architecture of a system must support all the necessary use cases, but does not know what all those use cases are.
3. Good architecture makes the system easy to change, in all the ways that it must change, by leaving options open.
4. The Humble Object pattern is a design pattern that was originally identified as a way to help unit testers to separate behaviors that are hard to test from behaviors that are easy



to test.

5. Testability is an attribute of good architectures.
6. We want certain modules to be immune to others.
7. If you can develop the high-level policy without committing to the details that surround it, you can delay and defer decisions about those details for a long time.
8. A good architecture makes the system easy to understand, easy to develop, easy to maintain, and easy to deploy.
9. Source code dependencies must point only inward, toward higher-level policies.
10. Good architects carefully separate details from policy, and then decouple the policy from the details so thoroughly that the policy has no knowledge of the details and does not depend on the details in any way.

Chapter 24 | Frases das páginas 3489-3740

1. Good architecture makes the system easy to understand, easy to develop, easy to maintain, and easy to deploy.
2. The goal of good software design? That goal is nothing less



than my utopian description: The goal of software architecture is to minimize the human resources required to build and maintain the required system.

3. A good architecture comes from understanding it more as a journey than as a destination, more as an ongoing process of enquiry than as a frozen artifact.
4. A simple way to break a cycle of components and reinstate the dependency graph as a DAG is to apply the Dependency Inversion Principle (DIP).
5. The business rules should remain pristine, unsullied by baser concerns such as the user interface or database used.
6. Good architects strive to position the majority of their components at those endpoints. However, in my experience, some small fraction of the components in a large system are neither perfectly abstract nor perfectly stable.
7. A good architecture leaves options open, the more experiments you can run, the more things you can try, and the more information you will have when you reach the



point at which those decisions can no longer be deferred.

8. A good architecture does not depend on the existence of some library of feature-laden software. This allows you to use such frameworks as tools, rather than forcing you to cram your system into their limited constraints.

9. Architects should oversee the connections across organizational boundaries between components to facilitate better collaboration and efficiency.

10. Software architecture is the art of drawing lines that I call boundaries.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

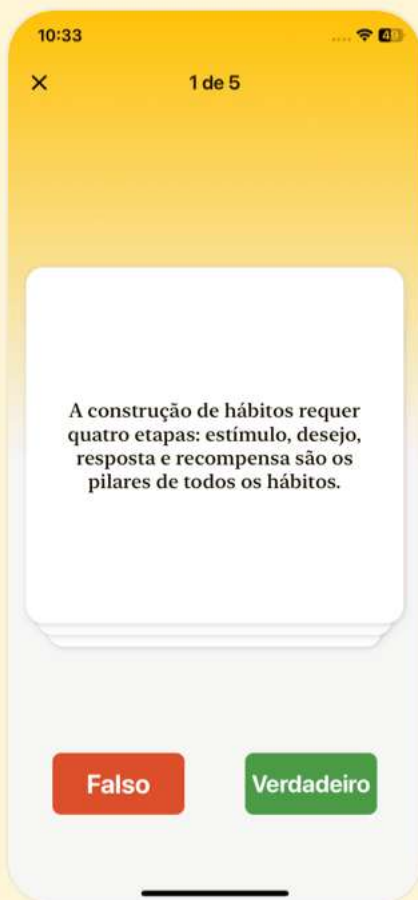


Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Escanear para baixar



Chapter 25 | Frases das páginas 3741-4006

1. The primary purpose of architecture is to support the life cycle of the system. Good architecture makes the system easy to understand, easy to develop, easy to maintain, and easy to deploy.
2. A good architecture allows decisions about frameworks, databases, web servers, and other environmental issues and tools to be deferred and delayed.
3. Software architecture is the art of drawing lines that I call boundaries. Those boundaries separate software elements from one another, and restrict those on one side from knowing about those on the other.
4. The goal of a software architecture, then, should be to make a system that can be easily deployed with a single action.
5. Policies that change for the same reasons, and at the same times, are at the same level and belong together in the same component. Policies that change for different reasons, or at different times, are at different levels and should be



separated into different components.

6. Good architecture makes the system easy to change, in all the ways that it must change, by leaving options open.

7. Good architects strive to position the majority of their components at those endpoints [of the Main Sequence].

Chapter 26 | Frases das páginas 4007-4271

1. I call this component Main. THE ULTIMATE
DETAIL

2. The Main component is the ultimate detail—the lowest-level policy.

3. Nothing, other than the operating system, depends on it.

4. Its job is to create all the Factories, Strategies, and other global facilities, and then hand control over to the high-level abstract portions of the system.

5. Imagine that you are looking at the blueprints of a building... What does the architecture of your application scream?

6. The architecture of a software system is the shape given to that system by those who build it.



7. A good architecture makes the system easy to change, in all the ways that it must change, by leaving options open.
8. Managers must learn to recognize the importance of separating the system's behavior from its implementation details.
9. A well-formed use case object should have no inkling about the way that data is communicated to the user, or to any other component.
10. The ultimate goal is to minimize the lifetime cost of the system and to maximize programmer productivity.

Chapter 27 | Frases das páginas 4272-4558

1. The architecture of a system is defined by boundaries that separate high-level policy from low-level detail and follow the Dependency Rule.
2. Services are, after all, just function calls across process and/or platform boundaries.
3. A good architecture leaves options open.
4. The architecture should reveal operation.
5. A good architecture is one in which decisions like these are



rendered ancillary and deferrable.

6. Don't depend on things you don't need.

7. The Dependency Rule: Source code dependencies must point only inward, toward higher-level policies.

Mais Livros Grátis



Escaneie para baixar



Ouça-o



Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Escanear para baixar



Chapter 28 | Frases das páginas 4559-4841

1. Tests are part of the system, and they participate in the architecture just like every other part of the system does.
2. Tests, by their very nature, follow the Dependency Rule; they are very detailed and concrete; and they always depend inward toward the code being tested.
3. Tests that are not well integrated into the design of the system tend to be fragile, and they make the system rigid and difficult to change.
4. The solution is to design for testability. The first rule of software design—whether for testability or for any other reason—is always the same: Don't depend on volatile things.
5. The purpose of the testing API is to decouple the tests from the application.
6. A program can be proven incorrect by a test, but it cannot be proven correct.

Chapter 29 | Frases das páginas 4842-5160



1. Although software does not wear out, firmware and hardware become obsolete, thereby requiring software modifications.
2. Although software does not wear out, it can be destroyed from within by unmanaged dependencies on firmware and hardware.
3. Stop writing so much firmware and give your code a chance at a long useful life.
4. Getting an app to work is what I call the App-titude test for a programmer.
5. A clean embedded architecture's software is testable off the target hardware.
6. Firmware does not mean code lives in ROM. It's not firmware because of where it is stored; rather, it is firmware because of what it depends on and how hard it is to change as hardware evolves.
7. A clean embedded architecture is good for your product's long-term health.

Chapter 30 | Frases das páginas 5163-5484

Mais Livros Grátis



Escaneie para baixar



Ouça-o

1. From an architectural point of view, the database is a non-entity—it is a detail that does not rise to the level of an architectural element.
2. A good architect does not allow low-level mechanisms to pollute the system architecture.
3. It is very unlikely that you leave the data in the form of files or tables.
4. But no matter how brilliant, useful, and mathematically sound a technology it is, it is still just a technology. And that means it's a detail.
5. Many data access frameworks allow database rows and tables to be passed around the system as objects. Allowing this is an architectural error.
6. To mitigate the time delay imposed by disks, you need indexes, caches, and optimized query schemes; and you need some kind of regular means of representing the data so that these indexes, caches, and query schemes know what they are working with.
7. That needs to be clear: I am not talking about the data



model. The structure you give to the data within your application is highly significant to the architecture of your system.

8. A good architecture makes the system easy to change, in all the ways that it must change, by leaving options open.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

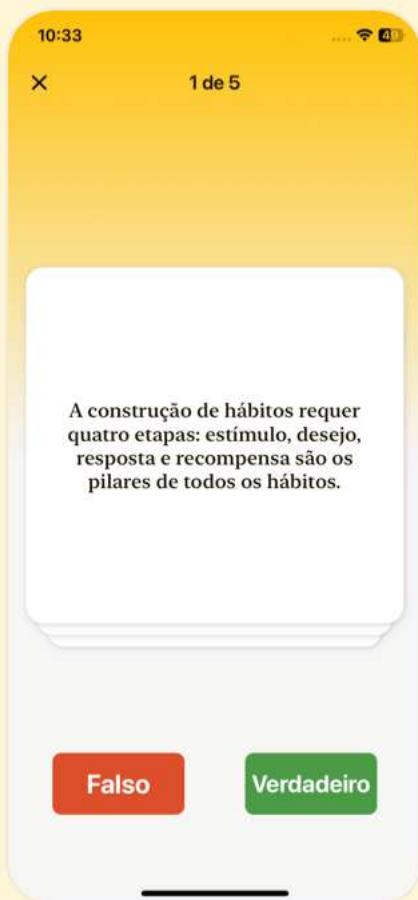


Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Escanear para baixar



Chapter 31 | Frases das páginas 5485-5802

1. Actually the web didn't change anything. Or, at least, it shouldn't have.
2. The GUI is a detail. The web is a GUI. So the web is a detail.
3. As architects, though, we have to look at the long term. Those oscillations are just short-term issues that we want to push away from the central core of our business rules.
4. There are always marketing geniuses out there just waiting to pounce on the next little bit of coupling you create.
5. The web is an IO device. In the 1960s, we learned the value of writing applications that were device independent.

Chapter 32 | Frases das páginas 5803-6127

1. Frameworks are not architectures—though some try to be.
2. The relationship between you and the framework author is extraordinarily asymmetric. You must make a huge commitment to the framework, but the framework author makes no commitment to you whatsoever.



3. Don't marry the framework! Oh, you can use the framework—just don't couple to it. Keep it at arm's length.
4. I now pronounce you ... Understand that when you marry a framework to your application, you will be stuck with that framework for the rest of the life cycle of that application.
5. The architecture of the framework is often not very clean. Frameworks tend to violate the Dependency Rule.
6. If the framework wants you to derive your business objects from its base classes, say no! Derive proxies instead, and keep those proxies in components that are plugins to your business rules.
7. The wedding ring is on your finger; and it's going to stay there.
8. Your problems will likely overlap with those other problems quite a bit. If this were not the case, frameworks would not be so popular.
9. Framework authors know their own problems, and the problems of their coworkers and friends. And they write their frameworks to solve those problems—not yours.



10. Frameworks are details that belong in one of the outer circles of the architecture. Don't let it into the inner circles.

Chapter 33 | Frases das páginas 6128-6457

1. The goal of good software design? That goal is nothing less than my utopian description: The goal of software architecture is to minimize the human resources required to build and maintain the required system.
2. It's not strictly necessary for me to create that abstraction. I could have left the abstract use case out of the diagram without compromising any of the features of the overall product.
3. Every time some new feature is added, or some existing feature is changed, that step will be taken to serve one of these actors.
4. Good architecture makes the system easy to understand, easy to develop, and easy to maintain, and easy to deploy.
5. Not all business rules are as pure as Entities. Some



business rules make or save money for the business by defining and constraining the way that an automated system operates.

6.The architecture also plays a huge role in determining the ease with which the system is deployed. The goal is 'immediate deployment.'

7.It is the tension between these principles that good architects seek to resolve.

8.A good architecture does not depend on the existence of some library of feature-laden software. This allows you to use such frameworks as tools, rather than forcing you to cram your system into their limited constraints.

9.Software architecture is the art of drawing lines that I call boundaries. Those boundaries separate software elements from one another, and restrict those on one side from knowing about those on the other.

10.The web is an IO device. In the 1960s, we learned the value of writing applications that were device independent. The motivation for that independence has



not changed.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

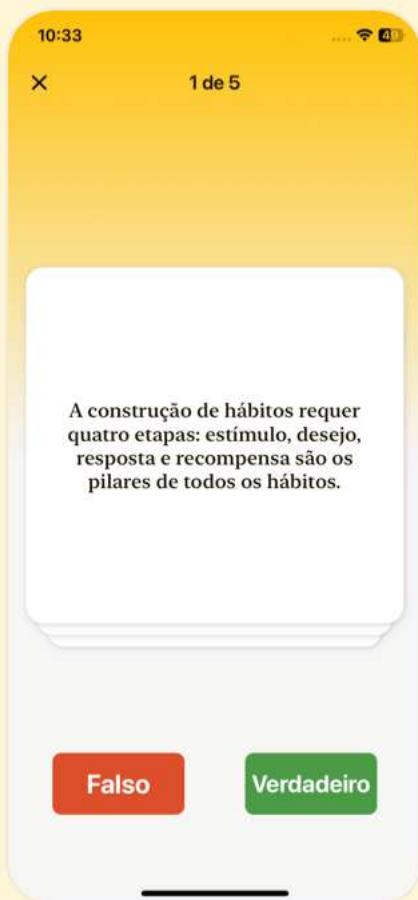


Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Escanear para baixar



Chapter 34 | Frases das páginas 6458-6826

- 1.The devil is in the implementation details, and it's really easy to fall at the last hurdle if you don't give that some thought, too.
- 2.The purpose of good architecture is to leave as many options open as possible, for as long as possible.
- 3.Software systems are statements of policy. Indeed, at its core, that's all a computer program actually is.
- 4.Good architecture makes the system easy to understand, easy to develop, easy to maintain, and easy to deploy.
- 5.The architecture should separate the code that different actors depend on.
- 6.A good architecture does not depend on the existence of some library of feature-laden software.
- 7.Don't depend on things you don't need.

Chapter 35 | Frases das páginas 6829-6894

- 1.The great claim to fame of the Datanet 30 was its capability to drive a large number of asynchronous terminals at relatively high speed.



2. When our processor booted, it would scan each chip and load the vector table at the start of each chip into the RAM vectors.
3. Every time we added or removed code from our 30K executable, it changed the addresses in which each instruction was loaded.
4. The architecture made it releasable, developable, and independently deployable.
5. You can't make a reusable framework until you first make a usable framework.
6. The idea was relatively simple, and deeply idealistic. Every object-oriented programmer wanted to have an object-oriented database in his or her system.
7. We had made the chips independently deployable. We had invented polymorphic dispatch. We had invented objects.
8. Architecture must be flexible enough to adapt to the size of the problem.



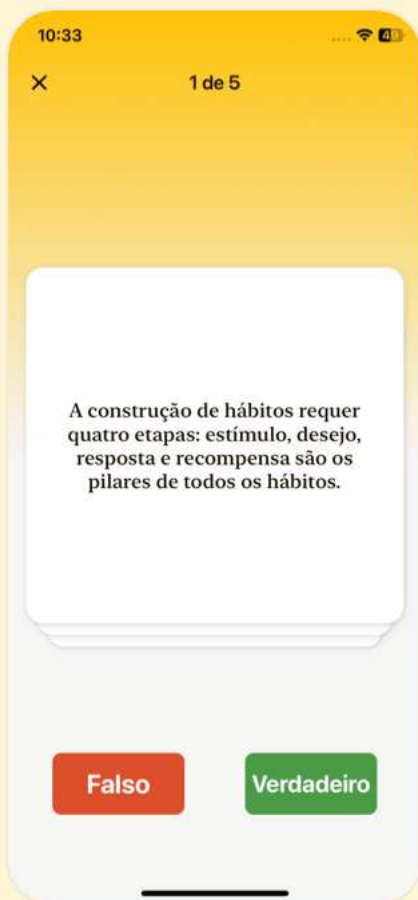


Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Escanear para baixar



Arquitetura Limpa. O Guia do Artesao Para Estrutura e Design de Software

Perguntas

Ver no site do Bookey

Capítulo 1 | O Que É Design e Arquitetura?| Perguntas e respostas

1.Pergunta

Qual é o objetivo principal do design e da arquitetura de software?

Resposta:O objetivo principal do design e da arquitetura de software é minimizar os recursos humanos necessários para construir e manter o sistema, garantindo que o esforço requerido para atender às necessidades dos clientes seja mantido baixo ao longo da vida útil do sistema.

2.Pergunta

Como o autor define a diferença entre design e arquitetura?

Resposta:O autor afirma que não há diferença entre design e arquitetura; ambos englobam estruturas de alto nível e

Mais Livros Grátis



Escaneie para baixar



Ouça-o

detalhes de baixo nível como parte de um todo unificado.

3.Pergunta

Qual é a consequência de um mau design de software segundo o autor?

Resposta:Um mau design de software leva a uma complexidade crescente, custos mais altos e baixa produtividade, causando, em última análise, frustração tanto para desenvolvedores quanto para executivos.

4.Pergunta

O que o autor sugere sobre a natureza do código bagunçado?

Resposta:O autor sugere que o código bagunçado acaba atrasando o desenvolvimento, contradizendo a crença comum de que ele permite uma entrega inicial mais rápida. Um código mais limpo leva a uma maior produtividade e menores custos a longo prazo.

5.Pergunta

Que lição o autor tira da história da Tartaruga e da Lebre?

Resposta:A história destaca que a autoconfiança, semelhante

Mais Livros Grátis



Escaneie para baixar



Ouça-o

à da lebre, pode levar os desenvolvedores a negligenciar a importância de um código limpo e disciplinado, acreditando que podem consertá-lo mais tarde.

6.Pergunta

O que o experimento de Jason Gorman demonstra sobre o Desenvolvimento Orientado a Testes (TDD)?

Resposta:O experimento de Gorman mostra que usar TDD resulta em tempos de conclusão mais rápidos e em uma produtividade melhorada em comparação a não usá-lo, reforçando a noção de que fazer as coisas bem pode, na verdade, aumentar a velocidade.

7.Pergunta

Como os desenvolvedores podem evitar cair na armadilha da autoconfiança em suas habilidades de codificação?

Resposta:Os desenvolvedores podem evitar essa armadilha levando a qualidade do software a sério, reconhecendo as complexidades de seu trabalho e comprometendo-se a manter uma Arquitetura Limpa e um bom design desde o início.

8.Pergunta

O que o autor implica ser necessário para uma

Mais Livros Grátis



Escaneie para baixar



Ouça-o

arquitetura de software eficaz?

Resposta: Uma arquitetura de software eficaz requer uma compreensão dos princípios que governam a qualidade do design, incluindo considerações a longo prazo para manutenção e flexibilidade.

9.Pergunta

Qual é a mensagem geral que o autor transmite sobre os esforços de desenvolvimento de software?

Resposta: O autor transmite que, ao contrário da crença popular, a única maneira de alcançar velocidade e eficiência no desenvolvimento de software é priorizar um bom design e uma boa arquitetura desde o início.

10.Pergunta

O que se entende por 'assinatura de uma bagunça' na produção de software?

Resposta: A 'assinatura de uma bagunça' refere-se ao declínio observável na produtividade e ao aumento nos custos que ocorrem quando os esforços de programação são descoordenados, caóticos e carecem de disciplina de design.



adequada.

Capítulo 2 | Uma História de Dois Valores|

Perguntas e respostas

1.Pergunta

Quais são os dois valores fundamentais que todo sistema de software oferece aos stakeholders?

Resposta:Comportamento e estrutura.

2.Pergunta

Por que muitos programadores acreditam erroneamente que seu trabalho se resume a corrigir bugs e implementar requisitos?

Resposta:Eles geralmente ignoram a importância da arquitetura de software, pensando que sua única responsabilidade é garantir que o sistema se comporte corretamente de acordo com as especificações.

3.Pergunta

O que o termo 'arquitetura' se refere no contexto de software?

Resposta:Arquitetura refere-se ao design estrutural do software que permite que seja fácil de mudar, preservando

Mais Livros Grátis



Escaneie para baixar



Ouça-o

assim sua natureza 'flexível'.

4.Pergunta

Qual é a principal diferença entre 'escopo' e 'forma' no desenvolvimento de software?

Resposta:Escopo refere-se ao tamanho ou à extensão de uma mudança, enquanto forma refere-se a como essa mudança se encaixa na arquitetura existente. A dificuldade em implementar mudanças está muitas vezes relacionada a desajustes entre os recursos solicitados e a arquitetura do software.

5.Pergunta

Qual valor os gerentes de negócios normalmente priorizam: comportamento ou arquitetura? Por que isso é um erro?

Resposta:Eles frequentemente priorizam o comportamento porque é urgente e importante para a funcionalidade imediata. No entanto, isso é um erro, pois se a arquitetura for negligenciada, mudanças futuras se tornarão cada vez mais difíceis e caras, afetando, em última análise, a viabilidade a longo prazo do sistema.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

6.Pergunta

Como a Matriz de Eisenhower categoriza a importância e a urgência no desenvolvimento de software?

Resposta:1. Urgente e Importante 2. Não Urgente, mas Importante 3. Urgente, mas Não Importante 4. Não Urgente e Não Importante. A arquitetura se encaixa nas duas primeiras categorias, indicando que é crítica para o sucesso a longo prazo.

7.Pergunta

Qual é o argumento central sobre a importância da arquitetura de software em relação ao comportamento?

Resposta:Uma arquitetura bem projetada (embora possa parecer menos urgente) é crucial para permitir que os sistemas se adaptem às mudanças nos requisitos, enquanto um sistema que foca somente no comportamento imediato pode tornar-se muito rígido e, eventualmente, inútil.

8.Pergunta

Qual é a responsabilidade dos desenvolvedores de software e arquitetos em relação à arquitetura?

Resposta:Eles são responsáveis por defender princípios



arquiteturais sólidos, enfatizando a necessidade de manter uma estrutura saudável, mesmo quando comportamentos imediatos são priorizados pela gestão.

9.Pergunta

O que a frase 'lute pela arquitetura' implica?

Resposta:Ela enfatiza a necessidade de os desenvolvedores de software defenderem a importância de uma arquitetura consistente e flexível em meio às pressões para entregar funcionalidade imediata.

10.Pergunta

Por que o autor argumenta que 'a única maneira de ir rápido é ir bem'?

Resposta:Esta afirmação ilustra que manter código limpo e bem estruturado permitirá um desenvolvimento mais rápido e eficiente a longo prazo, contrariando a concepção comum de que apressar o código pode economizar tempo.

Capítulo 3 | Visão Geral dos Paradigmas| Perguntas e respostas

1.Pergunta

O que é programação estruturada e por que é

Mais Livros Grátis



Escaneie para baixar



Ouça-o

significativa no desenvolvimento de software?

Resposta: A programação estruturada impõe disciplina na transferência direta de controle dentro de programas. Introduzida por Edsger Dijkstra, substituiu saltos irrestritos (como sentenças goto) por construções estruturadas como if/then/else e loops (do/while). Esse paradigma é significativo pois melhora a clareza e a manutenibilidade do programa, promovendo uma abordagem disciplinada ao fluxo de controle.

2.Pergunta

Como a programação orientada a objetos interage com os princípios da programação estruturada?

Resposta: A programação orientada a objetos, descoberta por Dahl e Nygaard, baseia-se na programação estruturada ao impor disciplina na transferência indireta de controle através de construções como classes e métodos. Ela permite que variáveis locais persistam após a execução da função, o que apoia o polimorfismo e a encapsulação, melhorando ainda



mais o design de software ao promover reutilização e reduzir complexidade.

3.Pergunta

O que define a programação funcional e que disciplina única ela introduz?

Resposta:A programação funcional, que tem origem no trabalho de Alonzo Church sobre cálculo Lambda, enfatiza a imutabilidade — isso significa que os valores simbólicos não mudam e, portanto, geralmente carece de declarações de atribuição. Isso introduz uma disciplina rigorosa sobre atribuições, resultando em um código mais previsível e menos propenso a efeitos colaterais, permitindo um raciocínio mais fácil sobre os programas e promovendo uma abordagem funcional para a resolução de problemas.

4.Pergunta

Que padrão abrangente os três paradigmas de programação revelam sobre capacidades e controle na programação?

Resposta:Os três paradigmas — estruturada, orientada a objetos e funcional — consistentemente removem certas

Mais Livros Grátis



Escaneie para baixar



Ouça-o

capacidades dos programadores enquanto impõem novas práticas disciplinares. Em vez de adicionar capacidades, cada um deles impõe restrições (como eliminar sentenças goto, forçar encapsulação ou limitar atribuições) para aumentar a clareza e a manutenibilidade do código. Esse padrão indica que um design eficaz de software frequentemente resulta da imposição de estrutura e disciplina, em vez de liberdade irrestrita.

5.Pergunta

Por que os paradigmas introduzidos no texto podem ser os únicos paradigmas de programação significativos que veremos?

Resposta:Os paradigmas apresentados — estruturada, orientada a objetos e funcional — emergiram em um curto intervalo de tempo, entre o final da década de 1950 até 1968, e desde então não surgiram novos paradigmas significativos. Isso sugere que a exploração dos limites e princípios fundamentais da programação alcançou saturação, levando-nos a entender que os avanços futuros



provavelmente serão construídos sobre esses paradigmas estabelecidos, em vez de introduzirem abordagens fundamentalmente novas.

6.Pergunta

O que o autor sugere sobre a necessidade de disciplina na arquitetura de software derivada desses paradigmas?

Resposta:O autor sugere que a disciplina é essencial para manter uma arquitetura limpa, que está alinhada com os princípios dos três paradigmas de programação. A remoção de certas liberdades (como saltos irrestritos, ponteiros de função e atribuições) serve para guiar os desenvolvedores em direção a escolhas arquitetônicas melhores, promovendo sistemas que são gerenciáveis, adaptáveis e, em última análise, mais fáceis de modificar à medida que os requisitos evoluem.

7.Pergunta

Como os princípios dos paradigmas de programação influenciam o desenvolvimento da arquitetura de software?

Resposta:Os princípios derivados dos paradigmas de

Mais Livros Grátis



Escaneie para baixar



Ouça-o

programação influenciam a arquitetura de software ao moldar como as funções são organizadas, forçando a separação de componentes e gerenciando dados de forma mais eficaz. Por exemplo, o uso de polimorfismo é crítico para a criação de arquiteturas modulares que facilitam mudanças, enquanto a programação funcional ajuda no controle de acesso a dados, impulsionando coletivamente o design em direção a sistemas mais robustos e manuteníveis.

Mais Livros Grátis



Escaneie para baixar



Ouça-o



Ler, Compartilhar, Empoderar

Conclua Seu Desafio de Leitura, Doe Livros para Crianças Africanas.

O Conceito



Esta atividade de doação de livros está sendo realizada em conjunto com a Books For Africa. Lançamos este projeto porque compartilhamos a mesma crença que a BFA: Para muitas crianças na África, o presente de livros é verdadeiramente um presente de esperança.

A Regra



Ganhe 100 pontos



Resgate um livro



Doe para a África

Seu aprendizado não traz apenas conhecimento, mas também permite que você ganhe pontos para causas beneficentes! Para cada 100 pontos ganhos, um livro será doado para a África.

Teste gratuito com Bookey



Capítulo 4 | Programação Estruturada| Perguntas e respostas

1.Pergunta

Qual foi o problema significativo que Edsger Dijkstra identificou na programação?

Resposta:Dijkstra reconheceu que programar é difícil e que os programadores muitas vezes não o fazem bem, uma vez que programas complexos contêm detalhes demais para o cérebro humano gerenciar sem ajuda.

2.Pergunta

Como Dijkstra propôs resolver os desafios enfrentados na programação?

Resposta:Ele sugeriu aplicar a disciplina matemática da prova à programação, imaginando uma hierarquia de postulados e teoremas semelhante à matemática euclidiana, que os programadores poderiam usar para provar a correção de seu código.

3.Pergunta

Quais estruturas de controle Dijkstra considerou

Mais Livros Grátis



Escaneie para baixar



Ouça-o

essenciais para criar módulos comprováveis?

Resposta: Dijkstra identificou que apenas um conjunto limitado de estruturas de controle—sequência, seleção (se/então/caso contrário) e iteração (faça/enquanto)—é capaz de permitir que os módulos sejam decompostos em unidades menores e comprováveis.

4. Pergunta

Qual foi o impacto da carta de Dijkstra intitulada "Go To Statement Considered Harmful"?

Resposta: A carta gerou um debate significativo na comunidade de programação sobre o uso de instruções goto, levando à sua remoção gradual das linguagens de programação modernas e ao surgimento da programação estruturada como um paradigma dominante.

5. Pergunta

O que se entende por decomposição funcional na programação estruturada?

Resposta: A decomposição funcional na programação estruturada permite que grandes problemas sejam divididos



em funções menores e manejáveis, que podem, então, ser decompostas recursivamente em funções ainda mais detalhadas, facilitando testes e compreensão.

6.Pergunta

O que Dijkstra concluiu sobre provas formais na programação?

Resposta:A aspiração de Dijkstra por uma hierarquia euclidiana completa de funções comprováveis acabou não sendo cumprida; a maioria dos programadores hoje não vê provas formais como práticas ou necessárias para produzir software de alta qualidade.

7.Pergunta

Como o desenvolvimento de software se relaciona à ciência, segundo Dijkstra?

Resposta:O desenvolvimento de software, assim como a ciência, não pode fornecer provas definitivas de correção. Em vez disso, demonstra a correção ao tentar provar algo como incorreto, enquanto abraça a ideia de que muitas leis científicas são consideradas verdadeiras até que se prove o



contrário.

8.Pergunta

Qual foi uma lição crítica da experiência de Dijkstra em relação ao teste de software?

Resposta:Dijkstra afirmou que os testes demonstram a presença, mas não a ausência de bugs; assim, embora os testes possam mostrar se um programa está incorreto, não podem provar que um programa está correto.

9.Pergunta

O que a programação estruturada exige das práticas de programação, segundo os princípios discutidos?

Resposta:A programação estruturada exige disciplina e restringe o uso de transferências de controle sem restrições, encorajando os programadores a escreverem códigos mais limpos e mais fáceis de manter através do uso de estruturas sequenciais, selecionadas e iteradas.

10.Pergunta

Por que a programação estruturada é valiosa hoje?

Resposta:A programação estruturada continua a ser valiosa porque ajuda a criar funções pequenas e comprováveis que

Mais Livros Grátis



Escaneie para baixar



Ouça-o

podem ser testadas, garantindo que sejam manejáveis e possam ser validadas em relação a comportamentos esperados.

Capítulo 5 | Programação Orientada a Objetos| Perguntas e respostas

1.Pergunta

Qual é uma definição satisfatória de Programação Orientada a Objetos (POO)?

Resposta:Programação Orientada a Objetos (POO) é, efetivamente, a aplicação combinada de dados e funções, juntamente com os princípios de encapsulamento, herança e polimorfismo.

2.Pergunta

Por que o encapsulamento é importante na Programação Orientada a Objetos?

Resposta:O encapsulamento é vital porque cria uma barreira que separa um conjunto coeso de dados e funções, ocultando assim os mecanismos internos e expondo apenas o que é necessário para o mundo exterior.

3.Pergunta

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Como o encapsulamento na programação orientada a objetos difere daquele em linguagens de programação procedural como C?

Resposta:Embora o encapsulamento exista em linguagens procedurais como C através do uso de arquivos de cabeçalho e declarações de structs, linguagens POO como C++ quebram o encapsulamento perfeito ao exigir que as variáveis de membro sejam declaradas em arquivos de cabeçalho, levando a um encapsulamento reduzido.

4.Pergunta

O que a herança proporciona nas linguagens orientadas a objetos?

Resposta:A herança facilita a re-declaração de funções e variáveis, permitindo a reutilização de código e a criação de hierarquias em estruturas de dados. No entanto, apresenta desafios, como a complicação de dependências e o aumento da sobrecarga de manutenção.

5.Pergunta

Como o polimorfismo funciona como uma característica da Programação Orientada a Objetos?

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Resposta:O polimorfismo permite que objetos sejam tratados como instâncias de sua classe pai, permitindo uma chamada de métodos mais dinâmica, onde a implementação específica que é executada é determinada em tempo de execução.

6.Pergunta

Por que o polimorfismo é considerado poderoso no design de software?

Resposta:O polimorfismo permite que os sistemas sejam mais flexíveis e mantíveis; torna a integração de novos dispositivos de entrada/saída ou recursos relativamente fácil, sem exigir mudanças extensivas no código existente.

7.Pergunta

O que é inversão de dependência e por que é significativo?

Resposta:Inversão de dependência é um princípio que permite que a direção das dependências do código-fonte seja invertida na estrutura de um programa, o que possibilita maior flexibilidade e design modular, promovendo o desenvolvimento independente dos componentes do sistema.



8.Pergunta

Os princípios de POO podem ser vistos como restritivos?

Se sim, como?

Resposta:Sim, os princípios de POO podem ser vistos como restritivos porque impõem disciplina aos desenvolvedores ao exigir princípios estruturais como encapsulamento e herança, impedindo-os de empregar métodos potencialmente menos organizados, mas mais simples.

9.Pergunta

Como a arquitetura POO suporta a implantação e o desenvolvimento independentes de componentes de software?

Resposta:Ao estabelecer interfaces claras entre os componentes, a arquitetura POO permite que cada parte seja desenvolvida, testada e implantada de forma independente, o que melhora significativamente a manutenibilidade e escalabilidade dos sistemas de software.

10.Pergunta

Qual pode ser considerada a mensagem principal sobre a essência da Programação Orientada a Objetos?

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Resposta:A essência da Programação Orientada a Objetos é a capacidade de gerenciar complexidade e dependências por meio de abstrações, que, em última análise, promovem sistemas de software mais robustos, adaptáveis e manuteníveis.

Chapter 6 | Functional Programming| Perguntas e respostas

1.Pergunta

What is the concept behind functional programming and how does it differ from other programming paradigms?

Resposta:Functional programming is based on the concept of immutability, meaning that once variables are defined, they cannot be changed. This differs from procedural or object-oriented programming where variables can be mutable, creating potential issues like race conditions in concurrent environments. In functional programming, computations are achieved using functions and data is transformed without altering the original state, leading to clearer, less error-prone



code.

2.Pergunta

Why is immutability considered an important architectural consideration?

Resposta:Immutability is crucial in architecture because it prevents race conditions, deadlocks, and other concurrency issues. If no variables are mutable, the complexities associated with shared state in multi-threaded applications are eliminated, resulting in more robust and predictable systems.

3.Pergunta

What are some strategies to implement immutability in software architecture?

Resposta:One approach is to segregate components into immutable and mutable elements, allowing immutable components to perform tasks functionally without mutable state disrupting them. Another strategy is to use transactional memory to protect mutable variables from concurrent updates effectively. This ensures that mutable state is



handled under strict controlled conditions.

4.Pergunta

How does event sourcing relate to immutability?

Resposta:Event sourcing is a method that maintains the state of an application by storing only the sequence of events that lead to that state instead of the state itself. This approach further exemplifies immutability, as it allows for all historical data to be preserved, minimizing mutable state and avoiding issues like concurrent updates since the historical data cannot change.

5.Pergunta

What is the significance of the comparison between Java and Clojure in handling variables?

Resposta:The comparison highlights the core difference in variable handling; Java employs mutable variables which can lead to mutable state issues, whereas Clojure exemplifies functional programming principles by using immutable variables. Clojure's functions operate on input data without modifying it, showcasing the benefits of functional

Mais Livros Grátis



Escaneie para baixar



Ouça-o

programming in avoiding common concurrency problems.

6.Pergunta

How does the concept of 'never-ending lists' in functional programming provide efficiency?

Resposta:In functional programming, constructions like 'never-ending lists' allow for lazy evaluation. Only the required elements are computed and accessed when needed, which saves resources and improves efficiency. This means computations are deferred until necessary, minimizing unnecessary processing or memory usage during execution.

7.Pergunta

What are the potential limitations of achieving full immutability in software applications?

Resposta:While immutability can significantly enhance the robustness of applications, achieving it might not be practical due to limitations in storage and processing resources.

Developers may need to compromise by utilizing mutable states in controlled ways or applying patterns like event sourcing to maintain performance without requiring infinite

Mais Livros Grátis



Escaneie para baixar



Ouça-o

resources.

8.Pergunta

Why might developers underestimate the importance of architecture over immediate functionality?

Resposta: Developers often prioritize urgency in delivering features for immediate market needs, overlooking the long-term importance of architecture. This can lead to technical debt as the structural organization of the code suffers, making future changes more complex and costly, ultimately reducing overall productivity.

9.Pergunta

How does the discipline of structured programming relate to functional programming?

Resposta: Structured programming imposes a discipline on the control flow of programs, requiring clear and logical structure which aids in program correctness. Functional programming builds on this by imposing discipline on variable assignments, further enforcing clarity and predictability in code execution, ultimately leading to simpler



maintenance and reduced errors.

10.Pergunta

What can be said about the evolution and adaptability of software architecture according to the text?

Resposta: The evolution of software architecture reflects a gradual adaptation towards imposing more discipline on coding practices. Despite technological advancements, fundamental principles of software remain static over decades. The ability to manage complexity through structures like immutability, functional programming, and other paradigms enhances simplifying code maintenance, yet the underlying constructs of programming have not dramatically transformed.

Mais Livros Grátis



Escaneie para baixar



Ouça-o



As melhores ideias do mundo desbloqueiam seu potencial

Essai gratuit avec Bookey



Escanear para baixar



Capítulo 7 | SRP: O Princípio da Responsabilidade Única| Perguntas e respostas

1.Pergunta

O que é o Princípio da Responsabilidade Única (SRP) conforme definido no capítulo?

Resposta:O Princípio da Responsabilidade Única afirma que um módulo deve ter uma, e apenas uma, razão para mudar. Isso significa que um módulo deve ser responsável por um, e somente um, ator.

2.Pergunta

Por que o nome 'Princípio da Responsabilidade Única' é considerado inadequado?

Resposta:O nome pode induzir os programadores a pensar que significa que cada módulo deve fazer apenas uma coisa, em vez de entender que se refere a um módulo ter uma razão para mudar.

3.Pergunta

Qual é um exemplo de violação do SRP em um módulo de software?

Resposta:Um exemplo é uma classe Funcionário com



métodos para calcular salários, relatar horas e salvar dados, servindo a três atores diferentes: o CFO, COO e CTO. Isso leva a um acoplamento indesejado entre funções responsáveis por diferentes atores.

4.Pergunta

Qual é um sintoma comum de violação do SRP discutido no capítulo?

Resposta:Um sintoma comum é a duplicação acidental, onde alterações feitas para um ator afetam inadvertidamente outro ator devido a código compartilhado em um único módulo.

5.Pergunta

Como o capítulo explica os riscos de mesclar arquivos de origem que violam o SRP?

Resposta:Mesclagens podem ocorrer quando diferentes desenvolvedores de equipes responsáveis por diferentes atores alteram o mesmo arquivo de origem, levando a potenciais conflitos, erros e um risco aumentado para as partes interessadas.

6.Pergunta

Quais soluções o capítulo sugere para aderir ao SRP?

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Resposta:As soluções incluem separar funcionalidades em diferentes classes, manter métodos relacionados e estruturas de dados coesas, e utilizar padrões como Facade para gerenciar a complexidade e as interações.

7.Pergunta

Qual pode ser uma desvantagem de separar funções em diferentes classes para manter o SRP?

Resposta:A desvantagem é que os desenvolvedores podem precisar rastrear e instanciar múltiplas classes, aumentando a complexidade em termos de código repetitivo e gerenciamento.

8.Pergunta

Como o SRP se relaciona com outros princípios de design?

Resposta:O SRP reaparece em níveis mais altos da arquitetura de software, como o Princípio do Fechamento Comum no nível de componente e afeta o Eixo de Mudança no nível arquitetônico.

9.Pergunta

O que o capítulo implica sobre a importância do SRP no

Mais Livros Grátis



Escaneie para baixar



Ouça-o

design de software?

Resposta: Aderir ao SRP é crucial, pois minimiza interdependências entre diferentes módulos, tornando o software mais fácil de mudar e gerenciar para várias partes interessadas.

10.Pergunta

Você pode resumir a principal conclusão sobre o SRP do capítulo?

Resposta: A principal conclusão é que entender e aplicar o Princípio da Responsabilidade Única ajuda a criar módulos de software que são coesos e fracamente acoplados, reduzindo a probabilidade de consequências indesejadas quando mudanças são feitas.

Chapter 8 | OCP: The Open-Closed Principle| Perguntas e respostas

1.Pergunta

What is the Open-Closed Principle (OCP)?

Resposta: The Open-Closed Principle states that a software artifact should be open for extension but closed for modification. This means that the



behavior of software can be extended without changing existing code.

2.Pergunta

How does the OCP relate to software architecture?

Resposta:The OCP emphasizes the importance of designing software systems in a way that allows for easy extensions without requiring significant modifications, which is crucial for maintaining a sustainable and adaptable architecture.

3.Pergunta

Can you provide a thought experiment to illustrate the OCP?

Resposta:Imagine a system displaying financial data. If stakeholders request a change to this data's format for a printed report, a well-architected system would allow this change with minimal or no alteration to existing code, demonstrating the OCP's efficacy.

4.Pergunta

What role does the Single Responsibility Principle (SRP) play in achieving OCP?

Resposta:By applying the SRP, we can separate different



responsibilities within a system, which ensures that changes for one reason do not adversely affect others, supporting easier extensions as per the OCP.

5.Pergunta

How do dependencies in a software architecture relate to the OCP?

Resposta:Dependencies should be organized such that higher-level components are protected from changes in lower-level components. This way, modifying lower-level components does not necessitate changes in higher-level ones.

6.Pergunta

What is the significance of unidirectional dependencies in software components?

Resposta:Unidirectional dependencies help maintain control over a system's architecture; if component A should be protected from changes in component B, then B must depend on A, not the other way around.

7.Pergunta

Why is the Interactor considered central to the

Mais Livros Grátis



Escaneie para baixar



Ouça-o

architecture in the context of the OCP?

Resposta: The Interactor contains the business rules and is positioned to be least affected by changes in other components, making it a critical element in preserving the architectural integrity of the system.

8.Pergunta

What does directional control entail in the context of OCP and software design?

Resposta: Directional control involves structuring dependencies correctly so that they point towards higher-level components. This prevents unnecessary impacts from changes in lower-level components.

9.Pergunta

How does information hiding contribute to compliance with the OCP?

Resposta: Information hiding protects components from needing to know the inner workings of other components, thereby reducing transitive dependencies and allowing for extensions without risk of breaking existing functionality.



10.Pergunta

In what way does the OCP aim to minimize the impact of change on a system?

Resposta: The OCP minimizes the impact of change by promoting a design that allows new functionality to be added through new code rather than altering existing code, reducing potential bugs and instability in the system.

Capítulo 9 | LSP: O Princípio da Substituição de Liskov| Perguntas e respostas

1.Pergunta

O que é o Princípio da Substituição de Liskov (LSP)?

Resposta: O Princípio da Substituição de Liskov afirma que se para cada objeto o_1 do tipo S , existe um objeto o_2 do tipo T tal que para todos os programas P definidos em termos de T , o comportamento de P não é alterado quando o_1 é substituído por o_2 , então S é um subtipo de T .

2.Pergunta

Você pode fornecer um exemplo de LSP no design de software?

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Resposta:Um exemplo é o uso de uma classe chamada Licença com subtipos LicençaPessoal e LicençaEmpresarial. O aplicativo de cobrança chama o método calcFee() no tipo Licença e, se ele funcionar corretamente sem saber se está lidando com uma LicençaPessoal ou LicençaEmpresarial, então os subtipos estão em conformidade com o LSP.

3.Pergunta

Qual é o problema do Quadrado/Retângulo e como ele viola o LSP?

Resposta:O problema do Quadrado/Retângulo ocorre porque um Quadrado, que requer que a largura e a altura mudem juntas, é tratado como um subtipo de Retângulo, que permite mudanças independentes na largura e na altura. Isso leva a inconsistências quando métodos que esperam o comportamento de um Retângulo recebem um Quadrado, violando a substituíbilidade.

4.Pergunta

Como a compreensão do LSP evoluiu de ser sobre herança para abranger um design de software mais amplo?

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Resposta:Inicialmente, o LSP era visto como um princípio que guiava a herança, mas se expandiu para abranger o design de interfaces e implementações, garantindo que os sistemas se baseiem em interfaces bem definidas e que as implementações sejam substituíveis.

5.Pergunta

O que pode acontecer arquitetonicamente quando o LSP é violado?

Resposta:Violação do LSP pode levar a sistemas se tornarem excessivamente complicados, exigindo mecanismos adicionais (como condicionais para diferentes tipos) que degradam a arquitetura, aumentam erros e complicam a manutenção.

6.Pergunta

Você pode descrever um exemplo prático de violação do LSP com um sistema de despacho de táxis?

Resposta:Em um serviço de despacho de táxis, se uma API REST para despacho não conformar a um formato consistente entre os serviços (por exemplo, táxis Acme



usando 'dest' em vez de 'destino'), um código especial lidando com essas exceções precisa ser adicionado, complicando a arquitetura e levando a potenciais erros.

7.Pergunta

Qual é a principal lição sobre o impacto de aderir ao LSP durante o desenvolvimento de software?

Resposta:Aderir ao LSP leva a um design de sistema mais limpo e simples, onde os componentes podem ser facilmente trocados sem modificar o código existente, melhorando a manutenibilidade, reduzindo erros e garantindo uma arquitetura mais robusta.

8.Pergunta

Por que entender o LSP é crucial para arquitetos de software?

Resposta:Entender o LSP ajuda os arquitetos a projetar sistemas que são adaptáveis a mudanças sem um redevelop significativo. Garante que os componentes sejam intercambiáveis, minimizando as interseções entre funcionalidades diferentes e, assim, melhorando a



integridade geral do sistema.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Ad



Escanear para baixar



Experimente o aplicativo Bookey para ler mais de 1000 resumos dos melhores livros do mundo

Desbloqueie **1000+** títulos, **80+** tópicos

Novos títulos adicionados toda semana

Product & Brand

Liderança & Colaboração

Gerenciamento de Tempo

Relacionamento & Comunicação

Estratégia de Negócios

Criatividade

Memórias

Conheça a Si Mesmo

Psicologia

Empreendedorismo

História Mundial

Comunicação entre Pais e Filhos

Autocuidado

Mente

Visões dos melhores livros do mundo

amento
pos

Os 7 Hábitos das
Pessoas Altamente
Eficazes



Mini Hábitos



Hábitos Atômicos



O Clube das 5
da Manhã



Como Fazer Amigos
e Influenciar
Pessoas



Com
Não



Teste gratuito com Bookey



Chapter 10 | ISP: The Interface Segregation Principle| Perguntas e respostas

1.Pergunta

What is the essence of the Interface Segregation Principle (ISP)?

Resposta:The essence of the Interface Segregation Principle (ISP) is to break down a larger interface into smaller, more specific ones that cater to the needs of different clients. This separation prevents clients from being forced to depend on methods they do not use, thereby reducing unnecessary recompilation and maintaining a clean architecture.

2.Pergunta

How does ISP relate to statically typed languages like Java compared to dynamically typed languages like Python?

Resposta:In statically typed languages like Java, dependencies are explicit and require declarations, which can lead to clients recompiling when unrelated methods change. In dynamically typed languages like Python, dependencies



are inferred at runtime, allowing for greater flexibility and fewer coupling issues.

3.Pergunta

Why is it harmful for a system to depend on modules that contain more than necessary?

Resposta:It's harmful because such dependencies can lead to unnecessary complications, like forced recompiling and redeploying of seemingly unrelated components when changes are made, which can increase the risk of introducing bugs and reducing the overall stability of the system.

4.Pergunta

Illustrate how the ISP can prevent unnecessary recompilation with an example.

Resposta:Imagine a class OPS with methods op1, op2, and op3. If User1 only needs op1, without ISP, any change in op2 or op3 forces User1 to recompile. With ISP, User1 would only depend on interface U1Ops that exposes op1, isolating User1 from changes in op2 and op3.

5.Pergunta

What broader architectural concerns are tied to the ISP?

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Resposta: The ISP extends beyond interfaces to architectural dependencies. An architect must ensure that components don't depend on frameworks or modules with features they don't use. This helps prevent issues like unnecessary redeployments or systemic failures due to unrelated feature changes.

6. Pergunta

What is the lesson learned from not adhering to the ISP in system architecture?

Resposta: The lesson is that depending on modules with unrelated functionality can create unforeseen complications and dependencies, leading to increased maintenance costs and limited flexibility in future system changes.

7. Pergunta

How does the ISP align with sustainable software development practices?

Resposta: The ISP promotes clean and maintainable architecture by reducing unnecessary dependencies, allowing teams to adapt and extend the software more efficiently as



requirements evolve, essential for sustainable software development.

8.Pergunta

What implications does the ISP have for software modularity?

Resposta: The ISP promotes high modularity by encouraging the creation of focused interfaces, which makes systems easier to understand, test, and maintain. Each module becomes a well-defined component that can be developed or modified independently.

9.Pergunta

Why should software architects consider the ISP in their designs?

Resposta: Software architects should consider the ISP to create architectures that are resilient to change, minimize rework and defects, and enable easier adaptation in response to evolving business requirements.

Chapter 11 | DIP: The Dependency Inversion Principle| Perguntas e respostas

1.Pergunta

Mais Livros Grátis



Escaneie para baixar



Ouça-o

What is the Dependency Inversion Principle (DIP)?

Resposta: The Dependency Inversion Principle (DIP)

asserts that source code dependencies should point only to abstractions (like interfaces or abstract classes) and not to concrete implementations. This approach aims to create more flexible systems by minimizing dependencies on concrete classes.

2.Pergunta

Why should source code dependencies refer to abstractions?

Resposta: Referencing abstractions allows systems to be less affected by changes in concrete implementations, leading to higher stability and flexibility in software design.

3.Pergunta

How does DIP help in managing changes in software?

Resposta: DIP facilitates handling changes by defining clear boundaries between high-level abstractions and low-level details, ensuring that changes to one part of the system do not require changes across the entire architecture.



4.Pergunta

What role do stable abstractions play in software architecture according to DIP?

Resposta:Stable abstractions are less volatile than concrete implementations; hence, they should be favored. Changes to abstract interfaces should ideally not necessitate changes to their concrete implementations, thereby reducing the risk of breakages during development.

5.Pergunta

What is an example of a concrete class that can be used without violating DIP?

Resposta:The String class in Java can be seen as a concrete class that is stable and utilized without violating DIP since its changes are rare and tightly controlled.

6.Pergunta

What is an Abstract Factory in the context of DIP?

Resposta:An Abstract Factory is a design pattern that provides an interface for creating objects but allows subclasses to alter the type of objects that will be created. In DIP, it helps manage dependencies without directly



depending on specific concrete classes.

7.Pergunta

How does the use of an Abstract Factory relate to the flow of control and dependencies?

Resposta:In an Abstract Factory, the flow of control might use an interface to instantiate concrete implementations, allowing dependencies to flow in the direction of abstractions rather than concretions, thus maintaining the principles of the Dependency Inversion.

8.Pergunta

What is the significance of not referencing volatile concrete classes according to DIP?

Resposta:Not referencing volatile concrete classes helps prevent the introduction of dependencies that are likely to change frequently, which in turn enhances the maintainability and stability of the codebase.

9.Pergunta

What conclusion can be drawn about the importance of managing dependencies in software architecture?

Resposta:Managing dependencies effectively through



principles such as DIP is crucial for creating clean, maintainable, and flexible software architectures by ensuring that changes in low-level systems do not cascade into high-level business rules.

10.Pergunta

What does the term 'volatile concrete components' imply in the context of DIP?

Resposta:Volatile concrete components are those parts of the software that are frequently changed or modified; under DIP, the goal is to minimize dependencies on these components to ensure the overall stability of the system.

Chapter 12 | Components| Perguntas e respostas

1.Pergunta

What are components in software architecture?

Resposta:Components are the fundamental units of deployment in software systems. They can be deployed independently, which means they can be developed and maintained in isolation. In languages like Java, components are typically represented as



jar files, in .NET as DLLs, and in Ruby as gems.

They can also be packaged into archives like .war files or deployed as independent plugins.

2.Pergunta

Why is it essential for components to be independently deployable?

Resposta:Independently deployable components allow for more flexibility and agility in software development. If a component can be updated or replaced without affecting the entire system, it minimizes the risk of introducing bugs across the application and facilitates easier maintenance and faster feature deployments.

3.Pergunta

What historical challenges did early programmers face concerning memory usage?

Resposta:Early programmers had to manage memory manually, specifying memory addresses and dealing with non-relocatable programs. Compiling libraries required integrating source code directly as memory was limited,



leading to slow compilation times. They faced challenges when application sizes grew, necessitating effective memory management beyond their initial allocations.

4.Pergunta

How did the introduction of relocatable binaries improve the software development process?

Resposta:Relocatable binaries allowed code to be loaded into memory at any address without needing to modify the code itself, significantly speeding up the linking process. This innovation meant that programmers could load only the functions they needed, reducing overall compilation time and enabling more complex applications.

5.Pergunta

What impact did Moore's law have on programming and linking times in the 1990s?

Resposta:As hardware improved under Moore's law with faster disks and cheaper memory, the time required for linking began to decrease to mere seconds for smaller jobs. This advancement allowed developers to adopt a more



dynamic linking approach, paving the way for plugin architectures and rapid development cycles.

6.Pergunta

What is the significance of linking loaders in historical software development?

Resposta:Linking loaders played a crucial role in separating the loading and linking processes in software applications. This allowed for faster execution of programs, as linking could be done ahead of time, reducing load times significantly when executing large applications and allowing for the efficient use of system resources.

7.Pergunta

What problems arose from the large-scale use of linking loaders in the 1960s and 70s?

Resposta:As software systems grew larger, linking loaders began to struggle with speed, taking excessive time to resolve many external references from large libraries stored on slow devices like magnetic tape. This necessitated a more sophisticated design in software development processes to



manage resource usage effectively.

8.Pergunta

What does the chapter highlight as a modern practice in component design?

Resposta:The chapter emphasizes that current best practices in component design focus on creating components that are not only independent but also easily plug-and-play, allowing for enhanced flexibility and modularity in applications, such as using .jar or DLL files as plugins.

9.Pergunta

What does 'the only way to go fast is to go well' mean in the context of software development?

Resposta:This phrase encapsulates the philosophy that maintaining clean, well-structured code is critical for long-term efficiency. Rushing to deliver features without proper architecture leads to messy code, which ultimately slows development and increases difficulty when making changes or adding features.

Mais Livros Grátis



Escaneie para baixar



Ouça-o



Escanear para baixar



Por que o Bookey é um aplicativo indispensável para amantes de livros



Conteúdo de 30min

Quanto mais profunda e clara for a interpretação que fornecemos, melhor será sua compreensão de cada título.



Clipes de Ideias de 3min

Impulsione seu progresso.



Questionário

Verifique se você dominou o que acabou de aprender.



E mais

Várias fontes, Caminhos em andamento, Coleções...

Teste gratuito com Bookey



Chapter 13 | Component Cohesion| Perguntas e respostas

1.Pergunta

What are the three principles of component cohesion discussed in Chapter 13 of Clean Architecture?

Resposta: The three principles of component cohesion discussed are: 1. The Reuse/Release Equivalence Principle (REP) 2. The Common Closure Principle (CCP) 3. The Common Reuse Principle (CRP).

2.Pergunta

How does the Reuse/Release Equivalence Principle (REP) guide component design?

Resposta: The REP states that the granule of reuse should match the granule of release, meaning that classes grouped into components should be cohesive and releasable together. This maximizes compatibility and informs developers about changes when components are updated, thus facilitating better management of component versions.

3.Pergunta

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Can you explain the Common Closure Principle (CCP)?

Resposta: The CCP emphasizes gathering classes that change together for the same reasons into the same component while separating classes that change for different reasons into different components. This minimizes the need for redeployment across multiple components and enhances maintainability by consolidating changes.

4.Pergunta

What is the significance of the Common Reuse Principle (CRP)?

Resposta: The CRP focuses on ensuring that components consist of classes that are reused together, preventing unnecessary dependencies among classes that are not tightly related. It asserts that users of a component should not be forced to depend on unused classes, thus optimizing the reuse process.

5.Pergunta

How does the interplay of REP, CCP, and CRP create tension in component architecture?

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Resposta: These principles often contradict each other: REP and CCP tend to create larger components for better reuse and maintenance, while CRP advocates for smaller, more focused components. This tension requires architects to find a balance based on the current goals of the development team, acknowledging that this balance may shift over time.

6.Pergunta

Why is it important to group classes into components based on cohesive purposes?

Resposta: Grouping classes into cohesive components ensures that the classes work together harmoniously under a common theme, which facilitates easier management of changes, reduces complexity during updates, and enhances the overall maintainability of the system.

7.Pergunta

In what way might the focus on component cohesion evolve as a project matures?

Resposta: As a project matures, the initial focus might prioritize maintainability (Common Closure Principle), but as



the codebase becomes more stable, the emphasis may shift towards reusability (Reuse/Release Equivalence Principle), leading to adjustments in component structure over time.

Chapter 14 | Component Coupling| Perguntas e respostas

1.Pergunta

What is the Acyclic Dependencies Principle (ADP)?

Resposta:The Acyclic Dependencies Principle states that there should be no cycles in the component dependency graph. This means that in a well-designed system, you should be able to track the dependencies of components without ever returning to the starting component. This prevents issues such as the 'morning after syndrome,' where changes made by one developer disrupt the work of another.

2.Pergunta

How does the weekly build method work and what are its downsides?

Resposta:In the weekly build method, developers work in



isolation for most of the week and only integrate their changes at the end of the week. While this allows for focused work, it leads to a significant integration burden at the end of the week. As project size increases, integration can lead to lengthy delays and frustration, spilling over into weekends and affecting overall team efficiency.

3.Pergunta

How can you eliminate dependency cycles in component architecture?

Resposta: You can eliminate dependency cycles by implementing the Dependency Inversion Principle (DIP), which involves creating abstractions or interfaces that decouple dependent components. Alternatively, you can create new components that both conflicting dependencies rely on, thereby breaking the cycle.

4.Pergunta

What is the Stable Dependencies Principle (SDP)?

Resposta: The Stable Dependencies Principle asserts that dependencies should flow towards stable components. A



component that is stable has fewer volatile dependencies and is unlikely to change frequently, while volatile components should not be depended on by stable ones to maintain overall flexibility and avoid disrupting stable functionality.

5.Pergunta

What is meant by stability in component architecture?

Resposta:Stability refers to how likely a component is to change. Stability can be measured by the number of components that depend on it: a stable component has many dependencies, making changes difficult, whereas an unstable component has few or no dependencies, allowing for easy modifications.

6.Pergunta

How do the concepts of stability and abstractness correlate according to the Stable Abstractions Principle?

Resposta:The Stable Abstractions Principle states that a stable component should be abstract and an unstable component should be concrete. This ensures that stable components remain adaptable and flexible to change without



being tied down by unnecessary implementations.

7.Pergunta

Why is it important to allow for independent deployability in components?

Resposta:Independent deployability allows components to be updated or modified without requiring the entire system to be redeployed. This reduces downtime and the effort needed to manage updates, enabling better responsiveness to changes and more efficient development processes.

8.Pergunta

What is the relationship between the Common Closure Principle (CCP) and the Single Responsibility Principle (SRP)?

Resposta:The Common Closure Principle is the component version of the Single Responsibility Principle. While the SRP aims to ensure that a class has one reason to change, the CCP advises that components should only contain classes that change for the same reasons, helping to localize changes and minimize the impact of modifications.

9.Pergunta

Mais Livros Grátis



Escaneie para baixar



Ouça-o

What does the Common Reuse Principle (CRP) state about class organization in components?

Resposta: The Common Reuse Principle states that classes that are reused together should be placed in the same component. Conversely, classes that are not likely to be reused together should not share a component; otherwise, unnecessary dependencies may arise that complicate maintenance and updates.

10.Pergunta

What are the implications of violating the acyclic dependencies principle?

Resposta: Violating the acyclic dependencies principle leads to increased complexity and risks in the architecture, such as the inability to isolate components for testing, difficulty in establishing a clear order of builds, and a tendency to create tight coupling between components, which in turn invites potential integration issues.

Chapter 15 | What Is Architecture?| Perguntas e respostas

1.Pergunta

Mais Livros Grátis



Escaneie para baixar



Ouça-o

What is the primary role of a software architect?

Resposta: A software architect is primarily a programmer who continues to write code while also guiding their team toward a design that maximizes productivity. They do not pull back from coding; their experiences with code help them understand and address issues within the team.

2.Pergunta

What is the main purpose of software architecture?

Resposta: The main purpose of software architecture is to facilitate the development, deployment, operation, and maintenance of a software system. It aims to minimize the lifetime cost of the system and maximize programmer productivity.

3.Pergunta

How does architecture affect the longevity and maintainability of a software system?

Resposta: Good architecture makes a software system easy to understand, develop, maintain, and deploy. It supports future



changes, reducing the technical debt and complexities associated with system evolution.

4.Pergunta

Why is it important to keep options open in software architecture?

Resposta:Keeping options open allows for flexibility in decision-making, enabling teams to adapt to changes as new information becomes available. It prevents premature commitment to specific technologies or implementations that might later prove unsuitable.

5.Pergunta

What are the two major elements into which all software systems can be decomposed?

Resposta:Software systems can be decomposed into two major elements: policy, which consists of business rules and procedures, and details, which include specific implementations like IO devices, databases, and protocols.

6.Pergunta

What is the significance of the statement 'a good software architecture communicates the operational needs of the

Mais Livros Grátis



Escaneie para baixar



Ouça-o

system'?

Resposta: This means that the architecture should provide clear visibility into the system's use cases and features, making it easier for developers to understand requirements and implement solutions effectively.

7.Pergunta

What are two types of value that software has, according to the text?

Resposta: The two types of value that software has are the value of its behavior and the value of its structure.

8.Pergunta

How does an architecture that supports good software maintenance mitigate costs?

Resposta: By separating the system into well-defined components and isolating those components through stable interfaces, good architecture illuminates future pathways for feature addition and reduces the risk of inadvertent breakage.

9.Pergunta

What is an example of device independence from the chapter?

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Resposta: An example of device independence is the transition from using hardware specific methods (like direct calls to a printer) to using operating system abstractions that allow programs to interact with IO devices generically, regardless of the underlying hardware.

10.Pergunta

What is meant by the term 'spelunking' in software maintenance?

Resposta: Spelunking refers to the cost and effort of navigating through existing software to identify the proper place and method for adding new features or fixing defects, often leading to higher risks of unintentional errors.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Ad



Escanear para baixar



App Store
Escolha dos Editores



22k avaliações de 5 estrelas

Feedback Positivo

Afonso Silva

...cada resumo de livro não só
..., mas também tornam o
...divertido e envolvente. O
...tizou a leitura para mim.

Fantástico!



Estou maravilhado com a variedade de livros e idiomas
que o Bookey suporta. Não é apenas um aplicativo, é
um portal para o conhecimento global. Além disso,
ganhar pontos para caridade é um grande bônus!

Brígida Santos

F



O
só
o
O

na Oliveira

...correr as
...ém me dá
...omprar a
...ar!

Adoro!



Usar o Bookey ajudou-me a cultivar um hábito de
leitura sem sobrecarregar minha agenda. O design do
aplicativo e suas funcionalidades são amigáveis,
tornando o crescimento intelectual acessível a todos.

Duarte Costa

Economiza tempo!



O Bookey é o meu apli
crescimento intelectual
perspicazes e lindame
um mundo de conheci

Aplicativo incrível!



Eu amo audiolivros, mas nem sempre tenho tempo para
ouvir o livro inteiro! O Bookey permite-me obter um resumo
dos destaques do livro que me interessa!!! Que ótimo
conceito!!! Altamente recomendado!

Estevão Pereira

Aplicativo lindo



Este aplicativo é um salva-vidas para
de livros com agendas lotadas. Os re
precisos, e os mapas mentais ajudar
o que aprendi. Altamente recomend

Teste gratuito com Bookey



Chapter 16 | Independence| Perguntas e respostas

1.Pergunta

What essential qualities should a good software architecture possess?

Resposta:A good software architecture should support the use cases and operation of the system, facilitate maintenance and development, enable deployment, and leave options open for future changes.

2.Pergunta

How does architecture influence the clear visibility of system behaviors?

Resposta:Good architecture clarifies and exposes system behaviors by structuring the system in a way that makes the intent and use cases visible. Developers can identify the behaviors without searching, as these become prominent elements within the architecture, represented by clearly named classes or functions.

3.Pergunta

Why is architecture important for system operation and

Mais Livros Grátis



Escaneie para baixar



Ouça-o

performance?

Resposta:Architecture determines if the system can handle required loads, like managing 100,000 customers per second or querying big data efficiently. It must be structured to meet performance demands, which can be achieved through methods such as parallelism or service arrangements.

4.Pergunta

What does Conway's Law imply about architecture and team organization?

Resposta:Conway's Law suggests that the architecture of a system will mimic the communication structures of the organization designing it. Therefore, an architecture must facilitate independent actions by development teams, requiring proper partitioning of systems into decoupled components.

5.Pergunta

How should architecture aid in system deployment?

Resposta:Good architecture should simplify deployment, aiming for immediate deployability without complex



configuration. It achieves this through proper component isolation, allowing a system to be deployed effectively after build.

6.Pergunta

What principles help maintain flexible and adaptable architecture?

Resposta:Principles such as the Single Responsibility Principle and the Common Closure Principle encourage separating components that change for different reasons and grouping those that change for the same reasons, thereby enhancing adaptability.

7.Pergunta

What happens if there is duplication in software design, and how should architects handle it?

Resposta:Duplication can lead to challenges if two pieces of code evolve differently over time. Architects should be cautious of merging code structures that seem similar at first glance, as they may diverge. It's essential to assess whether the duplication is true or accidental before eliminating it.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

8.Pergunta

Describe decoupling modes in the context of architecture.

Resposta:Decoupling can occur at different levels: at the source code level (modules that execute in the same address space), at the deployment level (independently deployable units), and at the service level (independent services communicating via network). The choice of decoupling mode may change over a project's life cycle.

9.Pergunta

How does good architecture help manage future changes in a system?

Resposta:By leaving options open for future modifications and separating concerns, good architecture enables easier adaptation to changes in requirements, technology, and team structure, ultimately minimizing costs and maximizing user satisfaction.

10.Pergunta

What is the role of abstraction in maintaining the stability of components in software architecture?

Resposta:Abstraction allows for components to be stable



while remaining flexible. By ensuring that stable components represent high-level policies through abstract classes or interfaces, architects can protect these components from changes in lower-level details.

Chapter 17 | Boundaries: Drawing Lines| Perguntas e respostas

1.Pergunta

What is the primary goal of software architecture as described in the text?

Resposta:The primary goal of software architecture is to minimize the human resources required to build and maintain a system by creating boundaries that allow decisions to be deferred and keep the core business logic free from unnecessary coupling and complications.

2.Pergunta

What does the author mean by 'premature decisions' in software architecture?

Resposta:Premature decisions are those made early in the development process that are unrelated to business



requirements, such as choices about frameworks and technologies that can be delayed until they are truly necessary, preventing them from contaminating the architecture.

3.Pergunta

How can drawing boundaries in architecture help in minimizing development effort?

Resposta:Drawing boundaries allows for separation of concerns, which ensures that components can evolve independently, preventing unnecessary dependencies that complicate changes and can multiply development efforts.

4.Pergunta

Can you provide an example from the text highlighting the issues caused by making premature architectural decisions?

Resposta:The story of company P illustrates this; they built a complex three-tier architecture in anticipation of a server farm, but all deployments ended up on a single server. This inflated their development workload due to unnecessary complexity that ultimately wasn't needed.



5.Pergunta

What lesson does the author's story about FitNesse convey regarding architectural decisions?

Resposta:The FitNesse story showcases the advantage of delaying decisions. By avoiding early choices about web frameworks and databases, the team focused on core features first, allowing them to adapt later without extensive rework. This flexibility resulted in significant time savings.

6.Pergunta

According to the text, what should be the focus when drawing boundaries between components?

Resposta:Boundaries should be drawn between things that matter, such as separating GUI from business rules or business rules from databases, to ensure that changes in one area do not unduly affect another.

7.Pergunta

What is the main takeaway regarding the relationship between business rules and databases?

Resposta:The main takeaway is that business rules should not depend on databases, meaning that they should interact only



through an abstraction that allows for the database to be changed without impacting the business logic.

8.Pergunta

How did the author exemplify the successful implementation of boundaries in software architecture?

Resposta: The author exemplified this through policies in the development of FitNesse, where they successfully delayed decisions regarding the database by implementing an interface that abstracted data access, allowing for flexibility in choosing the database later.

9.Pergunta

Discuss the duality of architectural decisions, as described in the chapter.

Resposta: Architectural decisions should balance minimizing immediate complexity while allowing for future changes.

These decisions need to accommodate evolving user needs and technological advancements without locking the architecture into rigid frameworks that could hamper future development.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

10.Pergunta

How does the text define a successful architecture in terms of operational support?

Resposta:A successful architecture provides clarity and visibility for important use cases, making it easier to understand and operate while accommodating scalability and changes needed to support those operational needs.

Chapter 18 | Boundary Anatomy| Perguntas e respostas

1.Pergunta

What is a boundary crossing in software architecture?

Resposta:A boundary crossing occurs when a function on one side of a boundary calls a function on the other side and passes along data. It is essential to manage source code dependencies during this process to prevent issues during compilation and deployment.

2.Pergunta

What is the significance of the dreaded monolith in boundary anatomy?



Resposta: The dreaded monolith lacks physical boundaries and represents a disciplined segregation of functions and data within a single executable. Despite having no visible boundaries in deployment, it still has significant architectural meaning by facilitating independent development and component testing.

3. Pergunta

How does dynamic polymorphism aid in managing dependencies across boundaries?

Resposta: Dynamic polymorphism allows runtime dependencies to oppose compile-time dependencies, enabling high-level clients to call lower-level services without tightly coupling them together. This helps maintain a clean separation of concerns.

4. Pergunta

What are deployment components in the context of boundary anatomy?

Resposta: Deployment components are dynamically linked libraries or packages (like .NET DLLs or Java JAR files) that



do not require compilation upon deployment. They allow for deployment-level decoupling, where components can be modified without recompiling the entire system.

5.Pergunta

How can threading be utilized within monolithic or deployment components?

Resposta:Threads can be used to organize execution within both monolithic and deployment components, helping to manage order and scheduling without creating architectural boundaries. They can enhance performance through concurrency across components.

6.Pergunta

What defines a local process boundary in software architecture?

Resposta:A local process boundary is a stronger architectural boundary where processes run in separate address spaces on the same physical or multicore machine, communicating primarily via interprocess communication methods like sockets or message queues.



7.Pergunta

What characterizes a service boundary, and why is it significant?

Resposta:A service boundary characterizes a communication process that can operate independently, often over a network, with no assumption about physical proximity. This boundary is significant due to its potential for scalability and flexibility, given that services can be deployed and modified independently.

8.Pergunta

What challenges arise from poorly managed boundary crossings in software systems?

Resposta:Poorly managed boundary crossings can lead to a tangled dependency structure that complicates development, increases the risk of errors, raises deployment costs, and creates difficulties during integration testing.

9.Pergunta

How can architecture encapsulate high-level policy while deferring operational detail decisions?

Resposta:Good architecture separates high-level policies



from operational details through abstraction, allowing developers to focus on business logic while deferring specific implementation choices until they are necessary, thereby maintaining flexibility.

10.Pergunta

What role should software architecture play in minimizing human resources for system maintenance?

Resposta:Software architecture should facilitate a clear structure that minimizes the effort needed for system maintenance by ensuring components are loosely coupled and can be modified independently of others, thus reducing the risk and complexity associated with changes.

Mais Livros Grátis



Escaneie para baixar



Ouça-o



Ler, Compartilhar, Empoderar

Conclua Seu Desafio de Leitura, Doe Livros para Crianças Africanas.

O Conceito



Esta atividade de doação de livros está sendo realizada em conjunto com a Books For Africa. Lançamos este projeto porque compartilhamos a mesma crença que a BFA: Para muitas crianças na África, o presente de livros é verdadeiramente um presente de esperança.

A Regra



Ganhe 100 pontos



Resgate um livro



Doe para a África

Seu aprendizado não traz apenas conhecimento, mas também permite que você ganhe pontos para causas beneficentes! Para cada 100 pontos ganhos, um livro será doado para a África.

Teste gratuito com Bookey



Chapter 19 | Policy and Level| Perguntas e respostas

1.Pergunta

What is the core concept of software architecture as described in this chapter?

Resposta:Software systems are ultimately statements of policy, detailing how inputs are transformed into outputs. Architecture involves carefully separating and organizing these policies based on how and when they change.

2.Pergunta

How should policies be grouped in software architecture?

Resposta:Policies that change for the same reasons and at the same times should be grouped together within the same component, while those that change for different reasons or at different times should be separated into different components.

3.Pergunta

What does it mean when we say a good architecture has low-level components depending on high-level components?



Resposta:It signifies that lower-level details are abstracted away, ensuring that changes in the lower-level components do not impact the higher-level policies directly, thus reducing the risk and complexity in the architecture.

4.Pergunta

How does the level of a policy relate to its distance from inputs and outputs?

Resposta:The level is defined as the distance from inputs and outputs; higher-level policies are further from these endpoints and therefore change less frequently as they involve more significant and strategic decisions.

5.Pergunta

Why is it important to maintain a clear separation between high-level policies and lower-level details in architecture?

Resposta:This separation minimizes the impact of changes: trivial changes at lower levels will not affect the more important higher-level policies, ensuring greater stability and flexibility in the system.

6.Pergunta

Mais Livros Grátis



Escaneie para baixar



Ouça-o

What does the term 'directed acyclic graph' refer to in the context of software architecture?

Resposta: It refers to a graph where the nodes represent components containing policies at the same level, and the directed edges indicate dependencies between these components, ensuring there are no cycles.

7.Pergunta

What are the benefits of having lower-level components as plugins to higher-level components?

Resposta: It allows independent changes and evolution of lower-level details without affecting higher-level policies, thus maintaining the overall integrity and functionality of the system.

8.Pergunta

In what ways can changes to lower-level policies affect higher-level policies?

Resposta: Changes to lower-level policies should primarily be independent unless they interact directly with the higher-level policies; ideally, lower-level changes should



have minimal to no impact on higher-level functionality.

9.Pergunta

Why do policies tend to change less frequently than details in software systems?

Resposta:Policies are often based on overarching business rules or strategic decisions, which tend to be more stable, whereas details like UI elements or data formats change regularly due to user requirements or technology advancements.

Chapter 20 | Business Rules| Perguntas e respostas

1.Pergunta

What qualifies as a Critical Business Rule?

Resposta:A Critical Business Rule is a fundamental procedure or guideline that directly contributes to making or saving money for a business, regardless of whether it is implemented through automated technologies. For example, a bank charging interest on a loan is a Critical Business Rule.

2.Pergunta

What is an Entity in the context of a software system?

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Resposta: An Entity is an object within a software system that encapsulates a small set of Critical Business Rules and operates on Critical Business Data, effectively binding the business rules and the necessary data together.

3. Pergunta

How do Use Cases differ from Entities?

Resposta: Use Cases define the specific ways an automated system is used, including input requirements, output expectations, and processing steps, while Entities represent the broader business rules and data that govern the system's operations.

4. Pergunta

Why is keeping request and response models independent important?

Resposta: Keeping request and response models independent avoids unnecessary dependencies on external systems or frameworks that may change over time, thus preserving the integrity of the use cases and reducing maintenance complexity.



5.Pergunta

How does the Dependency Inversion Principle apply to the relationship between Entities and Use Cases?

Resposta:According to the Dependency Inversion Principle, high-level concepts like Entities should not depend on lower-level concepts like Use Cases. Instead, Use Cases depend on Entities, ensuring that the core business logic remains stable while application-specific rules are flexible.

6.Pergunta

What principles help in minimizing human resources in software architecture?

Resposta:The principles include the Single Responsibility Principle (SRP), which advises keeping different responsibilities separated; the Open-Closed Principle (OCP), which encourages designs that can be extended without modifying existing code; and the Dependency Inversion Principle (DIP), which promotes referencing abstractions rather than concrete implementations.

7.Pergunta

What is the role of an architect in software design

Mais Livros Grátis



Escaneie para baixar



Ouça-o

according to the text?

Resposta: The architect's role involves ensuring that the software architecture minimizes the lifetime costs of the system by facilitating development, deployment, and maintenance while leaving as many options open for future changes as possible.

8. Pergunta

Why is it problematic if components of different stability (I metrics) depend on each other?

Resposta: If lower-level unstable components depend on stable high-level components, any change in the unstable component can potentially disrupt the stable component's functionality, introducing risks and complications that could undermine system stability.

9. Pergunta

What is the importance of separating policies from details in software architecture?

Resposta: Separating policies from details allows for flexible architectures that can adapt to changing requirements, as



changes in detail implementations will not affect the core business logic, helping maintain the integrity and longevity of the system.

10.Pergunta

How can using plugins improve software architecture?

Resposta:Using plugins can make the core business rules of a system independent of optional components or functionalities like GUIs or databases, allowing for easy interchangeability and reducing the impact of changes in optional components on the core business logic.

Chapter 21 | Screaming Architecture| Perguntas e respostas

1.Pergunta

What does the architecture of a software application scream about?

Resposta:It should scream about the use cases of the application, indicating its main purpose and functionality, rather than the frameworks or technologies used to build it.

2.Pergunta

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Why should good architecture prioritize use cases over frameworks?

Resposta: Good architecture allows for flexibility and easier changes in the future by decoupling the core functionality of the application (use cases) from the specific frameworks and environments that may change over time.

3.Pergunta

How should architects view frameworks?

Resposta: Frameworks should be viewed as tools to be used, not as architectures themselves—it is essential to ensure that the architecture does not become tied to a specific framework.

4.Pergunta

How can a good architecture help unit testing?

Resposta: A good architecture enables unit testing by ensuring that the business rules (entities) and use cases are decoupled from the frameworks, allowing tests to run without needing the web server or database.

5.Pergunta

What is the purpose of separating business rules and



databases in architecture?

Resposta: The separation allows for decisions about the database to be deferred and avoids unnecessary dependencies, making changes in one not affect the other.

6.Pergunta

What kind of decisions should an architecture aim to defer?

Resposta: Architecture should aim to defer decisions about frameworks, databases, and other technical details which do not directly affect the core functionality of the application.

7.Pergunta

What is meant by a system being 'device independent'?

Resposta: Device independence means that the application's core functionality should be agnostic of how it is delivered (web, mobile, etc.), allowing flexibility in how it is implemented or scaled.

8.Pergunta

What happens when the architecture of a system does not support its use cases effectively?

Resposta: It can increase maintenance costs, create



deployment difficulties, and make development more complex, leading to a system that is not user-friendly or adaptable to future changes.

9.Pergunta

What should the structure of a software architecture reveal?

Resposta:The structure should reveal the operational needs and use cases of the system, making it clear and understandable for developers.

10.Pergunta

How can decisions about individual use cases influence the overall architecture?

Resposta:By recognizing which use cases change for different reasons, architects can design the system to keep these elements separate and minimize the impact of changes.





As melhores ideias do mundo desbloqueiam seu potencial

Essai gratuit avec Bookey



Escanear para baixar



Chapter 22 | The Clean Architecture| Perguntas e respostas

1.Pergunta

What is the primary goal of Clean Architecture according to Robert C. Martin?

Resposta: The primary goal of Clean Architecture is the separation of concerns through the division of software into layers, allowing for easier maintenance, better testability, and the independence of various system components from external influences such as frameworks and user interfaces.

2.Pergunta

How does Clean Architecture ensure that business rules can be tested independently?

Resposta: Clean Architecture allows business rules to be tested independently by ensuring that they do not rely on external elements such as UI, databases, or other frameworks, thanks to a strict layer separation.

3.Pergunta

Mais Livros Grátis



Escaneie para baixar



Ouça-o

What does the Dependency Rule in Clean Architecture state?

Resposta: The Dependency Rule states that source code dependencies should only point inward, toward higher-level policies, meaning that inner layers should not be aware of the details in the outer layers.

4.Pergunta

Can you explain the concept of entities in Clean Architecture?

Resposta: Entities represent enterprise-wide critical business rules and encapsulate the most general rules of the application, making them reusable across different applications without being affected by external changes like UI or database modifications.

5.Pergunta

What role do use cases play within Clean Architecture?

Resposta: Use cases encapsulate and implement application-specific business rules, orchestrating the flow of data to and from entities and ensuring that changes in



application behavior don't affect the underlying business rules or entities.

6.Pergunta

What is the significance of interface adapters in Clean Architecture?

Resposta:Interface adapters adapt data between the use cases/entities and external interfaces such as databases or web services, isolating the inner layers from the specifics of corresponding external interactions, thereby adhering to the Dependency Rule.

7.Pergunta

How does Clean Architecture facilitate testable architectures?

Resposta:By separating business rules and ensuring they are independent of frameworks, databases, and UI elements, Clean Architecture ensures that business logic can be tested in isolation without requiring the entire system setup.

8.Pergunta

What is the importance of maintaining independence from frameworks according to Robert C. Martin?

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Resposta: Maintaining independence from frameworks allows developers to avoid being constrained by the limitations or complexities of specific frameworks, thus enabling flexibility and adaptability as project requirements evolve.

9. Pergunta

How does the concept of ‘options open’ relate to software architecture?

Resposta: The concept of keeping options open in software architecture involves designing systems in a way that decisions about tools, technologies, and specific implementations can be deferred and adjusted without jeopardizing core functionality.

10. Pergunta

What can lead to the so-called ‘morning after syndrome’ in software development?

Resposta: The morning after syndrome occurs when multiple developers work on interconnected components without a clear boundary configuration, resulting in unexpected breakages when one developer’s changes impact shared



elements used by others.

Chapter 23 | Presenters and Humble Objects| Perguntas e respostas

1.Pergunta

What is the Humble Object pattern and why is it important in Clean Architecture?

Resposta: The Humble Object pattern is a design pattern that helps separate parts of a system that are hard to test from those that are easy to test. It does this by dividing functionalities into two classes: one unchanged humble class that does not contain complex logic and is hard to test, and another testing-friendly presenter that encapsulates the business logic, making the architecture more testable and maintainable. Its importance lies in defining clear architectural boundaries to improve overall system organization and facilitate testing.

2.Pergunta

How do Presenters and Views work together in Clean Architecture?

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Resposta: In Clean Architecture, Views act as the humble objects responsible for displaying information, while Presenters handle data processing. The Presenter formats the data received from the application into a structure (the View Model) that the View can easily display. By separating the responsibilities, the Presenter is able to be unit tested independently, while the View remains simple and focused solely on the display aspects of the UI.

3.Pergunta

What role do database gateways play in Clean Architecture?

Resposta: Database gateways are interfaces between the use cases and the database, defining methods for all CRUD operations without directly embedding SQL into the use case layer. They follow the Humble Object pattern by acting as a humble layer that simply executes database commands, while the interactors (use case logic) remain free of database details and are designed for testability.

4.Pergunta

Mais Livros Grátis



Escaneie para baixar



Ouçá-o

What is the significance of data models like entities and use cases in a business application?

Resposta: Entities encapsulate critical business rules and data, representing core concepts that the system manages and ensuring that they remain stable despite changes in external factors. Use cases describe specific application behaviors and interactions, orchestrating how entities are used without exposing their complexities. Together, they allow for clean, maintainable code that can evolve with business needs.

5.Pergunta

Why is testability considered an attribute of good architecture?

Resposta: Testability indicates how easily a system can be validated against its requirements, which is essential for maintaining quality in software development. A well-architected system, by employing patterns like the Humble Object pattern, defines clear interfaces and decouples components, allowing automated tests to run against high-level business logic without the need for



underlying complexities, thereby reducing defect rates and facilitating faster development cycles.

6.Pergunta

How can a software architecture be designed to keep options open for future changes?

Resposta:To keep options open, the architecture should be built around abstractions that allow for swapping components (like databases, UI frameworks, or protocols) without affecting the core business logic. This can be achieved through interface-based designs, promoting the use of dependency injection, and separating the business rules from details such as user interfaces or databases, which allows for flexible adaptation as requirements evolve.

7.Pergunta

What mistakes do companies often make when implementing software architecture?

Resposta:Common mistakes include making premature decisions about tools, frameworks, and technologies before fully understanding the requirements or use cases, which can



result in rigid architectures that are difficult to change. They may also fail to decouple business rules from technical implementations, leading to systems that are overly complex and expensive to maintain.

8.Pergunta

In what way does the Clean Architecture emphasize the importance of separating concerns?

Resposta:The Clean Architecture emphasizes the importance of separating concerns by advocating for a structure where business logic is distinct from the user interface and external dependencies. This clear separation ensures that changes to one aspect of the system do not adversely impact others, promoting maintainability and allowing each part to evolve independently.

9.Pergunta

What are some architectural patterns associated with Clean Architecture?

Resposta:Architectural patterns associated with Clean Architecture include Hexagonal Architecture (Ports and

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Adapters), DCI (Data, Context, and Interaction), and BCE (Behavior, Context, and Entity). Each of these patterns aims to achieve the same goal: separation of concerns, leading to more maintainable and flexible systems.

10.Pergunta

How does the Dependency Rule relate to Clean Architecture?

Resposta:The Dependency Rule is a fundamental tenet of Clean Architecture that states all source code dependencies should point inward toward higher-level policies, rather than allowing inner layers to depend on outer layers. This way, the architecture minimizes the impact of changes and facilitates greater adaptability, maintainability, and ease of testing.

Chapter 24 | Partial Boundaries| Perguntas e respostas

1.Pergunta

What is a partial boundary in software architecture?

Resposta:A partial boundary allows software architects to create boundaries between components

Mais Livros Grátis



Escaneie para baixar



Ouça-o

without fully implementing all the necessary reciprocal interfaces and dependency management required for complete isolation. This approach can defer resource-intensive work while still maintaining some level of separation.

2.Pergunta

Why are architectural boundaries considered expensive to maintain?

Resposta:Architectural boundaries require comprehensive management, including reciprocal polymorphic interfaces and careful handling of input/output data structures, making them costly in terms of initial setup and ongoing maintenance.

3.Pergunta

How does anticipatory design conflict with Agile philosophy?

Resposta:Anticipatory design, which involves creating boundaries and structures for features that may not be immediately needed, can violate the YAGNI principle ('You



Aren't Gonna Need It') often emphasized in Agile development.

4.Pergunta

What is the benefit of skipping the last step when implementing partial boundaries?

Resposta: Skipping the last step allows for the creation of components with all the necessary code and design architecture without the overhead of managing multiple deployable components, reducing complexity in version tracking and release management.

5.Pergunta

What risk is associated with the strategy of partial boundaries as demonstrated in FitNesse?

Resposta: The risk is that as dependencies between components progress over time, the initially intended separation may weaken, leading to a more tangled architecture that becomes challenging to untangle.

6.Pergunta

What is a one-dimensional boundary, and how does it differ from a full-fledged boundary?

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Resposta: A one-dimensional boundary uses reciprocal interfaces to maintain separation in a simpler structure, exemplified by the Strategy pattern. It is less expensive to maintain than a full-fledged architectural boundary, which requires more complex management of dependencies.

7.Pergunta

How can a Facade pattern simplify boundary management?

Resposta: The Facade pattern simplifies boundary management by providing a simplified interface that allows clients to access services without exposing all the complexities and dependencies, though it risks creating transitive dependencies.

8.Pergunta

What is the importance of isolating clients from service implementations?

Resposta: Isolating clients from service implementations enhances maintainability as it reduces the likelihood of changes in service implementation affecting clients and



allows for easier swapping of implementations.

9.Pergunta

How should architects decide on implementing architectural boundaries?

Resposta:Architects should assess where a boundary might eventually exist based on their understanding of current and future system needs, balancing the costs of partial versus full boundaries against organizational and project requirements.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Ad



Escanear para baixar



Experimente o aplicativo Bookey para ler mais de 1000 resumos dos melhores livros do mundo

Desbloqueie **1000+** títulos, **80+** tópicos

Novos títulos adicionados toda semana

Product & Brand

 Liderança & Colaboração

 Gerenciamento de Tempo

 Relacionamento & Comunicação

 Estratégia de Negócios

 Criatividade

 Memórias

 Conheça a Si Mesmo

 Psicologia

Empreendedorismo

 História Mundial

 Comunicação entre Pais e Filhos

 Autocuidado

 Mente

Visões dos melhores livros do mundo

amento
pos

Os 7 Hábitos das
Pessoas Altamente
Eficazes



Mini Hábitos



Hábitos Atômicos



O Clube das 5
da Manhã



Como Fazer Amigos
e Influenciar
Pessoas



Com
Não



Teste gratuito com Bookey



Chapter 25 | Layers and Boundaries| Perguntas e respostas

1.Pergunta

What key architectural principle allows the separation of UI from business logic, making it easier to adapt different languages or UIs?

Resposta: The Dependency Rule allows for the separation between UI and business logic, facilitating adaptation and reuse of the same core business rules without concern for the UI language or format.

2.Pergunta

In the context of a game like 'Hunt the Wumpus', what are potential architectural boundaries to consider for enhancing flexibility?

Resposta: Potential architectural boundaries include decoupling the game rules from the UI and establishing language-independent APIs that allow the game to be adapted for localization or different communication mechanisms.



3.Pergunta

How does the organizational structure of components impact a system's ability to adapt to changes without extensive rework?

Resposta:By organizing components by their functionality and isolating dependencies, high-level components can remain stable even as low-level implementations change, significantly reducing the effort required to adapt to new requirements.

4.Pergunta

Why is it important to maintain the independence of business rules from external frameworks and databases?

Resposta:Maintaining independence allows business rules to remain stable and unaffected by changes in external systems, enabling easier maintenance, deployment, and adaptation of the software architecture.

5.Pergunta

What analogy is used to explain why software architecture should be flexible and open for changes?

Resposta:The analogy of a building's architecture is



used—just like the architectural plans for a house allow for different material choices, software architecture should allow flexibility in frameworks and technologies without locking into specific decisions too early.

6.Pergunta

What role do interfaces play in ensuring that high-level policies remain unaffected by changes in lower-level components?

Resposta:Interfaces act as a barrier, enabling high-level policies to interact with lower-level components without knowing the specifics of their implementation, thus allowing for easier modifications and enhancements.

7.Pergunta

What is the significance of creating a clear distinction between core business rules and use cases in software architecture?

Resposta:Core business rules represent the fundamental policies that dictate business functions, while use cases dictate how users interact with those rules; keeping them distinct allows for clear responsibility and easier adaptations.



8.Pergunta

How does the principle of 'Single Responsibility' extend beyond code structure to architectural design?

Resposta:In architectural design, the Single Responsibility Principle encourages each component to handle one aspect of functionality, ensuring that changes in one area do not inadvertently affect others, thus enhancing maintainability.

9.Pergunta

What fundamental truth about the nature of software does Robert C. Martin emphasize regarding the relationship between simplicity and architecture?

Resposta:The relationship emphasizes that complex systems often arise from attempts to manage over-engineering, while effective architecture focuses on simplicity, leaving options open without overwhelming the software with unnecessary complexity.

10.Pergunta

In what way do polymorphic interfaces contribute to the robustness of software architecture?

Resposta:Polymorphic interfaces create abstractions that



allow different implementations to be used interchangeably, minimizing dependencies and promoting flexibility within the architecture.

11.Pergunta

Why might a software architect choose to implement a partial boundary instead of a full architectural boundary initially?

Resposta:A partial boundary allows for anticipation of future needs without the initial overhead of full implementation, helping to manage development costs while leaving room for future enhancements.

12.Pergunta

What is the relationship between dependency management metrics and the quality of software architecture?

Resposta:Dependency management metrics help assess the stability, cohesion, and abstraction levels of components, guiding architects in maintaining a structured and balanced architecture that minimizes complexity and enhances flexibility.



13.Pergunta

How does the tension between different architectural principles (like REP, CCP, and CRP) manifest during design and implementation?

Resposta: This tension requires architects to balance the desire for reuse against the need for manageable complexity and the risks of changes, ensuring that architectural decisions support the long-term maintenance and adaptability of the system.

14.Pergunta

What effect does following the Dependency Inversion Principle have on the organization of code and components?

Resposta: Following the Dependency Inversion Principle helps ensure that high-level modules remain independent from low-level module changes, promoting more stable and adaptable architecture.

Chapter 26 | The Main Component| Perguntas e respostas

1.Pergunta

Mais Livros Grátis



Escaneie para baixar



Ouça-o

What is the role of the Main component in a software system?

Resposta: The Main component acts as the ultimate detail and lowest-level policy of the system. It serves as the entry point, creating and coordinating other components, handling initializations, and distributing dependencies without relying on a Dependency Injection framework.

2.Pergunta

Why is the Main component considered dirty or the dirtiest of all components?

Resposta: The Main component is considered dirty because it directly deals with low-level details such as loading resources and initializing configurations, which are typically avoided in clean architectures to keep higher-level components decoupled from implementation specifics.

3.Pergunta

How does the Main component interact with the overall architecture?



Resposta: The Main component hands control over to higher-level policies and abstracts away specific implementations, ensuring that changes in higher-level components do not necessitate recompilation of Main. It facilitates a clean separation between the initialization of the system and its business logic.

4. Pergunta

What principle is highlighted when discussing how the Main component handles dependencies?

Resposta: The Dependency Inversion Principle is highlighted, emphasizing that dependencies should be managed in such a way that they promote high-level abstraction while keeping low-level details isolated from the core business logic.

5. Pergunta

In what ways can multiple Main components enhance a software system?

Resposta: Having multiple Main components allows for different configurations tailored for various environments such as Development, Testing, and Production. This



configuration flexibility enhances the maintainability and adaptability of the system.

6.Pergunta

What can we deduce about the significance of separating concerns in the Main component?

Resposta:Separating concerns within the Main component ensures that higher-level policies can evolve independently from low-level details, leading to a more robust and maintainable architecture that minimizes the cost of change.

7.Pergunta

Why is it important for architecture to allow changes in requirements without extensive rework?

Resposta:Allowing changes without extensive rework is crucial because it reduces the cost of modification and enhances developer productivity, making it easier to adapt the system to new conditions and requirements over time.

8.Pergunta

How does the Main component exemplify the concept of plugins in software architecture?

Resposta:The Main component exemplifies plugins by being



a customizable entry point that can be replaced or extended for various configurations, enabling different runtime behaviors without altering the underlying system architecture.

Chapter 27 | Services: Great and Small| Perguntas e respostas

1.Pergunta

What is the main argument presented by Robert C. Martin regarding service-oriented architectures?

Resposta:Martin argues that just because a system is structured around services, it doesn't automatically mean it constitutes a proper architectural framework. He contends that true architecture is defined by boundaries that separate high-level policies from low-level details, hence services that merely separate application behaviors may not hold significant architectural weight.

2.Pergunta

How does the author challenge the assumption of services being strongly decoupled?

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Resposta: Martin states that while services may seem decoupled since they operate in different processes, they can still be tightly coupled through shared resources or data. Changes in shared data structures can necessitate updates across multiple services, revealing that the perceived decoupling is often illusory.

3. Pergunta

Can you give an example of the complications that can arise from a service-oriented architecture, as discussed in the chapter?

Resposta: An illustrative example is the taxi aggregator system where adding a kitten delivery feature necessitated changes across all existing services. This highlighted that despite being designed as independent services, they were inherently coupled due to their reliance on shared data and functionality.

4. Pergunta

What role do object-oriented design principles play in managing service architecture?

Resposta: The object-oriented design principles, particularly



the SOLID principles, encourage the creation of flexible and decoupled components. This allows new features or services to be added without altering the existing code, thereby improving the architectural integrity amid increasing complexity.

5.Pergunta

What does the author imply about the responsibility of software architects?

Resposta:Software architects must recognize that architecture requires careful planning and foresight. They need to design systems that allow for future changes and enhancements without entangling new features with existing ones, ensuring a balance between flexibility and maintainability.

6.Pergunta

In what way are services compared to expensive function calls?

Resposta:Martin explains that if services merely exist to delineate behaviors without enforcing substantial architectural boundaries or separation of concerns, they can



be likened to expensive function calls, as they do not contribute meaningfully to system architecture.

7.Pergunta

What is meant by 'cross-cutting concerns' in the context of service architecture?

Resposta:Cross-cutting concerns refer to features or functionalities that affect multiple parts of a system in a tangled manner, rather than being localized. This can lead to dependencies where changes in one service necessitate adjustments in others, complicating maintenance and scalability.

8.Pergunta

How does the chapter suggest that the structure of services should be designed?

Resposta:The chapter emphasizes that the architecture of services should be inwardly directed, designed in line with the Dependency Rule. Services must have their internal structures defined by components that adhere to this rule, isolating their intricate interdependencies.





Escanear para baixar



Por que o Bookey é um aplicativo indispensável para amantes de livros



Conteúdo de 30min

Quanto mais profunda e clara for a interpretação que fornecemos, melhor será sua compreensão de cada título.



Clipes de Ideias de 3min

Impulsione seu progresso.



Questionário

Verifique se você dominou o que acabou de aprender.



E mais

Várias fontes, Caminhos em andamento, Coleções...

Teste gratuito com Bookey



Chapter 28 | The Test Boundary| Perguntas e respostas

1.Pergunta

How do tests participate in software architecture?

Resposta:Tests are considered integral components of software architecture, participating just as other parts of the system do. They follow the Dependency Rule, depending inward towards the application code rather than being dependent upon by it. This architectural position allows tests to support development while being independently deployable.

2.Pergunta

What is the impact of fragile tests on software systems?

Resposta:Fragile tests can make a system rigid and difficult to change because they are strongly coupled to the implementation. When changes occur in the production code, these tests often fail, discouraging developers from making necessary changes to the system.

3.Pergunta

What is the solution to mitigate the Fragile Tests

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Problem?

Resposta: Design for testability by creating a specific testing API that allows tests to verify business rules without relying on volatile elements like GUIs. This allows tests to operate at a higher level of abstraction, making them more stable and less dependent on the structure of the application.

4.Pergunta

Why is decoupling the structure of tests from the application important?

Resposta: Decoupling allows both the tests and the production code to evolve independently. If tests are closely tied to the specifics of the application structure, any changes in the production code can lead to many broken tests, complicating maintenance and discouraging code changes.

5.Pergunta

What is the purpose of a testing API?

Resposta: A testing API serves to decouple tests from the application, creating an intermediary that enables tests to interact with the application without requiring knowledge of



its structure. This facilitates easier modifications in both the tests and the production code.

6.Pergunta

What is 'structural coupling' in the context of tests?

Resposta:Structural coupling refers to a tight relationship where tests are directly influenced by the structure of the code they validate. If a test suite mirrors the application's structure, changes to that structure can lead to widespread test failures.

7.Pergunta

How should tests be viewed in terms of system importance?

Resposta:Tests should be regarded as vital components of the system that require proper design to ensure they remain effective, maintainable, and supportive of ongoing development without causing unnecessary overhead or rigidity.

8.Pergunta

What does 'design for testability' entail?

Resposta:Design for testability involves creating a system



architecture that facilitates testing by allowing tests to validate functionality without being tightly integrated with the application's other components, especially volatile features like the UI.

9.Pergunta

What characteristics should tests embody to be effective?

Resposta:Effective tests should be independent, decoupled from the specifics of the application structure, easily maintainable, and able to function without the system's UI or other external dependencies.

10.Pergunta

Why is avoiding volatile dependencies crucial in test design?

Resposta:Avoiding volatile dependencies is crucial because tests that depend on unstable parts of the application are more likely to break with minor code changes, which can lead to reduced confidence in the test suite and increased maintenance overhead.

Chapter 29 | Clean Embedded Architecture|
Perguntas e respostas

Mais Livros Grátis



Escaneie para baixar



Ouça-o

1.Pergunta

What is the importance of separating software from firmware according to James Grenning?

Resposta:Grenning emphasizes that while software can have a long useful life, firmware can become obsolete and create unmanaged dependencies that can 'infect' software, restricting its longevity. By separating software from firmware, we can maintain a clean architecture that allows for easier updates and modifications without the complications posed by hardware dependencies.

2.Pergunta

What is the App-titude test and why is it important for embedded systems developers?

Resposta:The App-titude test consists of three stages: first making it work, then making it right, and finally making it fast. It's important for embedded systems developers to understand that focusing solely on getting the app to work can lead to poor architecture. Developers should be mindful



of structuring their code for longevity and maintainability.

3.Pergunta

How does the target-hardware bottleneck affect embedded software development?

Resposta: The target-hardware bottleneck refers to the situation where developers can only test their code on the actual hardware, which can slow down the development process significantly. By structuring embedded software with clean architecture principles, developers can mitigate this bottleneck and facilitate off-target testing.

4.Pergunta

What are some strategies mentioned by Grenning to keep embedded software architecture clean?

Resposta: Strategies include using a Hardware Abstraction Layer (HAL) to isolate hardware specifics from software, defining clear interfaces for firmware and software, ensuring that software structure does not reveal any hardware details, and applying principles like the Dependency Inversion Principle to maintain separation and minimize coupling.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

5.Pergunta

What is the significance of layered architecture in embedded systems according to Grenning?

Resposta:Layered architecture allows developers to separate concerns within the system, ensuring that hardware, firmware, and software dependencies are properly managed. This organization supports better testing, maintenance, and the ability to adapt to changes without extensive rewrites, thus contributing to the software's longevity.

6.Pergunta

What does Grenning suggest leads to the destruction of potential long-term software functionality?

Resposta:Grenning suggests that unmanaged dependencies on firmware and hardware can infect embedded software, leading to its obsolescence and rendering it unable to adapt to new requirements, which effectively destroys its potential for long-term functionality.

7.Pergunta

What are the three activities described by Kent Beck in building software, and how should they be prioritized?



Resposta: The three activities are: 1) Make it work - ensuring functionality; 2) Make it right - refactoring to improve clarity and adaptability; and 3) Make it fast - optimizing performance. The priority should be to first ensure correctness before refactoring and optimizing, as this lays the groundwork for sustainable software development.

8.Pergunta

Why should embedded developers avoid writing too much firmware according to Grenning?

Resposta: Embedding too much firmware can tie the software to specific hardware, making it brittle and hard to modify when hardware changes occur. This can negate the potential longevity of the software by creating dependencies that are difficult to manage.

9.Pergunta

How does Grenning relate the concepts of firmware and software to the evolving nature of hardware?

Resposta: He explains that while software may age well, it can be negatively impacted by the rapid evolution of



firmware and hardware unless carefully managed.

Well-structured, clean architectures can help mitigate these impacts by ensuring clearer separations of layers from hardware.

10.Pergunta

What is the hardware abstraction layer (HAL) and why is it crucial in embedded systems?

Resposta:The HAL is an interface that abstracts hardware details from software, allowing the latter to remain independent of specific hardware implementations. This is crucial for maintaining long-term software usability and adaptability as hardware evolves.

11.Pergunta

In what way does Grenning suggest that embedded systems development differs from non-embedded development?

Resposta:Grenning notes that embedded systems developers face unique challenges such as real-time constraints, limited resources, and hardware dependencies that demand a higher level of architectural consideration to ensure flexibility and

Mais Livros Grátis



Escaneie para baixar



Ouça-o

longevity of the software.

Chapter 30 | The Database Is a Detail| Perguntas e respostas

1.Pergunta

Why is the database considered a detail in software architecture?

Resposta: The database is deemed a detail because it is merely a utility that provides access to data, without significant influence on the overall architecture of the software system. Just as a doorknob is a detail in the architecture of a home, the database's mechanics should not overshadow the primary policies and structures defining the system.

2.Pergunta

What is the key difference between a database and a data model from an architectural perspective?

Resposta: The data model shapes how data is structured and used within the application, which is architecturally significant. In contrast, the database itself is implemented technology that serves the purpose of data storage and

Mais Livros Grátis



Escaneie para baixar



Ouça-o

retrieval, and is considered low-level detail.

3.Pergunta

What has contributed to the prevalence of database systems in software architecture?

Resposta:Database systems have become prevalent largely due to the historical reliance on disk technology for data storage, which has been the mainstay for decades, driving the adoption of relational models due to their elegance and efficiency.

4.Pergunta

How should architectures handle database access to avoid coupling business rules to relational database structures?

Resposta:Architectures should encapsulate data access in utility functions or gateways that interact with the database, restricting knowledge of the database's relational structure to the outer layers of the architecture. This approach prevents direct coupling of business rules to database design.

5.Pergunta

What does the author suggest regarding performance concerns related to databases in architecture?

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Resposta: Performance concerns should be encapsulated as low-level implementation details, handled by efficient data access mechanisms, separate from the high-level business rules and overall architecture of the system.

6.Pergunta

Why is developing software without a database advantageous during the early phases of a project?

Resposta: Avoiding a database in the early phases prevents the team from getting bogged down with unwanted complexities related to database schema, queries, and connections, allowing for rapid development and flexible architectural decisions.

7.Pergunta

What lesson does the anecdote about the startup's reluctance to use an RDBMS convey?

Resposta: The anecdote illustrates that sometimes architectural decisions are driven by customer expectations rather than technical needs. As a result, architects should be cautious about integrating low-level technology based merely



on market pressures.

8.Pergunta

How can architects ensure their systems remain flexible as requirements evolve?

Resposta:By maintaining clear architectural boundaries and emphasizing high-level policies while deferring decisions about low-level details like databases or frameworks, architects can better accommodate future changes and requirements.

9.Pergunta

What is the significance of the Dependency Rule in Clean Architecture?

Resposta:The Dependency Rule mandates that source code dependencies must only point inward toward higher-level policies, ensuring that inner components remain unaffected by changes in outer components, thereby promoting maintainability and flexibility.

10.Pergunta

In the context of Clean Architecture, why are entities considered significant?

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Resposta:Entities encapsulate critical business rules and represent the core logic of the application, making them central to the architecture. Their independence from lower-level implementation details allows them to be reused across different applications without being affected by changes in technology.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Ad



Escanear para baixar



App Store
Escolha dos Editores



22k avaliações de 5 estrelas

Feedback Positivo

Afonso Silva

... cada resumo de livro não só
..., mas também tornam o
... divertido e envolvente. O
... tou a leitura para mim.

Fantástico!



Estou maravilhado com a variedade de livros e idiomas
que o Bookey suporta. Não é apenas um aplicativo, é
um portal para o conhecimento global. Além disso,
ganhar pontos para caridade é um grande bônus!

Brígida Santos

F



O
só
o
O

na Oliveira

... correr as
... ém me dá
... comprar a
... ar!

Adoro!



Usar o Bookey ajudou-me a cultivar um hábito de
leitura sem sobrecarregar minha agenda. O design do
aplicativo e suas funcionalidades são amigáveis,
tornando o crescimento intelectual acessível a todos.

Duarte Costa

Economiza tempo!



O Bookey é o meu apli
crescimento intelectual
perspicazes e lindame
um mundo de conheci

Aplicativo incrível!



Eu amo audiolivros, mas nem sempre tenho tempo para
ouvir o livro inteiro! O Bookey permite-me obter um resumo
dos destaques do livro que me interessa!!! Que ótimo
conceito!!! Altamente recomendado!

Estevão Pereira

Aplicativo lindo



Este aplicativo é um salva-vidas para
de livros com agendas lotadas. Os re
precisos, e os mapas mentais ajudar
o que aprendi. Altamente recomend

Teste gratuito com Bookey



Chapter 31 | The Web Is a Detail| Perguntas e respostas

1.Pergunta

What does Robert C. Martin mean by saying 'The web is a detail'?

Resposta:Robert C. Martin argues that the web does not fundamentally change the architecture of software systems; it is merely a new mechanism for presenting applications. As such, it should not dictate the core business logic and structure of applications. He emphasizes the importance of separating the user interface from business rules to maintain flexibility against marketing pressures and technological shifts.

2.Pergunta

How can software architects protect applications from unnecessary changes driven by marketing demands?

Resposta:Architects can protect applications by decoupling business rules from the user interface (UI). This means designing the application in a way that the core business



logic is independent of how it is presented to users. By focusing on creating interfaces that remain constant even as the UI changes, architects can prevent forced alterations due to trends or marketing directions.

3.Pergunta

What are the implications of the endless oscillations between centralized and decentralized computing as discussed in the text?

Resposta: These oscillations highlight the industry's ongoing struggle to determine the optimal distribution of computing power. This back-and-forth indicates that software architectures must be resilient and adaptable to changes in technology without affecting core business functionalities.

4.Pergunta

Why is it important for business rules to be insulated from lower-level details such as UI and database technologies?

Resposta: It is crucial for business rules to remain insulated because changes to UI or database technologies could lead to significant redesign efforts if they are directly intertwined

Mais Livros Grátis



Escaneie para baixar



Ouça-o

with business logic. Such insulation ensures that business rules can evolve with the needs of the organization while maintaining stability in the user experience and data management.

5.Pergunta

What example does Martin provide about a company poorly adapting to web technologies, and what could they have done differently?

Resposta:Martin shares the story of Company Q, which altered its user interface to resemble a web browser due to market trends, resulting in user dissatisfaction. Had the architecture decoupled core business rules from UI components, they could have adapted the interface without impacting business logic, allowing for a smoother transition and protecting user experience.

6.Pergunta

What is the 'Dependency Rule' mentioned in Martin's discussion of clean architecture?

Resposta:The Dependency Rule states that source code dependencies must point inward, towards higher-level



policies and business logic, rather than outward towards lower-level details such as frameworks, UI components, and databases. This ensures that high-level business rules are insulated from changes in lower-level implementations.

7.Pergunta

In the context of clean architecture, what role do use cases play?

Resposta: Use cases represent application-specific business rules, directing how user interactions correspond to system functions. They serve to orchestrate the interaction between the user interface and core business logic, ensuring that functionality adheres to the requirements laid out for the application.

8.Pergunta

How does Martin differentiate between 'business rules' and 'use cases'?

Resposta: Business rules are overarching principles that govern how an organization operates and its decisions, while use cases are specific implementations of those rules within



the context of an application. Use cases provide a structured approach to how users interact with the business logic in the software.

9.Pergunta

What lessons can be derived regarding the management of software components and their dependencies?

Resposta:Effective management of software components and their dependencies relies on maintaining clear boundaries, adhering to decoupling principles, and understanding that high-level components should not be dependent on lower-level, volatile components. This approach allows for flexibility, easier maintenance, and reduces the risk of bugs during modifications.

Chapter 32 | Frameworks Are Details| Perguntas e respostas

1.Pergunta

What is the main message about frameworks in software architecture?

Resposta:Frameworks are not architectures; they are details. They help develop software but can also



impose significant coupling, making it difficult to change or substitute them later. You should keep frameworks at a distance and not let them dictate the structure of your core application.

2.Pergunta

Why is relying heavily on a framework risky for a software project?

Resposta:Because frameworks can evolve, impose rigid structures, and may become obsolete. Once you tightly couple to a framework, switching to a better solution can become exceedingly difficult.

3.Pergunta

How should frameworks be integrated into an architecture?

Resposta:Frameworks should be treated as details that belong in the outer circles of the architecture. Use them without tightly coupling core business logic to the framework's implementation.

4.Pergunta

What advice does Robert C. Martin give regarding the

Mais Livros Grátis



Escaneie para baixar



Ouça-o

relationship between developers and framework authors?

Resposta: Framework authors generally do not have the developers' best interests in mind because they do not understand the developers' specific problems. Developers should remain cautious and critical of frameworks.

5.Pergunta

What is the metaphor used to describe the relationship with frameworks?

Resposta: The metaphor is akin to marriage. Using a framework requires a commitment, and once that commitment is made, it can be hard to separate.

6.Pergunta

What should developers do if they must use a specific framework?

Resposta: They should maintain the flexibility of their architecture by deriving proxies or wrapping the framework, rather than coupling directly with it. This allows for changes without significant rewrites or compromises.

7.Pergunta

Why is it important to keep frameworks out of core

Mais Livros Grátis



Escaneie para baixar



Ouça-o

business code?

Resposta:It preserves the independence of the business rules from external dependencies, allowing for easier maintenance, testing, and potential substitution of different technologies in the future.

8.Pergunta

In what ways can applying Clean Architecture principles help in the long-term maintenance of software?

Resposta:By isolating and encapsulating the core business rules, allowing for testability independent from frameworks, and deferring decisions regarding external technologies, Clean Architecture enhances adaptability and reduces technical debt.

9.Pergunta

How can developers avoid coupling their applications too tightly to a framework they are using?

Resposta:Developers can create abstract interfaces and use dependency injection to maintain a layer of separation between their application logic and the framework



operations.

10.Pergunta

Summarize the risk of building software that is too tightly integrated with a specific framework.

Resposta:The risk is that any changes to the framework or its APIs could necessitate significant changes to the application code, leading to increased maintenance costs and reduced flexibility in the future.

Chapter 33 | Case Study: Video Sales| Perguntas e respostas

1.Pergunta

What key considerations should be taken into account when designing software architecture for a video sales system?

Resposta:Key considerations include identifying the main actors (such as individual users, business customers, video authors, and administrators), organizing use cases around these actors to allow for independent changes, and employing principles like the Single Responsibility Principle to ensure each



component serves a distinct purpose without unnecessary coupling.

2.Pergunta

How does the Dependency Rule influence the architecture of the system?

Resposta:The Dependency Rule states that source code dependencies must point inward, toward higher-level policies, which helps to ensure that changes in lower-level components do not negatively impact the higher-level business rules or policies.

3.Pergunta

What is the significance of use case analysis in software architecture?

Resposta:Use case analysis helps identify the interactions between users and the system, ensuring the architecture is designed to support the specific functionalities required by different actors, thus maintaining clarity and focus on user needs.

4.Pergunta

Why is it important to keep the database as a detail in the

Mais Livros Grátis



Escaneie para baixar



Ouça-o

architecture?

Resposta: Keeping the database as a detail prevents it from affecting the core business logic of the application. This allows for more flexibility in changing data management solutions without impacting the overall system architecture.

5.Pergunta

What are some ways to ensure that the architecture supports independent development and deployment?

Resposta: Using layered architecture to separate concerns, defining clear interfaces to minimize dependencies, and implementing components that can be deployed or tested in isolation to ensure that teams can work on parts of the system without interfering with each other.

6.Pergunta

How does the concept of 'keeping options open' apply to architectural decisions?

Resposta: Designing the architecture in a way that defers decisions related to frameworks, databases, and user interfaces until they are absolutely necessary allows for



greater agility and responsiveness to change, minimizing long-term costs and risks.

7.Pergunta

What is the role of presenters in software architecture as discussed in the chapter?

Resposta:Presenters serve as a bridge between the user interface (which may be a 'humble object') and the business logic, formatting data for presentation without containing any business rules, thus enabling easier testing and maintaining clean separation between UI and core application logic.

8.Pergunta

What lessons can be taken away from the case study of video sales in regard to software architecture?

Resposta:The case study highlights the importance of identifying actors, structuring use cases based on their needs, applying architectural principles to maintain separation of concerns, and ensuring that the architecture is flexible enough to adapt to changes while minimizing disruption to core business logic.



9.Pergunta

Why is it recommended to treat frameworks as details in an architecture?

Resposta: Frameworks can impose unnecessary coupling and constraints on the architecture. Treating them as details allows the system to remain flexible and decoupled from specific implementations, which empowers better testing, maintenance, and potential changes in the future.

10.Pergunta

What is the importance of testing in the context of architectural design as discussed?

Resposta: Testing must be integrated into the architectural design, ensuring that business rules can be validated without relying on external interfaces or hardware dependency, thereby promoting cleaner code and greater reliability throughout the system lifecycle.

Mais Livros Grátis



Escaneie para baixar



Ouça-o



Ler, Compartilhar, Empoderar

Conclua Seu Desafio de Leitura, Doe Livros para Crianças Africanas.

O Conceito



Esta atividade de doação de livros está sendo realizada em conjunto com a Books For Africa. Lançamos este projeto porque compartilhamos a mesma crença que a BFA: Para muitas crianças na África, o presente de livros é verdadeiramente um presente de esperança.

A Regra



Ganhe 100 pontos



Resgate um livro



Doe para a África

Seu aprendizado não traz apenas conhecimento, mas também permite que você ganhe pontos para causas beneficentes! Para cada 100 pontos ganhos, um livro será doado para a África.

Teste gratuito com Bookey



Chapter 34 | The Missing Chapter| Perguntas e respostas

1.Pergunta

Why is it important to separate code into components based on functionality?

Resposta: Separating code into components allows for better organization, clearer boundaries, and controlled dependencies. It helps in managing complexity and enhances maintainability, making it easier to adapt to changes in requirements or technology.

2.Pergunta

What are the negatives of a traditional layered architecture?

Resposta: Traditional layered architecture can become unwieldy as a system grows, leading to large interconnected layers that are difficult to manage. It often lacks domain-specific clarity, making it hard to understand business logic.

3.Pergunta

Mais Livros Grátis



Escaneie para baixar



Ouça-o

What does 'package by feature' mean and why is it beneficial?

Resposta: 'Package by feature' organizes code around distinct features or business domains rather than by technical layers. This approach makes it easier to locate related code, promote better cohesion, and improve the readability of the codebase.

4.Pergunta

How do ports and adapters architecture help in decoupling?

Resposta: Ports and adapters create a separation between domain logic and technical implementation details, allowing the core business logic to remain agnostic of external factors like frameworks or databases.

5.Pergunta

What is the role of the 'Dependency Rule' in architecture?

Resposta: The Dependency Rule states that source code dependencies should point inward toward higher-level policies. This means inner layers should not know about



anything in the outer layers, promoting separation of concerns and enhancing maintainability.

6.Pergunta

Why is the use of public access modifiers in Java potentially harmful?

Resposta:Overusing public access modifiers can expose implementation details and create unnecessary coupling between components, leading to difficulties in maintaining or changing code without affecting other parts of the system.

7.Pergunta

What are some architectural options available for organizing source code dependencies?

Resposta:Options include package by layer, package by feature, use of modular frameworks like OSGi, or separating code into different structures for domain logic and infrastructure.

8.Pergunta

How can enforcing encapsulation through access modifiers improve software architecture?

Resposta:By using more restrictive access modifiers,

Mais Livros Grátis



Escaneie para baixar



Ouça-o

developers can better control which parts of the codebase can access certain components. This reduces the potential for unintentional dependencies and enhances the integrity of the architecture.

9.Pergunta

What is the significance of decoupling layers in a software architecture?

Resposta:Decoupling layers allows individual parts of the software to evolve independently. This means changes in one layer won't necessarily require changes in another, making the system more adaptable and resilient to change.

10.Pergunta

Why should business rules be kept separate from user interface concerns?

Resposta:Keeping business rules separate from the user interface ensures that changes to the UI do not impact the core functionality of the business logic and vice versa. This allows for scalability and flexibility in adapting to new technologies or designs.



11.Pergunta

What is the main principle behind the Clean Architecture as described by Robert C. Martin?

Resposta: The main principle of Clean Architecture is to structure software in such a way that the core business logic is independent of external concerns, like frameworks and UI, thus making it easy to change or replace those details without affecting the core functionality.

12.Pergunta

How can a testing API improve the testability of a system?

Resposta: A testing API can provide direct access to the system's business logic while bypassing UI and other complexities. This allows tests to be more focused, reducing dependencies on the system's infrastructure and enhancing their reliability.

13.Pergunta

What are the benefits of defining clear boundaries in software architecture?

Resposta: Defining clear boundaries helps isolate

Mais Livros Grátis



Escaneie para baixar



Ouça-o

components, reducing dependencies that may create maintenance headaches. It also improves clarity in how components interact, facilitates easier changes, and supports effective testing.

14.Pergunta

What is meant by 'programming to interfaces'?

Resposta:'Programming to interfaces' refers to the practice of designing software such that classes and components interact through defined interfaces rather than concrete implementations. This promotes flexibility and allows for easier integration of new components.

15.Pergunta

Why is following SOLID principles crucial in software design?

Resposta:Following SOLID principles helps create systems that are robust, maintainable, and easy to change, leading to a higher quality of software that can better adapt to evolving business requirements.

16.Pergunta

How do architectural patterns like microservices present

Mais Livros Grátis



Escaneie para baixar



Ouça-o

both opportunities and challenges?

Resposta:Microservices allow for independent development and scaling of services, fostering flexibility and resilience; however, they also introduce complexity in terms of communication, data management, and overall system cohesion.

17.Pergunta

What cycle of development issues can arise from a system with tight coupling?

Resposta:Tight coupling can result in frequent integration problems, where one change necessitates changes across many modules, making the system difficult to maintain, extend, and causing increases in bugs and development time.

18.Pergunta

How does the concept of 'The Devil is in the Details' apply to software architecture?

Resposta:It emphasizes that even a well-planned architecture can fail if the implementation details are mishandled. Small errors in implementation can lead to significant issues that



compromise the overall structure and functionality.

19.Pergunta

What is a common anti-pattern related to the design of architectural boundaries?

Resposta:A common anti-pattern is allowing for cross-layer dependencies, where low-level details impact higher-level components, leading to a tightly coupled system that is hard to test, maintain, and evolve.

20.Pergunta

What effect does the misuse of conditional compilation have on code maintainability?

Resposta:Misusing conditional compilation can lead to code that is hard to understand, cluttered with multiple unnecessary variations, and increases the complexity of maintaining the code across different platforms.

21.Pergunta

Why is it important to maintain a separation of concerns in software architecture?

Resposta:Separation of concerns allows different aspects of software to evolve independently, simplifying code



management, improving clarity, and enhancing the system's ability to adapt to changes.

22.Pergunta

What approach should be taken when designing for testability in a clean architecture?

Resposta:Designing for testability involves ensuring that core business logic is decoupled from external frameworks, allowing for unit tests to be executed in isolation, and relying on interfaces and stubs for testing.

Chapter 35 | Appendix A Architecture Archaeology| Perguntas e respostas

1.Pergunta

What valuable lesson can we learn about software architecture from the transition from the Datnet 30 to Varian 620/f?

Resposta:The transition from the Datnet 30 to the Varian 620/f teaches us the importance of adapting architecture to leverage new technology while managing complexity. By moving to the Varian, the team implemented an overlay system, which

Mais Livros Grátis



Escaneie para baixar



Ouça-o

improved efficiency in memory usage and allowed applications to swap in and out dynamically. This architectural decision highlights the need for flexibility and scalability in software design.

2.Pergunta

How does the concept of software boundaries evolve in the different systems mentioned throughout the chapter?

Resposta: The evolution of software boundaries in the chapter reflects a progressive understanding of abstraction and decoupling. Initially, systems had blurred boundaries with tightly coupled components (as seen in the SAC), leading to maintenance nightmares. Over time, as seen in the DLU/DRU and the 4-TEL redesign, clearer boundaries formed, enabling easier maintenance and feature enhancements, showcasing the importance of defining interfaces and dependencies.

3.Pergunta

What does the experience of developing the vectorization project reveal about the challenges of firmware and software updates?

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Resposta: The vectorization project illustrates the complexities involved in managing firmware updates and the importance of creating modular and independently deployable components. The need to recompile all 30 chips for minor updates exemplifies how tight coupling can hinder agility. By establishing a method for independently updating firmware through vectors, the team learned how to improve maintainability and adaptability of embedded systems.

4. Pergunta

Why is writing clean, isolated code critical based on the SAC experience?

Resposta: The SAC experience emphasizes that clean, isolated code is vital for maintainability and scalability.

Poorly organized and tightly coupled code led to difficulties in handling hardware changes and debugging. The realization that modems' control logic was scattered throughout the application reinforced the necessity of encapsulating functionality and promoting separation of concerns, which ultimately enhances code quality and reduces technical debt.



5.Pergunta

How did the leap to C and structured programming impact software development at Clear Communications?

Resposta: The transition to C and structured programming at Clear Communications marked a pivotal change in development practices, enabling clearer organization of code and better management of complexity. It paved the way for adopting object-oriented principles, leading to improved architecture and more maintainable codebases as developers embraced structured methodologies, ultimately influencing future projects.

6.Pergunta

What does the tale of the Electronic Receptionist reveal about market understanding in product development?

Resposta: The Electronic Receptionist's outcome highlights the critical importance of aligning product development with market needs. Despite having innovative technology, the team failed to penetrate the office equipment market due to a lack of understanding of customer requirements and sales

Mais Livros Grátis



Escaneie para baixar



Ouça-o

strategies. This underscores that technological prowess alone is insufficient without a sound go-to-market strategy.

7.Pergunta

What lessons can we take from the development and rewriting of the ETS framework regarding the importance of usability before reusability?

Resposta: The journey with the ETS framework underscores that usability must precede reusability. Developing a usable framework that accommodates immediate project needs ensures that it can be effectively reused later. The initial failure to create a usable framework led to frustrations and delays, teaching us that a thorough understanding of end-user requirements is essential for successful framework design.

8.Pergunta

In what ways did the experiences documented in the projects contribute to Robert C. Martin's philosophy on clean architecture?

Resposta: The experiences cited in the projects profoundly shaped Robert C. Martin's philosophy on clean architecture by emphasizing the significance of clear boundaries,

Mais Livros Grátis



Escaneie para baixar



Ouça-o

maintainability, isolation of business logic from infrastructure concerns, and prioritizing the user's needs during design. These lessons fostered the understanding that well-structured software is integral to successful long-term outcomes in production.

Mais Livros Grátis



Escaneie para baixar



Ouça-o



As melhores ideias do mundo desbloqueiam seu potencial

Essai gratuit avec Bookey



Escanear para baixar



Arquitetura Limpa. O Guia do Artesao Para Estrutura e Design de Software

Quiz e teste

Ver a resposta correta no site do Bookey

Capítulo 1 | O Que É Design e Arquitetura?| Quiz e teste

- 1.Design e arquitetura no desenvolvimento de software são essencialmente a mesma coisa e trabalham juntas como um todo.
- 2.Um bom design de software deve aumentar os recursos humanos necessários para construir e manter o sistema.
- 3.Codificação rápida sem consideração pela qualidade leva ao sucesso a longo prazo em projetos de software.

Capítulo 2 | Uma História de Dois Valores| Quiz e teste

- 1.O valor principal do software é seu comportamento, que inclui atender às necessidades das partes interessadas e resolver bugs.
- 2.A integridade arquitetônica é frequentemente priorizada em relação a recursos urgentes de acordo com a matriz de

Mais Livros Grátis



Escaneie para baixar



Ouça-o

Eisenhower.

3.Os desenvolvedores devem defender uma arquitetura sólida para evitar complicações e aumento de custos no futuro.

Capítulo 3 | Visão Geral dos Paradigmas| Quiz e teste

- 1.A programação estruturada foi popularizada pela primeira vez por Edsger W. Dijkstra em 1968, enfatizando o uso de instruções goto.
- 2.A programação orientada a objetos foi desenvolvida por Ole Johan Dahl e Kristen Nygaard em 1966, permitindo que variáveis de função locais persistissem.
- 3.A programação funcional tem suas raízes na invenção do » - cálculo por Alonzo Church, que foi des 1940.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

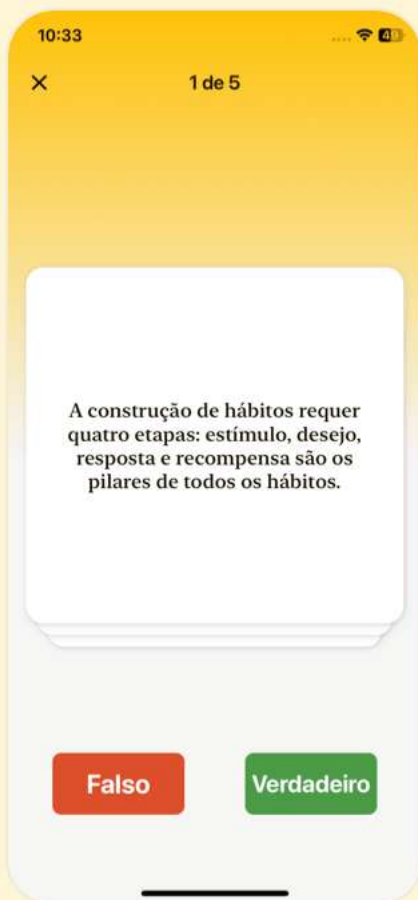


Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Escanear para baixar



Capítulo 4 | Programação Estruturada| Quiz e teste

- 1.Edsger Dijkstra foi o primeiro programador na Holanda.
- 2.Dijkstra acreditava que o uso de instruções 'goto' era benéfico para as tarefas de programação.
- 3.A programação estruturada enfatiza a produção de funções complexas e não testáveis.

Capítulo 5 | Programação Orientada a Objetos| Quiz e teste

- 1.A Programação Orientada a Objetos (POO) integra dados e funções.
- 2.A encapsulação em linguagens POO oculta os dados completamente do acesso externo e não permite que funções externas acessem esses dados.
- 3.O polimorfismo permite que funções se comportem de maneira diferente com base no tipo de entrada e pode promover uma arquitetura de plugin.

Chapter 6 | Functional Programming| Quiz e teste

- 1.The concepts of functional programming predate



programming itself, as they are rooted in

» - calculus invented by Alonzo Church in 1930s.

2. Primitive programming languages universally utilize mutable variables, making them crucial to software development.

3. Event sourcing allows for storing the current state of an application, minimizing concurrency issues.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

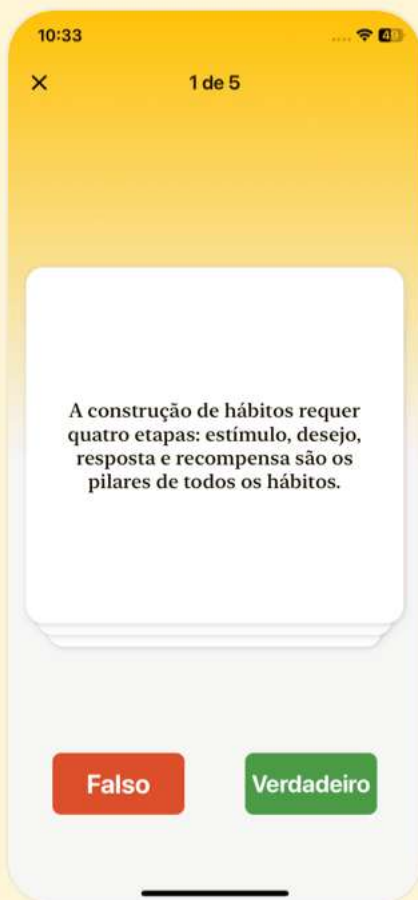


Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Escanear para baixar



Capítulo 7 | SRP: O Princípio da Responsabilidade Única| Quiz e teste

1.O Princípio da Responsabilidade Única (SRP)

afirma que um módulo deve ter apenas uma razão para mudar.

2.De acordo com o SRP, cada módulo em uma arquitetura de software deve desempenhar apenas uma função sem qualquer relação com partes interessadas.

3.Violar o SRP pode levar a complicações durante a integração de código, especialmente quando vários desenvolvedores modificam uma classe que atende a diferentes atores.

Chapter 8 | OCP: The Open-Closed Principle| Quiz e teste

1.The Open-Closed Principle (OCP) states that a software artifact should be closed for extension but open for modification.

2.Good software architecture allows for existing code to be minimally changed when new features are added.

3.Internal workings of components should be exposed to

Mais Livros Grátis



Escaneie para baixar



Ouça-o

maintain necessary dependencies between them.

Capítulo 9 | LSP: O Princípio da Substituição de Liskov| Quiz e teste

- 1.O Princípio da Substituição de Liskov (LSP) foi definido por Barbara Liskov em 1988.
- 2.O problema do quadrado/retângulo não viola o Princípio da Substituição de Liskov porque o Quadrado pode ser tratado como um Retângulo.
- 3.O LSP é relevante apenas para herança e não se aplica ao design de software ou interfaces.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

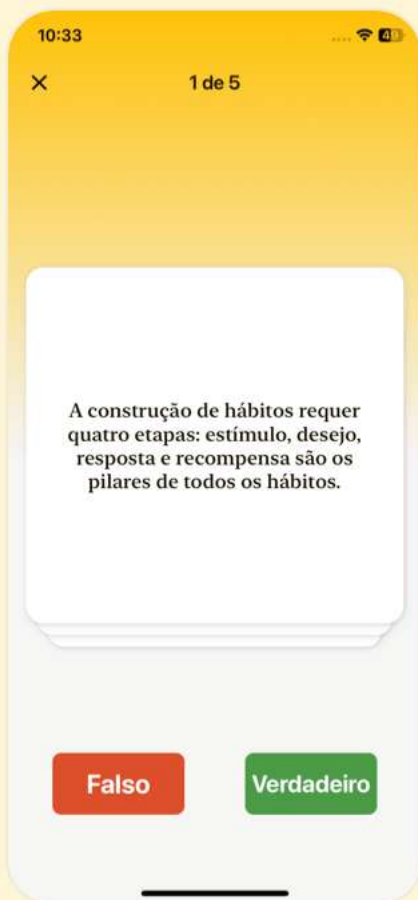


Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Escanear para baixar



Chapter 10 | ISP: The Interface Segregation Principle| Quiz e teste

- 1.The Interface Segregation Principle (ISP) states that a client should have access to all methods of an interface, even if it does not use them.
- 2.By following the ISP, changes in one part of a system can affect other parts, leading to potential recompilation and difficulties.
- 3.In statically typed languages like Java, adhering to ISP is crucial to prevent additional unwanted dependencies from affecting clients.

Chapter 11 | DIP: The Dependency Inversion Principle| Quiz e teste

- 1.The Dependency Inversion Principle (DIP) encourages systems to depend on concrete implementations rather than abstractions.
- 2.Stable abstractions are crucial for building stable architectures and changes in concrete classes typically do not require interface modifications.
- 3.Using a Factory pattern to instantiate concrete objects helps



prevent direct dependencies on underlying concrete classes, which aligns with the DIP.

Chapter 12 | Components| Quiz e teste

- 1.Components are the smallest deployable units in a system and can vary by programming language.
- 2.The invention of relocatable binaries eliminated the need for any external references in compiled programs.
- 3.The component plugin architecture developed in the 1980s and 90s allows for dynamic linking of components at runtime.



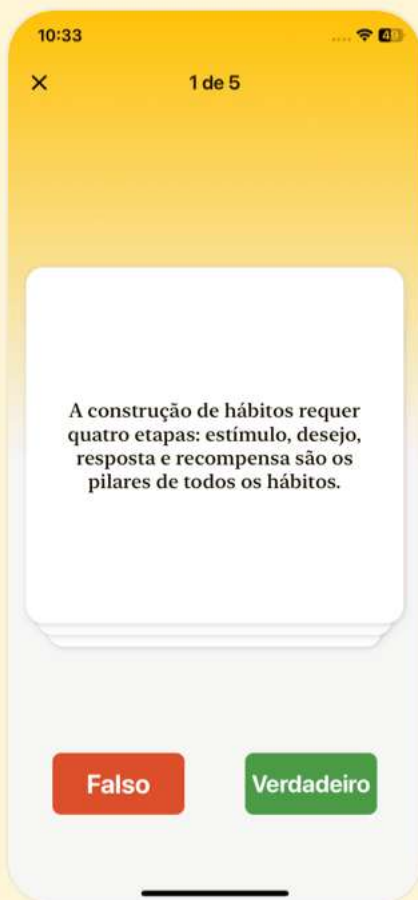


Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Escanear para baixar



Chapter 13 | Component Cohesion| Quiz e teste

- 1.The Reuse/Release Equivalence Principle states that the granule of reuse is the granule of release.
- 2.The Common Reuse Principle advises putting all classes within a single component, regardless of dependency.
- 3.The Common Closure Principle suggests that classes that change together for the same reasons should be gathered in the same component.

Chapter 14 | Component Coupling| Quiz e teste

- 1.The Acyclic Dependencies Principle (ADP) states that there should be cycles in the component dependency graph.
- 2.Components should be designed to depend in the direction of volatility rather than stability.
- 3.A component should be as abstract as it is stable according to the Stable Abstractions Principle.

Chapter 15 | What Is Architecture?| Quiz e teste

- 1.A software architect is merely a project manager who does not engage in programming tasks.



2. Good software architecture includes the clear definition of components and their effective communication.
3. Maintaining a software system is considered to be the least expensive aspect of its lifecycle.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

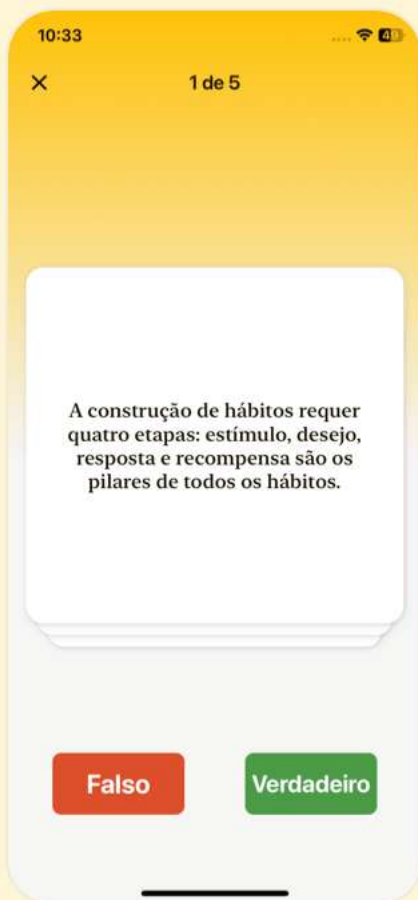


Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Escanear para baixar



Chapter 16 | Independence| Quiz e teste

1. A good architecture must support use cases and operational support among other goals.
2. Conway's Law states that a system's design does not reflect a team's communication structure.
3. Decoupling components helps reduce inter-team interference and allows for independent deployment.

Chapter 17 | Boundaries: Drawing Lines| Quiz e teste

1. Software architecture should avoid drawing boundaries between components to allow free interaction among them.
2. A plugin architecture allows for the replacement of optional components without affecting the core functions of a system.
3. Premature architectural decisions can simplify the development process and lead to better system performance.

Chapter 18 | Boundary Anatomy| Quiz e teste



1. The architecture of a system consists solely of software components and does not include any boundaries separating them.
2. Boundary crossings in system architecture are significant because they involve transferring data across different modules, which may lead to changes or recompilations in those modules.
3. Threads serve as architectural boundaries by themselves, allowing for independent execution within a system.



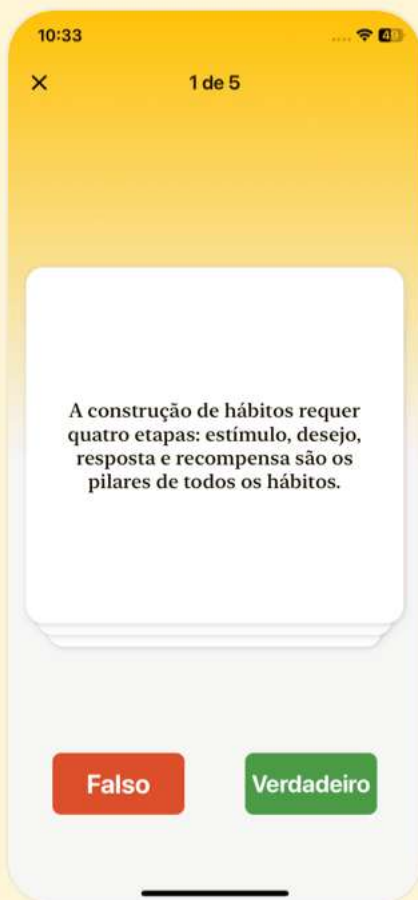


Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Escanear para baixar



Chapter 19 | Policy and Level| Quiz e teste

1. Software systems are fundamentally expressions of policy that describe how inputs are transformed into outputs.
2. Higher-level policies are closer to inputs and outputs and change more frequently than lower-level policies.
3. Separating higher-level policies from lower-level details increases the impact of changes in input/output policies.

Chapter 20 | Business Rules| Quiz e teste

1. Business rules can only be applicable and meaningful when they are automated.
2. Entities in Clean Architecture encapsulate critical business rules and data while remaining unaffected by external concerns.
3. Use cases need to rely heavily on user interface specifics to be effective in execution order and accessibility.

Chapter 21 | Screaming Architecture| Quiz e teste

1. Architecture serves the same purpose as blueprints of a building, easily signifying the intended usage



of a software application.

2. Good architecture is solely about choosing the right frameworks for implementation.

3. Frameworks should dictate architecture decisions as they are essential to building the application.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

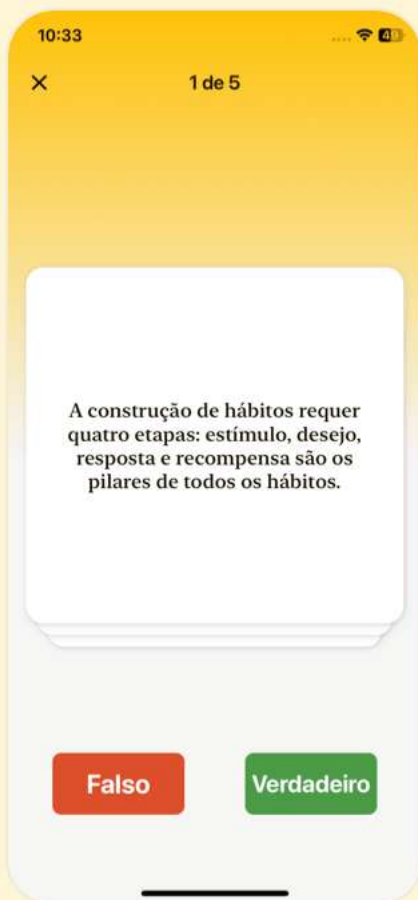


Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Escanear para baixar



Chapter 22 | The Clean Architecture| Quiz e teste

1. Layers in Clean Architecture are designed to be independent from frameworks, allowing them to be treated as tools.
2. In Clean Architecture, the entity layer changes frequently based on operational context.
3. The Dependency Rule states that dependencies must point outward towards lower-level details.

Chapter 23 | Presenters and Humble Objects| Quiz e teste

1. The Humble Object pattern divides behaviors into testable and untestable classes.
2. The Presenter processes data while the View is responsible for formatting it.
3. Testability is not essential for good architecture according to the Humble Object pattern.

Chapter 24 | Partial Boundaries| Quiz e teste

1. Full-fledged architectural boundaries are always necessary for effective design and maintenance.



2. Using the Facade pattern sacrifices some dependency inversion but helps in defining architectural boundaries.
3. Creating a partial boundary allows components to be deployed as a single unit to save on administrative overhead.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

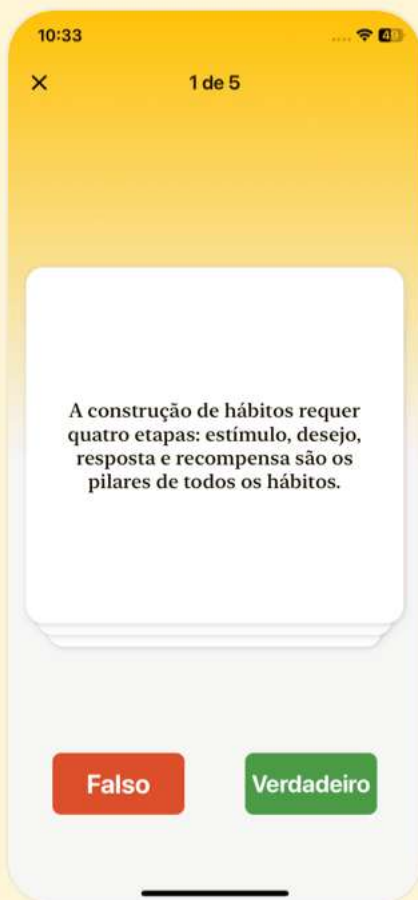


Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Escanear para baixar



Chapter 25 | Layers and Boundaries| Quiz e teste

1. Systems can usually be simplified into three components: User Interface (UI), Business Rules, and Database.
2. The Hunt the Wumpus example demonstrates that the Game Rules component must be tightly coupled with UI for effective functionality.
3. Effective architecture requires identifying boundaries and ensuring that higher-level components are dependent on lower-level ones.

Chapter 26 | The Main Component| Quiz e teste

1. The Main component is considered the 'dirtiest' part of a software system because it manages dependencies and orchestrates the system without exposing high-level functionality.
2. The Main component should handle high-level functionalities directly to ensure clean architecture.
3. In a software system, the Main component does not play a significant role in maintaining cleanliness and organization.



Chapter 27 | Services: Great and Small| Quiz e teste

1. Service-oriented and micro-service architectures inherently define an architecture on their own.
2. Services can be tightly coupled through shared data and interfaces despite running in different processes.
3. Proper management of components allows for better separation of concerns in service architecture.



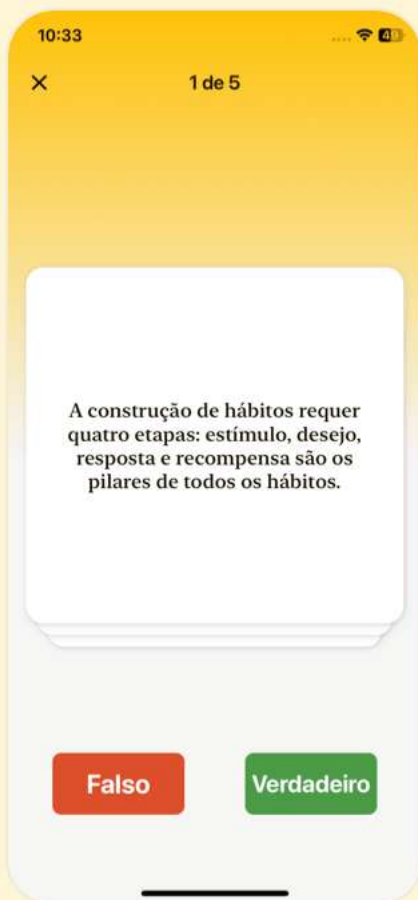


Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Escanear para baixar



Chapter 28 | The Test Boundary| Quiz e teste

1. Tests are considered integral components of the architecture of a system, just like any other part.
2. Designing tests to be isolated from system design is beneficial as it leads to greater flexibility.
3. A specialized Testing API can improve test independence by allowing tests to bypass certain application constraints.

Chapter 29 | Clean Embedded Architecture| Quiz e teste

1. Firmware is defined as software that can become obsolete due to hardware dependencies.
2. Clean architecture principles suggest that embedded software should be tightly coupled with the operating system for better functionality.
3. The Hardware Abstraction Layer (HAL) is introduced to increase dependencies that can complicate embedded software.

Chapter 30 | The Database Is a Detail| Quiz e teste

1. The database should be considered a fundamental



part of software architecture.

2.Relational databases impact the architectural decisions of software applications significantly.

3.Performance considerations should influence the overall architecture of software.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

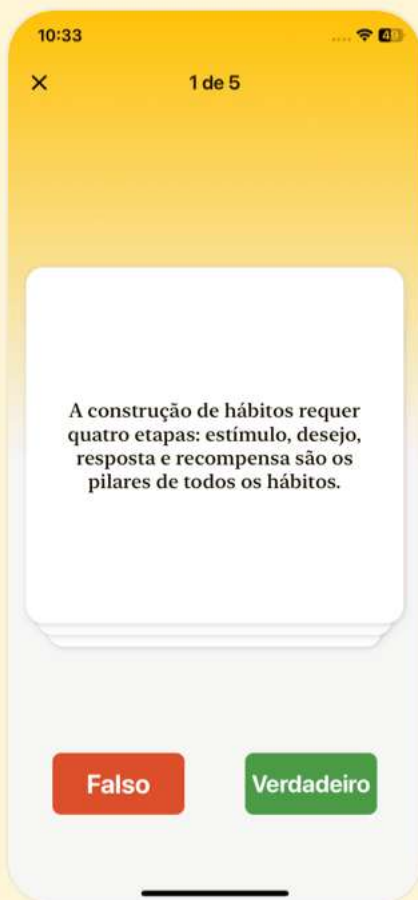


Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Escanear para baixar



Chapter 31 | The Web Is a Detail| Quiz e teste

- 1.The web represents a fundamental shift in technology rather than just a detail in system architecture.
- 2.Decoupling business logic from GUIs is essential to protect the architecture from marketing influences.
- 3.Device independence should not be a concern for architects when designing systems that utilize web interfaces.

Chapter 32 | Frameworks Are Details| Quiz e teste

- 1.Frameworks are equivalent to software architecture.
- 2.A commitment to a framework is typically one-sided, with users investing heavily and receiving no guarantees.
- 3.It is advisable to tightly couple your application to a framework for better interaction.

Chapter 33 | Case Study: Video Sales| Quiz e teste

- 1.The architecture of the video sales case study allows changes in low-level components to impact higher-level policies.



2. Identifying actors and their use cases is essential to ensure that changes for one actor won't affect others in the video sales platform.
3. The component architecture in the case study suggests that each component corresponds to multiple actors to maximize efficiency.

Mais Livros Grátis



Escaneie para baixar



Ouça-o

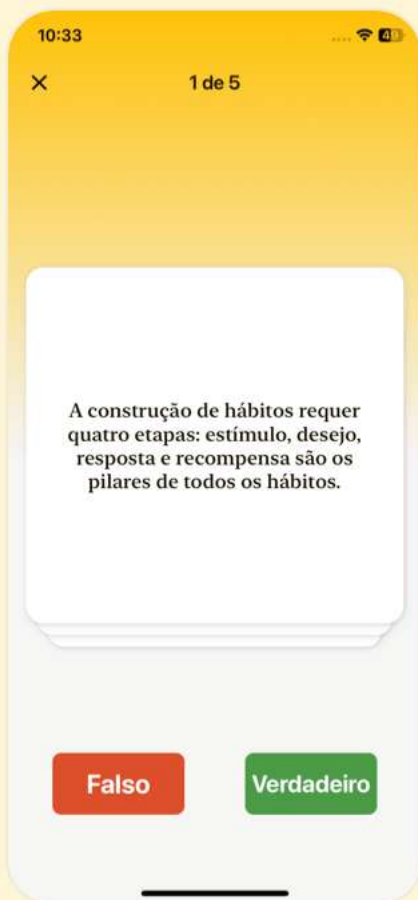


Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Escanear para baixar



Chapter 34 | The Missing Chapter| Quiz e teste

- 1.The chapter argues that the traditional 'Package by Layer' approach is always the best choice for software design.
- 2.'Package by Feature' provides better visibility into business context compared to 'Package by Layer'.
- 3.Proper implementation practices, such as strictly controlling access modifiers, can help maintain architectural integrity.

Chapter 35 | Appendix A Architecture Archaeology| Quiz e teste

- 1.The chapter discusses the development of a real-time monitoring system for aluminum die-cast using IBM's System/7 without any libraries or operating systems.
- 2.Decoupling architecture improves response times and memory efficiency, as illustrated by the transition of the 4-TEL system.
- 3.The chapter suggests that software architecture does not



need to adapt to specific requirements and environments.

Mais Livros Grátis



Escaneie para baixar



Ouça-o



Baixe o app Bookey para desfrutar

Mais de 1000 resumos de livros com quizzes

Teste grátis disponível!

Escanear para baixar

